

SILAB[®]

Version 7.1 Referenz SILAB-Konfiguration und Bedienung

INHALTSVERZEICHNIS

1	EINLEITUNG	5
1.1	Über dieses Handbuch	5
1.2	Aufbau dieser Anleitung	5
2	KONFIGURATION VON SILAB	6
2.1	Einführung	6
2.1.1	Data Processing Unit (DPU)	6
2.1.2	Erstellen einer DPU-Konfiguration	10
2.1.3	Rechnernetze	12
2.1.4	DPU-Pools	15
2.1.5	Computer-Pools und Aliasnamen	19
2.1.6	Gesamte Beispielkonfiguration	22
2.1.7	Weitere Möglichkeiten	24
2.2	Allgemeines	29
2.2.1	Editieren und Überprüfen von Konfigurationsdateien	29
2.2.2	Notation	30
2.2.3	include	32
2.2.4	Kommentare	33
2.2.5	Dateibezeichnung	33
2.3	Grundlegender Aufbau einer Konfiguration	34
2.4	SILAB System Abschnitt	34
2.4.1	LaunchProcess im Systemabschnitt	34
2.4.2	Launchdaten im Systemabschnitt	35
2.4.3	Watchdogparameter im Systemabschnitt	37
2.4.4	Unterdrückung / Aktivierung von Logausgaben	39
2.4.5	Systemsteuerungsparameter	39
2.5	Rechner- und DPU-Konfiguration	40
2.6	Computer-Konfiguration	41
2.7	DPU-Konfiguration	45
2.7.1	Instanziierung von DPUs, Zuweisung von Rechnern, Berechnungsreihenfolge	45
2.7.2	Verknüpfungen von DPUs	47
2.7.3	Variableninitialisierung	49
2.7.4	Beispiel:	49
2.8	Mutithreading	56
2.8.1	Überblick	56
2.8.2	Definition von Threads und Zuweisung von DPUs	57
2.8.3	Anforderungen und Einschränkungen	58

3	STANDARD-KONFIGURATIONSARCHITEKTUR	58
3.1	Überblick und Verwendung	58
3.2	Vordefinierte Rechnergruppen	61
3.3	Interfaces	62
3.4	Dateien	63
4	SYSTEMKONFIGURATION	65
5	WEITERE ELEMENTE DER BENUTZEROBERFLÄCHE VON SILAB	69
5.1	Anpassen der Baumansicht	69
5.2	Tooltips in der Baumansicht	72
5.3	Zwischenablage	73
5.4	Displays	73
5.4.1	Displaytypen	73
5.4.2	Display „ValueEdit“	73
5.4.3	Display „Oszi“	74
5.4.4	Display „Text“	80
5.4.5	Display „Wertetabelle“	80
5.4.6	Display „Schalter“	81
5.4.7	Display „Slider“	83
5.4.8	Display „Status“	83
5.5	Anzeige der Applikation auf Remote-Rechnern	86
6	SILABADMIN	87
6.1	Überblick	87
6.2	Konfiguration	87
6.3	Hauptfenster	88
6.4	Menübefehle	89
6.4.1	Aktionen/System-Informationen ... 	90
6.4.2	Aktionen/Update Verzeichnis ... 	90
6.4.3	Aktionen/Update Dateien ... 	90
6.4.4	Aktionen/Wake up ... 	90
6.4.5	Aktionen/Reboot ... 	90
6.4.6	Aktionen/Shutdown ... 	91
7	SILABEXPLORER	92

7.1	Überblick	92
7.2	Arbeiten mit SILABExplorer	93

1 EINLEITUNG

1.1 Über dieses Handbuch

Dieses Handbuch richtet sich an Benutzer von *SILAB professional* und *SILAB enterprise*. Es beschreibt detailliert

- die Architektur und Funktionsweise von SILAB.
- die Möglichkeiten, die Funktionalität von SILAB durch Erstellen entsprechender Konfigurationsdateien anzupassen und zu erweitern.
- Elemente der grafischen Benutzeroberfläche, die zur Steuerung und Kontrolle einer Simulation mit SILAB verwendet wird.
- das Tool SILABAdmin, mit dem das Rechnernetz eines Simulators verwaltet werden kann.
- das Tool SILABExplorer, mit dem SILAB über ein Netzwerk überwacht und gesteuert werden kann.

Das Handbuch setzt voraus, dass der Leser mit der grundlegenden Bedienung von SILAB (wie im Handbuch „Handbuch SILAB Essential“ beschrieben) vertraut ist.

1.2 Aufbau dieser Anleitung

Die Basis für Anpassungen und Erweiterungen von SILAB bilden sog. Konfigurationsdateien. Der grundsätzliche Aufbau solcher Konfigurationsdateien wird in Abschnitt 2 detailliert beschrieben.

In der Regel ändert sich die grundlegende Funktionalität eines Fahrsimulators nicht häufig. Deswegen befinden sich im Lieferumfang von SILAB bereits vorgefertigte Konfigurationsdateien, in denen diese Funktionalität implementiert wird. Aufbauend auf diesen Standard-Konfigurationsdateien kann der Anwender SILAB für seine Fragestellungen anpassen. Die Arbeit mit der sog. Standard-Konfigurationsarchitektur ist in Abschnitt 0 dargestellt.

Die Möglichkeiten der grafischen Benutzeroberfläche zur Steuerung Kontrolle eines Simulationslaufs werden in Abschnitt 5 beschrieben.

Das Programm „SILABAdmin“, mit dem das Rechnernetz eines Fahrsimulators verwaltet werden kann und Updates von SILAB installiert werden, wird in Abschnitt 6 beschrieben.

Kapitel 7 beschäftigt sich schließlich mit dem Programm „SILABExplorer“, über den eine Simulation unter SILAB über das Netzwerk ferngesteuert werden kann.

2 KONFIGURATION VON SILAB

2.1 Einführung

2.1.1 Data Processing Unit (DPU)

Eine Fahrsimulation unter SILAB besteht aus mehreren Softwarekomponenten, von denen jede eine spezielle Aufgabe übernimmt. Für eine einfache Fahrsimulation sind diese Komponenten in Abbildung 1 dargestellt.

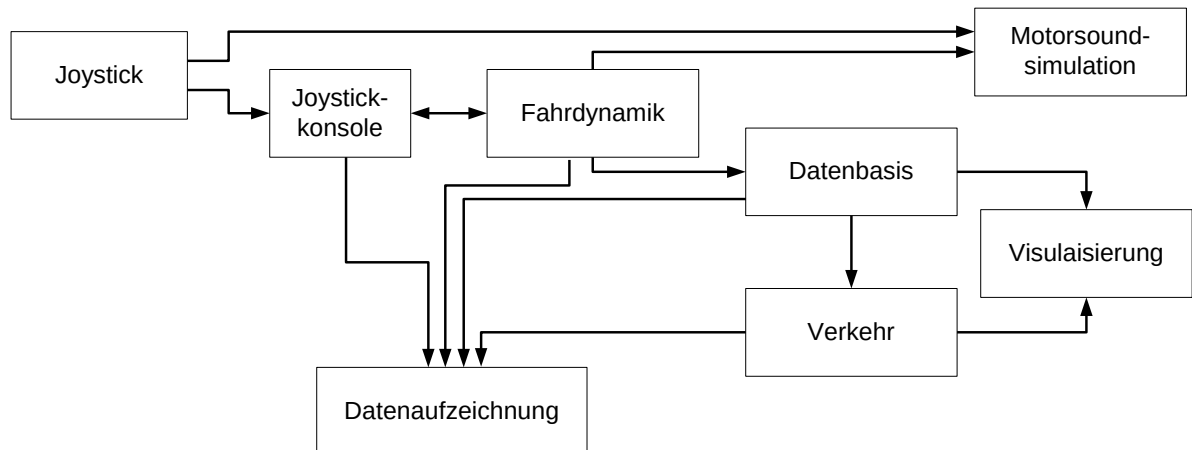


Abbildung 1: Softwarekomponenten einer Fahrsimulation

Die dargestellten Komponenten haben folgende Aufgaben:

- **Joystick:** Übernimmt die Abfrage eines handelsüblichen Spiele-Joysticks oder Gamepads
- **Joystick-Konsole:** Rechnet die vom Joystick gelieferten Werte in Gaspedal, Bremse und Lenkung um
- **Fahrdynamik:** Berechnet anhand eines physikalischen Modells eines Fahrzeugs aus Gaspedal, Bremse und Lenkung, die aktuelle Geschwindigkeit und die aktuelle Position des Fahrzeugs.
- **Motorsoundsimulation:** Simuliert je nach Motordrehzahl, Gaspedalstellung und Geschwindigkeit die Motorgeräusche eines Fahrzeugs.
- **Datenbasis:** Enthält Informationen über das Straßennetzwerk und die Landschaft, in der sich das Fahrzeug bewegt.
- **Verkehr:** Simuliert das Verhalten anderer Verkehrsteilnehmer.
- **Visualisierung:** Berechnet ein 3D-Bild der Szenerie aus der Sicht des Fahrers.
- **Datenaufzeichnung:** Schreibt die für eine spätere Auswertung einer Fahrt (z.B. in einem Statistik-Programm) interessanten Werte (z.B. Geschwindigkeit, Güte der Spurhaltung usw.) in jedem Simulationsschritt in eine Datei.

Während der Simulation kommunizieren diese Komponenten. In jedem Schritt berechnet jede Komponente anhand von Eingabedaten, die sie von anderen Komponenten bezieht, Ausgabewerte. Diese Ausgabewerte stehen im nächsten Simulationsschritt den restlichen Komponenten wiederum als Eingaben zur Verfügung.

Die angesprochenen Softwarekomponenten werden in SILAB „**DPUs**“ (**Data Processing Units**) genannt. In Abbildung 2 ist als Beispiel die DPUJoystickConsole dargestellt. Sie besitzt verschiedene Eingänge, die mit den Ausgängen der Joystick-DPU verbunden werden müssen. Die DPUJoystickConsole stellt eine Schnittstelle dar: Sie rechnet ihre Eingänge, mit denen der Joystick verbunden ist, so um, dass an ihren Ausgängen Signale wie Gaspedalstellung, Bremspedalstellung und Lenkwinkel zur Verfügung stehen. Diese Signale werden von der physikalischen Fahrzeugsimulation (Softwaremodul „Fahrdynamik“) benötigt. Beispielsweise wird die Bewegung des Joysticks entlang seiner y-Achse in zwei Teile aufgespaltet: Ein Drücken des Joysticks nach vorne wird zu einer Betätigung des Gaspedals, eine Bewegung nach hinten wird als Betätigen der Bremse interpretiert.

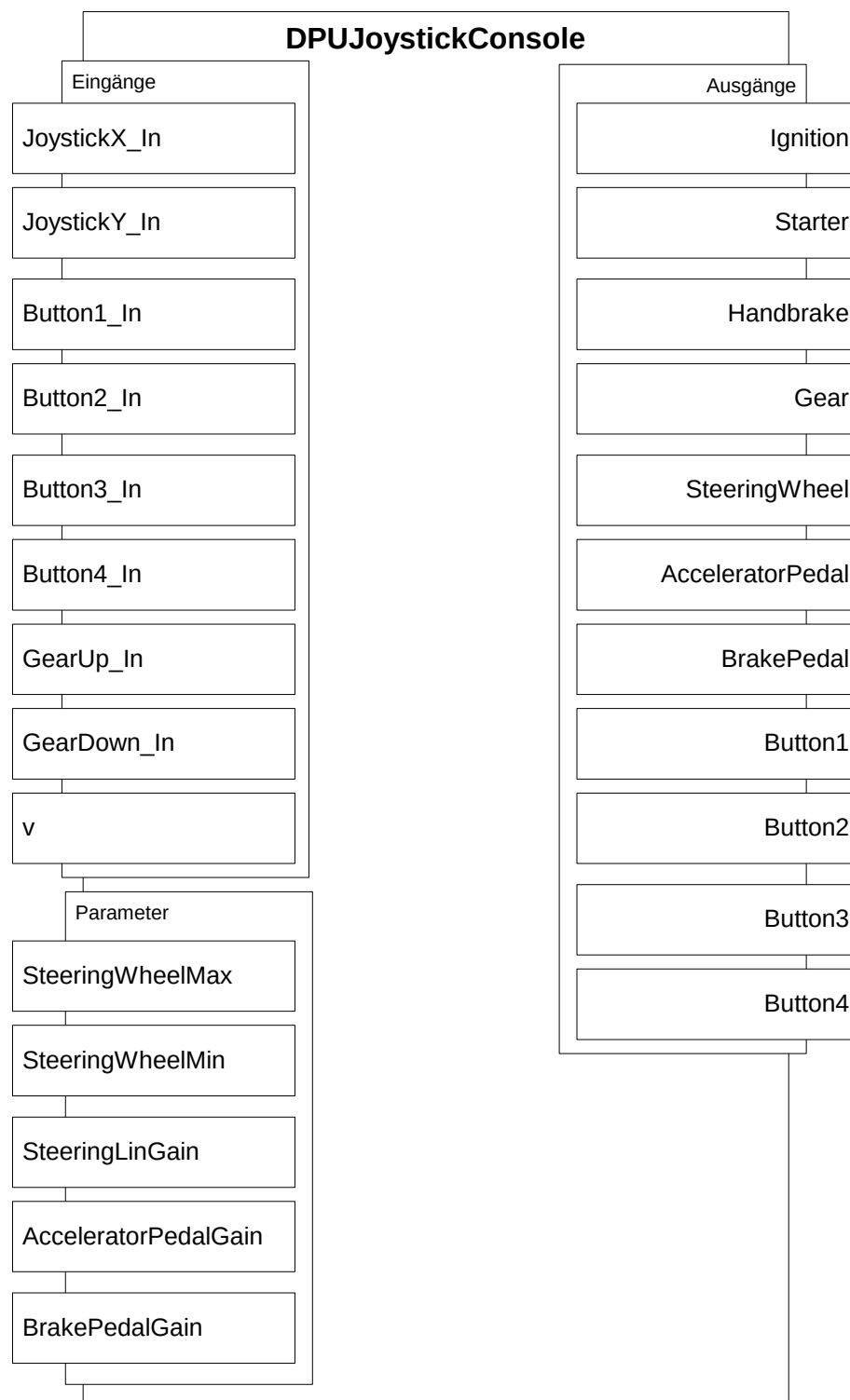


Abbildung 2: Aufbau der DPUJoystickConsole

Zusätzlich zu den Ein- und Ausgängen besitzt eine DPU verschiedene Parameter. Diese Parameter werden grundsätzlich in **statische** und **dynamische Parameter**

unterteilt. Dynamische Parameter verhalten sich so wie Eingänge, d.h. während der Simulation können diese Parameter online verändert werden und haben auch eine sofortige Auswirkung. Statische Parameter werden nur beim Start der Simulation abgefragt und eine Änderung während der Simulation hat keine Auswirkungen mehr.

Neben den speziellen, auf die Fahrsimulation zugeschnittenen DPUs, gibt es zahlreiche **allgemeine DPU**s, die – je nach Verschaltung ihrer Ein- und Ausgänge – verschiedenste Aufgaben erfüllen können. Ein einfaches Beispiel ist die DPUAmplifier (s. Abbildung 3).

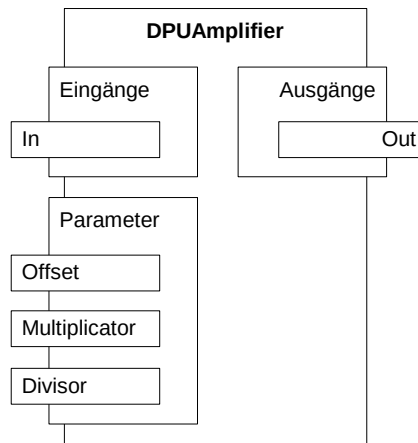


Abbildung 3: DPUAmplifier

In jedem Schritt der Simulation führt sie folgende Berechnung durch:

$$Out = \frac{In \cdot Multiplicator}{Divisor} + Offset$$

Wird ihr Eingang *In* beispielsweise mit einem Geschwindigkeitssignal in m/s verbunden, so liegt die Geschwindigkeit durch die Parameterwahl

Multiplicator = 1

Divisor = 3.6

Offset = 0

an ihrem Ausgang *Out* in der Einheit km/h an.

Weitere Beispiele solcher universellen DPUs sind:

- DPUBooleanGate (Berechnung logischer Verknüpfungen)
- DPUComparator (Vergleich von Signalen)
- DPUMultiplexer, DPUDemultiplexer
- DPUFader (Aus-/Einblenden von Signalen)
- DPULimiter (Begrenzung von Signalen)
- DPMixAmplifier (Mischen von Signalen)
- DPURandom (Generierung von Zufallswerten)
- DPULookupTable (2D-Lookup Tabellen mit linearer Interpolation)

Jede DPU, insbesondere eine der universellen DPUs, kann in einer Simulation mehrfach eingesetzt werden und das jeweils mit unterschiedlichen Parametereinstellungen. Im Folgenden wird für diesen Sachverhalt der Begriff der **Instanziierung einer DPU** (auch als „Instanziierung eines DPU-Typs“ bezeichnet) verwendet. Eine Instanz einer DPU bekommt einen eindeutigen Namen und eigene Parametereinstellungen. Verknüpft werden immer die Ein- und Ausgänge von Instanzen. Beispielsweise könnte eine Instanz der DPUAmplifier namens „ms2kmh“ obige Einheitenumrechnung durchführen. Eine weitere Instanz von DPUAmplifier namens „xyz“ übernimmt eine völlig andere Aufgabe.

Grundsätzlich kann jeder Ausgang einer DPU-Instanz mit den Eingängen/dynamischen Parametern einer oder mehrerer anderer DPU-Instanzen verschaltet werden. Die Menge aller DPU-Instanzen und deren Verschaltung wird „**DPU-Konfiguration**“ genannt.

Die Vorgehensweise beim Erstellen einer DPU-Konfiguration gleicht der beim Bau elektronischer Schaltungen. Einzelne Bauteile (entspricht DPU-Instanzen) mit bestimmten Eigenschaften (z.B. Widerstandswert) (entspricht Parametereinstellungen) eines bestimmten Typs (z.B. Widerstand, Transistor) (entspricht DPU-Typ) werden verdrahtet (entspricht Verschaltung von DPU Ein- und Ausgängen).

Funktionsweise, Ein-/Ausgänge und Parameter aller DPUs sind in gesonderten Dokumenten beschrieben. Erläuterungen zu einer DPUXXX finden sich im Dokument DPUXXX.pdf (z.B. DPUAmplifier.pdf).

Ist eine Aufgabe durch Verschaltung der im Lieferumfang enthaltenen DPUs nicht lösbar, kann SILAB mit Microsoft Visual C++ , Matlab/Simulink, JAVA oder der Skript-Sprache Ruby um DPUs erweitert werden (vgl. „Handbuch SILAB Enterprise.pdf“).

2.1.2 Erstellen einer DPU-Konfiguration

Eine DPU-Konfiguration wird als Textdatei¹ in einer Script-Sprache erstellt. Nach dem Start von „SILAB.exe“ wird eine solche Textdatei geöffnet. SILAB durchsucht die Datei nach den in ihr aufgeführten DPU-Instanzen und sorgt dafür, dass die DPU-Instanzen gemäß der Verschaltung ihre Eingabedaten erhalten.

Eine detaillierte Referenz dieser Sprache findet sich ab Abschnitt 2.3 dieses Dokuments. Im Folgenden wird nur ein Überblick über die wichtigsten Teile einer DPU-Konfiguration gegeben.

Als Beispiel soll eine DPU-Konfiguration erstellt werden, die zwei Sinus-Signale unterschiedlicher Frequenz überlagert². Das Signal mit der niedrigeren Frequenz soll gegenüber dem mit der höheren Frequenz verstärkt werden. Die benötigten DPU-Typen sind:

¹ genau genommen sind DPU-Konfigurationen nur ein Teil einer solchen Textdatei. Um sie in SILAB öffnen zu können, werden noch mehr Informationen benötigt. Hierauf wird später eingegangen

² Eine beispielhafte DPU-Konfiguration für eine Fahrsimulation wird in Abschnitt 2.7 beschrieben. In diesem Kapitel sollen DPU-Konfigurationen der Einfachheit halber anhand eines sehr primitiven Beispiels dargestellt werden.

- DPUOscillator: Signalgenerator
- DPUMixAmplifier: Mischer/Verstärker

Das folgende Diagramm zeigt die DPU-Konfiguration im Überblick. Es sind nur die für die Aufgabe relevanten Ein-/Ausgänge und Parameter der DPUs gezeigt:

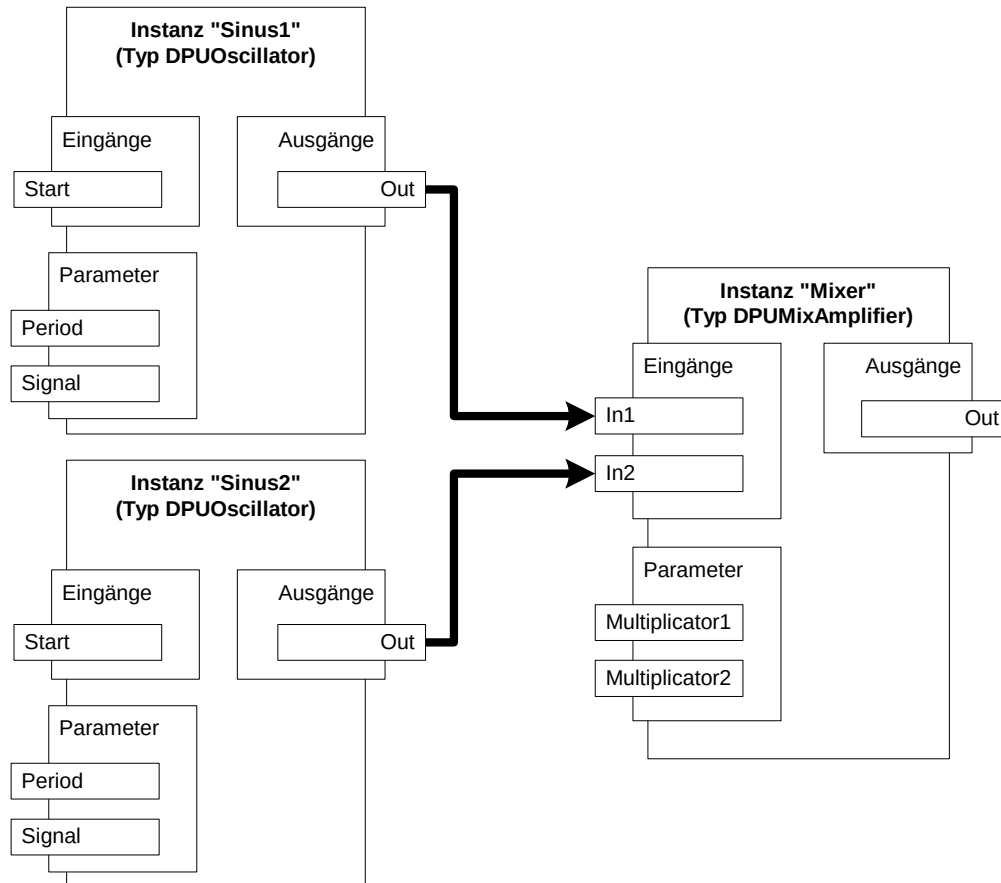


Abbildung 4: Beispiel einer DPU-Konfiguration

Es gibt zwei Instanzen der DPUOscillator, „Sinus1“ (erzeugt Sinus mit niedriger Frequenz) und „Sinus2“ (Sinus mit hoher Frequenz). Die Ausgänge der beiden Instanzen werden mit den Eingängen der Instanz „Mixer“ der DPUMixAmplifier verknüpft. Das Ergebnis liegt am Ausgang von „Mixer“ an.

In der Script-Sprache wird dieser Sachverhalt so formuliert:

```
DPUOscillator Sinus1
{
    Period = 1000;
    Signal = 0;
};

DPUOscillator Sinus2
{
    Period = 200;
    Signal = 0;
};

DPU MixAmplifier Mixer
{
    Multiplier1 = 3.0;
    Multiplier2 = 1.0;
};

Connections =
{
    Sinus1.Out -> Mixer.In1,
    Sinus2.Out -> Mixer.In2
};
```

Obiges Beispiel zeigt, dass die Instanz einer DPU durch die Syntax

```
<DPUTyp> <Instanzname>
```

erzeugt wird. Es folgt ein durch { und }; umschlossener Block, in dem Werte für die Parameter und nicht verschalteten Eingänge der DPU-Instanz definiert werden. Beispielsweise wird in der Instanz `Sinus1` die Periode des Signals auf 1000 ms und der Signaltyp auf „0“ (= Sinus) gesetzt.

Die Verschaltung wird durch `Connections =` eingeleitet. Es folgt eine durch { und }; umschlossene, durch Kommata getrennte Liste von Verbindungen in beliebiger Reihenfolge. Eine Verbindung hat die Syntax

```
<Instanzname>.<Ausgang> -> <Instanzname>.<Eingang>
```

2.1.3 Rechnernetze

In komplexen Simulatoren werden bestimmte Aufgaben von Spezialrechnern übernommen. Beispielsweise gibt es für jeden Bildkanal einen speziell auf Grafikleistung abgestimmten Rechner. Bei 180° Blickwinkel nach vorne (entspricht drei Bildkanälen à 60°) und drei Rückspiegeln sind an der Simulation üblicherweise 6 Grafikrechner beteiligt. Deswegen können Simulatoren unter SILAB aus einem oder mehreren Rechnern bestehen.

Damit SILAB die Aufgaben, d.h. die entsprechenden DPU-Instanzen, auf die richtigen Rechner verteilen kann, müssen die beteiligten Rechner zunächst identifiziert werden. Das geschieht in einem gesonderten Abschnitt der Konfigurationsdatei, in

der auch – wie im vorhergehenden Kapitel beschrieben – die DPU-Konfiguration festgelegt wird.

Als Beispiel soll die DPU-Konfiguration aus dem vorhergehenden Kapitel auf zwei Rechnern laufen. Diese Rechner haben im Betriebssystem die Namen „COMP1“ und „COMP2“³. In der Textdatei werden die Rechner mit folgender Syntax aufgeführt:

```
Computer COMP1
{
    PeriodMS = 10;
    IP = „10.1.2.21“;
    Operator = true;
};

Computer COMP2
{
    PeriodMS = 10;
    IP = „10.1.2.45“;
};
```

In dem mit { und }; umschlossenen Block werden einige Parameter für den Rechner festgelegt:

- **PeriodMS** : Periode in Millisekunden, mit der ein Rechner seine DPUs berechnet. In obigem Beispiel laufen beide Rechner mit einer Frequenz von 100 Hz.
- **IP** : IP-Adresse des Rechners
- **Operator** : Wird dieser Parameter auf „true“ gesetzt, dann wird die Simulation von diesem Rechner aus bedient (Operator-Rechner). Das bedeutet, dass „SILAB.exe“ auf diesem Rechner gestartet wird. In einer Simulation kann es nur einen Operator-Rechner geben. Im Beispiel ist dies „COMP1“.

Jetzt muss noch festgelegt werden, welcher Rechner welche DPUs berechnet. Dies erfolgt für jede DPU-Instanz in dem Abschnitt, in welchem auch die Parameter der DPU-Instanz festgelegt werden. Die DPU-Konfiguration aus dem vorhergehenden Abschnitt wird also wie folgt erweitert (fett):

³ Rechnernamen bestehen innerhalb des SILAB-Systems immer aus Großbuchstaben.

```
DPUOscillator Sinus1
{
    Computer = {COMP1};
    Period = 1000;
    Signal = 0;
};

DPUOscillator Sinus2
{
    Computer = {COMP2};
    Period = 200;
    Signal = 0;
};

DPU MixAmplifier Mixer
{
    Computer = {COMP1};
    Multiplicator1 = 3.0;
    Multiplicator2 = 1.0;
};

Connections =
{
    Sinus1.Out -> Mixer.In1,
    Sinus2.Out -> Mixer.In2
};
```

COMP1 berechnet den tieffrequenten Sinus-Generator und den Mixer, COMP2 übernimmt die Berechnung des hochfrequenten Sinus-Generators.

Alle DPU-Instanzen, die auf einem bestimmten Rechner laufen, werden mit der unter `PeriodMS` angegebenen Periode aktualisiert.

Sollen die DPUs eines Rechners in einer bestimmten Reihenfolge aktualisiert werden, muss dies explizit festgelegt werden. Im Beispiel muss in jedem Simulationsschritt die Berechnung des Mixers nach der von `Sinus1` erfolgen, weil der Mixer Daten von `Sinus1` bezieht⁴. Zur Festlegung der Reihenfolge wird die DPU-Konfiguration erweitert (fett):

⁴ Wird die Berechnung von `Mixer` vor der von `Sinus1` durchgeführt, erhält `Mixer` die Eingangsdaten um einen Simulationsschritt verzögert.

```
DPUOscillator Sinus1
{
    Computer = {COMP1};
    Index = 10;
    Period = 1000;
    Signal = 0;
};

DPUOscillator Sinus2
{
    Computer = {COMP2};
    Period = 200;
    Signal = 0;
};

DPUMixAmplifier Mixer
{
    Computer = {COMP1};
    Index = 20;
    Multiplicator1 = 3.0;
    Multiplicator2 = 1.0;
};

Connections =
{
    Sinus1.Out -> Mixer.In1,
    Sinus2.Out -> Mixer.In2
};
```

Wird kein Parameter `Index` angegeben, ist die Berechnungsreihenfolge undefiniert.

Der Datenaustausch zwischen den Rechnern wird von SILAB automatisch anhand der DPU-Verknüpfungen geregelt.

2.1.4 DPU-Pools

In der Praxis steht man oft vor ähnlichen Situationen wie der folgenden: DPU-Konfigurationen für eine Fahrsimulation unterscheiden sich nur in wenigen DPUs (z.B. Assistenzsysteme, Nebenaufgaben für den Fahrer). Ein großer Teil der DPUs und Verknüpfungen, die eine Art Grundkonfiguration definieren, bleibt gleich (Mockup, Fahrdynamik, Datenbasis, Verkehr, usw.). Deswegen wäre es praktisch, wenn man eine bestehende Grundkonfiguration verwenden und sie leicht um einige zusätzliche DPUs erweitern könnte. Auf den Mechanismus von SILAB, mit dem dies möglich ist, wird im Folgenden eingegangen.

Wie in den vorhergehenden Kapiteln beschrieben, besteht eine DPU-Konfiguration aus einer Menge von DPU-Instanzen und Verknüpfungen zwischen diesen Instanzen. Eine solche Konfiguration wird zu einem sog. DPU-Pool zusammengefasst. Für obiges Beispiel sieht das so aus:

```

Pool Base
{
    DPUOscillator Sinus1
    {
        Computer = {COMP1};
        Index = 10;
        Period = 1000;
        Signal = 0;
    };

    DPUOscillator Sinus2
    {
        Computer = {COMP2};
        Period = 200;
        Signal = 0;
    };

    DPUMixAmplifier Mixer
    {
        Computer = {COMP1};
        Index = 20;
        Multiplicator1 = 3.0;
        Multiplicator2 = 1.0;
    };

    Connections =
    {
        Sinus1.Out -> Mixer.In1,
        Sinus2.Out -> Mixer.In2
    };
};

```

Jeder Pool bekommt einen Namen (hier: `Base`). In einer Konfigurationsdatei können beliebig viele Pools stehen. Dies hat dann folgende Struktur:

```

Pool FirstPool
{
    <DPUs, Connections >
};
Pool AnotherPool
{
    <DPUs, Connections>
};
...
Pool LastPool
{
    <DPUs, Connections>
};

```

Beim Öffnen dieser Konfiguration in SILAB erscheint eine Auswahl-Box mit allen in der Datei enthaltenen Pools, von denen einer dann für die Simulation selektiert werden muss.

Anhand des Beispiels in diesem Kapitel lässt sich das eingangs erwähnte Problem so nachbilden: Ein neuer DPU-Pool `Extension` soll die bereits erstellte DPU-Konfiguration des Pools `Base` im Wesentlichen beibehalten. Er soll sich aber in zwei Punkten unterscheiden:

1. Die Periode der DPU-Instanz `Sinus1` soll verdoppelt werden.

2. Das Ausgangssignal von `Sinus2` soll mit einem Offset von +1 versehen werden.

Folgende Konfiguration löst das Problem:

```
Pool Base
{
    Executable = true;

    DPUOscillator Sinus1
    {
        Computer = {COMP1};
        Index = 10;
        Period = 1000;
        Signal = 0;
    };

    DPUOscillator Sinus2
    {
        Computer = {COMP2};
        Period = 200;
        Signal = 0;
    };

    DPUMixAmplifier Mixer
    {
        Computer = {COMP1};
        Index = 20;
        Multiplicator1 = 3.0;
        Multiplicator2 = 1.0;
    };

    Connections =
    {
        Sinus1.Out -> Mixer.In1,
        Sinus2.Out -> Mixer.In2
    };
};

Pool Extension : Base
{
    Executable = 1;

    Sinus1.Period = 2000;

    DPUAmplifier Offset
    {
        Computer = {COMP1};
        Index = 15;
        Offset = 1.0;
    };

    Connections =
    {
        Sinus2.Out -> Offset.In,
        Offset.Out -> Mixer.In
    };
};
```

Der neue Pool `Extension` wird vom alten Pool `Base` abgeleitet⁵. Die Syntax hierfür ist im Beispiel oben fett gedruckt. Ein abgeleiteter Pool (`Extension`) „erbt“ alle DPU-Instanzen und Verknüpfungen seines Vorgänger-Pools (`Base`). Dabei ist wichtig, dass der Vorgänger-Pool in der Konfiguration vor (d.h. in der Konfigurationsdatei *oberhalb*) dem abgeleiteten Pool definiert wird.

In einem abgeleiteten Pool können die Parameter (und nicht verknüpften Eingänge) von DPU-Instanzen des Vorgänger-Pools neu gesetzt werden. Im Beispiel soll die Periode von `Sinus1` im abgeleiteten Pool verdoppelt werden. Die Syntax hierfür ist rot oben hervorgehoben. Sie lautet:

```
<Name DPU-Instanz>.<Name Parameter> = <Wert>;
```

Wie in jedem DPU-Pool können in abgeleiteten Pools DPU-Instanzen erzeugt und verknüpft werden. Eine wichtige Besonderheit sind Verknüpfungen dieser neuen DPU-Instanzen mit DPU-Instanzen aus dem Vorgänger-Pool. Solche Verknüpfungen überschreiben die aus dem Vorgänger-Pool. Wird ein Eingang einer DPU-Instanz aus dem Vorgängerpool mit dem Ausgang einer Instanz des abgeleiteten Pools verknüpft, dann wird die Verknüpfung des Eingangs, die im Vorgänger-Pool definiert wurde, überschrieben. Im Beispiel wird das ausgenutzt, um `Sinus2` mit einem Offset zu versehen. Das Offset wird von einer Instanz namens `Offset` der DPUAmplifier erzeugt. Die alte Verbindung zwischen `Sinus2.Out` und `Mixer.In` wird aufgebrochen und `Offset` dazwischen geschaltet. Die folgende Abbildung verdeutlicht den Vorgang. Die Farben entsprechen denen aus dem Beispiel.

⁵ Der Mechanismus gleicht dem in objektorientierten Programmiersprachen.

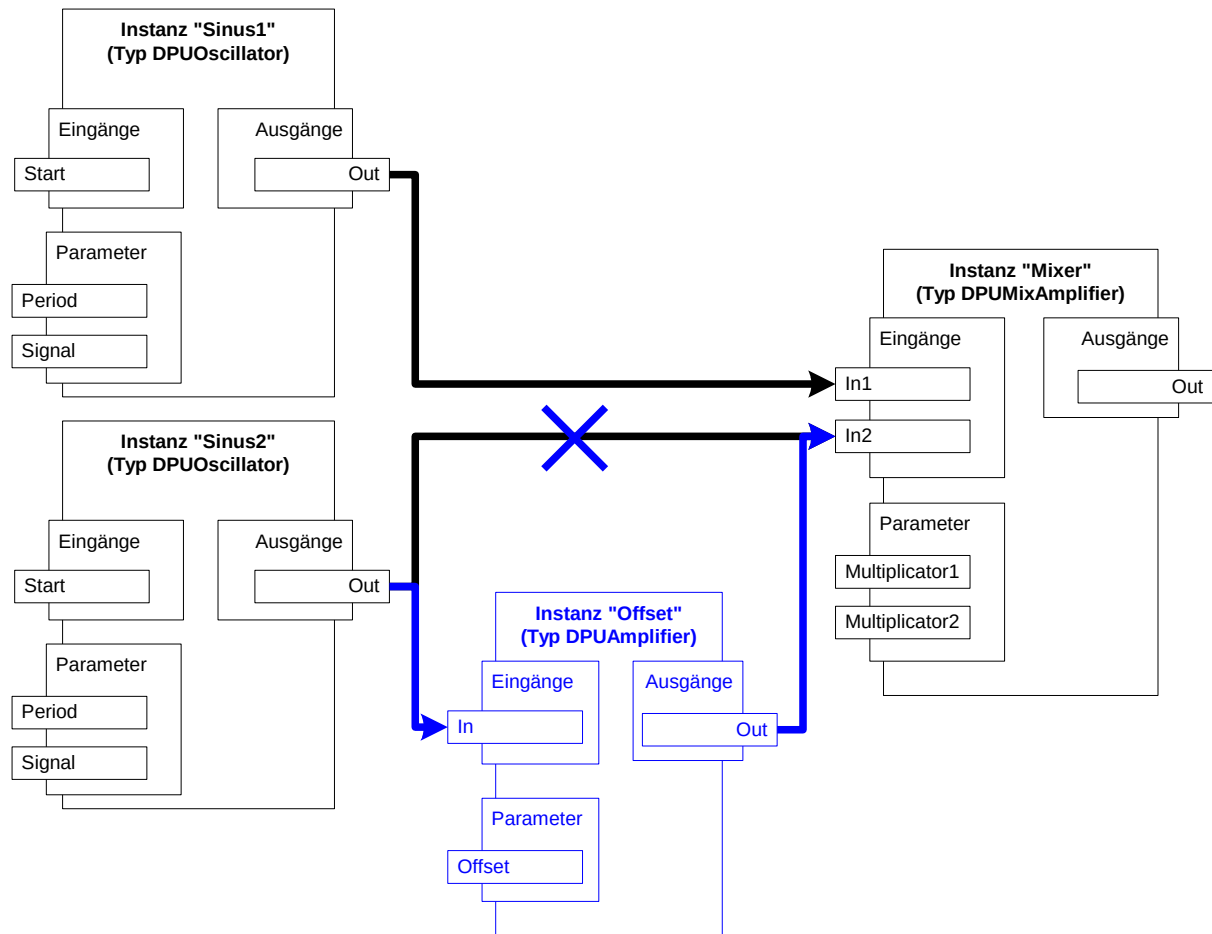


Abbildung 5: Der DPU-Pool Extension

Wenn ein Vorgänger-Pool keine für sich vollständige DPU-Konfiguration enthält, sondern nur durch die Ergänzungen in einem abgeleiteten Pool vervollständigt wird, dann soll der Vorgänger-Pool auch nicht in der Auswahlliste von SILAB erscheinen. Dies wird erreicht, indem man Pools, die in der Auswahlliste auftauchen sollen, mit `Executable = true` markiert (im Beispiel grün hervorgehoben). Obige Konfigurationsdatei lässt beide Pools für die Auswahl zu.

Der Mechanismus der Vererbung erlaubt also, bestehende Pools (z.B. Grundkonfiguration einer Fahrsimulation) leicht um zusätzliche DPU-Instanzen (z.B. Assistenzsysteme) zu erweitern. In den abgeleiteten Pools können Parameter von „alten“ DPU-Instanzen neu definiert und „alte“ Verknüpfungen überschrieben werden. Es gibt noch einige weitere Möglichkeiten, durch Ableitung von DPU-Pools Konfigurationen sehr flexibel zu gestalten. Darauf wird in Abschnitt 2.1.7 eingegangen.

2.1.5 Computer-Pools und Aliasnamen

Angenommen, es stehen zwei mit SILAB betriebene Simulatoren zur Verfügung. Die beiden bestehen aus Rechnern mit den gleichen Aufgaben, allerdings mit unterschiedlichen Betriebssystem-Rechnernamen. Um DPU-Konfigurationen zwischen den Simulatoren auszutauschen, müsste jedes Mal der Eintrag `Computer = { ... }` in

den DPU-Instanzen angepasst werden. Dieses Problem lässt sich mit dem Konzept von ComputerPools und Aliasnamen lösen.

Um die Vorgehensweise am Beispiel dieses Kapitels zu erläutern, wird angenommen, dass es neben den Rechnern `COMP1` und `COMP2` zwei andere gibt mit den Betriebssystem-Rechnernamen `COMP3` und `COMP4`.

Analog zu DPU-Pools kann eine Konfiguration auch mehrere Rechner-Pools haben. Im Beispiel gibt es zwei Pools, einen (`Sim1`) mit den Rechnern `COMP1` und `COMP2`, einen weiteren (`Sim2`) mit den Rechnern `COMP3` und `COMP4`:

```
Pool Sim1
{
    Executable = true;
    Computer COMP1
    {
        PeriodMS = 10;
        IP = "10.1.2.21";
        Operator = true;
        Alias = {COMP_A};
    };

    Computer COMP2
    {
        PeriodMS = 10;
        IP = "10.1.2.45";
        Alias = {COMP_B};
    };
};

Pool Sim2
{
    Executable = true;
    Computer COMP3
    {
        PeriodMS = 10;
        IP = "10.1.2.27";
        Operator = true;
        Alias = {COMP_A};
    };

    Computer COMP4
    {
        PeriodMS = 10;
        IP = "10.1.2.87";
        Alias = {COMP_B};
    };
};
```

Jedem Rechner können beliebig viele sog. Aliasnamen gegeben werden. Hat ein Rechner X einen Aliasnamen Y dann bedeutet dies: Rechner X ist SILAB auch unter dem Namen Y bekannt. In obigem Beispiel ist `COMP1` in SILAB auch unter dem Namen `COMP_A` bekannt (fett). Gleichzeitig ist auch `COMP3` unter dem Aliasnamen `COMP_A` bekannt.

Aliasnamen von Rechnern dürfen in der Zuweisung `Computer = {...}` einer DPU-Instanz stehen. Ändert man die DPU-Konfiguration des Beispiels wie folgt ab (fett),

dann kann sie sowohl auf den Rechnern des Pools `Sim1` als auch auf denen von `Sim2` ausgeführt werden:

```
Pool BASE
{
    Executable = true;

    DPUOscillator Sinus1
    {
        Computer = {COMP_A};
        Index = 10;
        Period = 1000;
        Signal = 0;
    };

    DPUOscillator Sinus2
    {
        Computer = {COMP_B};
        Period = 200;
        Signal = 0;
    };

    DPUMixAmplifier Mixer
    {
        Computer = {COMP_A};
        Index = 20;
        Multiplicator1 = 3.0;
        Multiplicator2 = 1.0;
    };

    Connections =
    {
        Sinus1.Out -> Mixer.In1,
        Sinus2.Out -> Mixer.In2
    };
};

Pool Extension : Base
{
    Executable = true;

    Sinus1.Period = 2000;

    DPUAmplifier Offset
    {
        Computer = {COMP_A};
        Index = 15;
        Offset = 1.0;
    };

    Connections =
    {
        Sinus2.Out -> Offset.In,
        Offset.Out -> Mixer.In
    };
};
```

Welcher Rechner-Pool bei einer Simulation verwendet soll, kann beim Laden einer Konfiguration in SILAB aus einer Auswahlliste gewählt werden. Analog zu den DPU-

Pools erscheinen in der Auswahlliste nur diejenigen Rechner-Pools, die mit `Executable = 1` gekennzeichnet sind.

Wie schon erwähnt, kann ein einzelner Rechner mehrere Aliasnamen haben. Um alle DPU-Instanzen des Beispiels auf einem einzigen Rechner mit dem Rechnernamen `COMP5` laufen zu lassen, muss folgender Pool `Sim3` erstellt werden:

```
Pool Sim3
{
    Computer COMP5
    {
        PeriodMS = 10;
        IP = "10.1.2.11";
        Operator = true;
        Alias = {COMP_A, COMP_B};
    };
};
```

Voraussetzung für einen reibungslosen Simulationsablauf ist natürlich, dass `COMP5` genügend leistungsfähig ist. Bei den DPUs des Beispiels stellt dies kein Problem dar. In einer Fahrsimulation werden jedoch wesentlich rechenintensivere DPUs verwendet (z.B. Fahrdynamiksimulation, Datenbasis, Verkehrssimulation, Visualisierung), was leicht zu einer Überlastung eines Rechners führen kann.

SILAB bietet bei der Definition von Aliasnamen für Rechner und bei der Zuweisung von Rechnern an DPUs einige weitere praktische Möglichkeiten, auf die in Abschnitt 2.1.7 eingegangen.

2.1.6 Gesamte Beispielfunktion

Um die erarbeitete Beispielfunktion in SILAB öffnen zu können, müssen noch drei syntaktische Konstrukte ergänzt werden:

1. Die Menge aller Computer-Pools wird in dem Block

```
Computerconfiguration <Name>
{
    ....
};
```

zusammengefasst (rot hervorgehoben). Der Name ist beliebig.

2. Die Menge aller DPU-Pools wird in dem Block

```
DPUConfiguration <Name>
{
    ....
};
```

zusammengefasst (blau hervorgehoben). Der Name ist beliebig.

3. Die gesamte Konfiguration wird in dem Block

```
SILAB Configuration
{
    ....
};
```

zusammengefasst (grün hervorgehoben).

```
SILAB Configuration
{
  Computerconfiguration computer
  {
    Pool Sim1
    {
      Computer COMP1
      {
        PeriodMS = 10;
        IP = "10.1.2.21";
        Operator = true;
        Alias = {COMP_A};
      };
      Computer COMP2
      {
        PeriodMS = 10;
        IP = "10.1.2.45";
        Alias = {COMP_B};
      };
    };
    Pool Sim2
    {
      Computer COMP3
      {
        PeriodMS = 10;
        IP = "10.1.2.27";
        Operator = true;
        Alias = {COMP_A};
      };
      Computer COMP4
      {
        PeriodMS = 10;
        IP = "10.1.2.87";
        Alias = {COMP_B};
      };
    };
    Pool Sim3
    {
      Computer COMP5
      {
        PeriodMS = 10;
        IP = "10.1.2.11";
        Operator = true;
        Alias = {COMP_A, COMP_B};
      };
    };
  };
};
```

```

DPUConfiguration dpus
{
    Pool Base
    {
        Executable = true;
        DPUOscillator Sinus1
        {
            Computer = {COMP_A};
            Index = 10;
            Period = 1000;
            Signal = 0;
        };
        DPUOscillator Sinus2
        {
            Computer = {COMP_B};
            Period = 200;
            Signal = 0;
        };
        DPUMixAmplifier Mixer
        {
            Computer = {COMP_A};
            Index = 20;
            Multiplicator1 = 3.0;
            Multiplicator2 = 1.0;
        };
        Connections =
        {
            Sinus1.Out -> Mixer.In1,
            Sinus2.Out -> Mixer.In2
        };
    };

    Pool Extension : Base
    {
        Executable = 1;
        Sinus1.Period = 2000;
        DPUAmplifier Offset
        {
            Computer = {COMP_A};
            Index = 15;
            Offset = 1.0;
        };
        Connections =
        {
            Sinus2.Out -> Offset.In,
            Offset.Out -> Mixer.In
        };
    };
};
};
};

```

2.1.7 Weitere Möglichkeiten

2.1.7.1 Automatische Aliasnamen

Zusätzlich zu den in der Konfiguration definierten Alias-Namen eines Computers wird automatisch ein betriebssystemspezifischer Alias-Name definiert. Er lautet stets

WINDOWS oder LINUX.

Beispiel: Wird im Computer-Pool Sim1


```
Pool Sim1
{
    Computer COMP1
    {
        PeriodMS = 10;
        IP = "10.1.2.21";
        Operator = true;
        Alias = {COMP_A};
    };
    Computer COMP2
    {
        PeriodMS = 10;
        IP = "10.1.2.45";
        Alias = {COMP_B};
    };
};
```

der Rechner `COMP1` unter Windows, `COMP2` aber unter Linux ausgeführt, dann ist `COMP1` mit den Aliasnamen `COMP_A` sowie `WINDOWS` ansprechbar, `COMP2` hingegen mit `COMP_B` und `LINUX`.

Diese Möglichkeit kann genutzt werden, wenn Konfigurationen erstellt werden, die für mehrere Simulatoren verwendet werden können, wobei betriebssystemabhängig unterschiedliche DPUs zum Einsatz kommen.

2.1.7.2 Gruppierung von Rechnern

Die Rechner eines Computer-Pools können in Gruppen eingeteilt werden. Ein Rechner kann dabei Mitglied in mehreren Gruppen sein.

Das Konzept soll an einem Beispiel erläutert werden: In der Konfiguration einer Fahrsimulation mit SILAB gibt es eine DPU namens `DPUSCNX`, welche die sog. Datenbasis `SCNX` repräsentiert. Die Datenbasis liefert anderen Modulen der Simulation wichtige Informationen über das Straßennetzwerk, die umgebende Landschaft, Verkehrsschilder usw. Diese Informationen sind so umfangreich, dass sie nicht mehr über Variablenverknüpfungen ausgetauscht werden können. Das Problem ist in SILAB so gelöst, dass die `DPUSCNX` auf mehreren Rechnern parallel ausgeführt wird. Module auf diesen Rechnern, die Informationen von der Datenbasis brauchen, können dadurch direkt und sehr effizient auf die Datenstrukturen von `SCNX` zugreifen.

Angenommen, folgende Rechner einer Simulation müssen auf die Datenbasis `SCNX` zugreifen:

- `MAIN` : Hauptrechner der Simulation
- `GRAPHICS1`, `GRAPHICS2` : Bildgeneratoren

Dann ist es nahe liegend, diese Rechner zu einer Gruppe „DATABASE“ zusammen zu fassen. Dies wird durch Vergabe des Aliasnamen `DATABASE` für alle Rechner der Gruppe erreicht (fett):

```

Pool Sim1
{
    ...
    Computer MAIN
    {
        PeriodMS = 10;
        IP = "10.1.2.21";
        Operator = true;
        Alias = {DATABASE};
    };
    Computer GRAPHICS1
    {
        PeriodMS = 10;
        IP = "10.1.2.45";
        Alias = {VISUALIZATION, DATABASE};
    };
    Computer GRAPHICS2
    {
        PeriodMS = 10;
        IP = "10.1.2.45";
        Alias = {VISUALIZATION, DATABASE};
    };
    ...
};

```

Im DPU-Pool wird die Instanz der DPUSCNX dann wie folgt definiert:

```

DPUSCNX SCNX
{
    Index = 123;
    Computer = {DATABASE};
    ...
};

```

Ein Rechner kann mehreren Gruppen angehören. In obigem Ausschnitt eines Computerpools gehört Beispielsweise der Rechner `GRAPHICS1` sowohl zur Gruppe `VISUALIZATION` als auch zur Gruppe `DATABASE`.

2.1.7.3 DPU-Aliasnamen

Zusätzlich zu dem bei der Instanzierung vergebenen DPU-Namen können für eine DPU auch zusätzliche DPU-Aliasnamen vergeben werden. Dazu können eine oder mehrere Aliasnamen in der Menge **Alias = { ... }** angegeben werden. Beispiel:

```

Pool Extension : Base
{
    Executable = true;

    DPUAmplifier Offset
    {
        Computer = {COMP1};
        Alias = {MyFavoriteDPU};
        Index = 15;
        Offset = 1.0;
    };

    Connections =
    {
        Sinus2.Out -> Offset.In,
        MyFavorite.Out -> Mixer.In
    };

    MyFavoriteDPU.Multiplicator = 3;
};

```

Die DPU-Aliasnamen können bei Variablenverknüpfungen und bei einer Variableninitialisierung im DPU-Pool (siehe Abschnitt 2.7.3) genau wie der ursprüngliche DPU-Name verwendet werden.

Hinweis: Die DPU-Aliasnamen sind nur in der Konfigurationsdatei sichtbar. Zur Laufzeit der Simulation wird im Treeview von SILAB (siehe Abschnitt 5.2) und in allen geöffneten Displays (siehe Abschnitt 5.3) nur der bei der Instanzierung vergebene DPU-Name angezeigt. Um die Konfiguration möglichst übersichtlich zu halten, empfehlen wir, DPU-Aliasnamen nur sparsam zu verwenden. In der SILAB-Standardkonfiguration werden sie lediglich dazu eingesetzt, um die Kompatibilität von Strecken über verschiedene SILAB-Versionen hinweg zu erhöhen. Dazu werden für einige DPUs der Standardkonfiguration (z.B. *DPUSGEGView*) zusätzlich zu ihrem eigentlichen DPU-Instanznamen (*SGEGView*) noch ein DPU-Aliasname definiert, der gleich lautet wie der DPU-Instanzname in früheren SILAB-Versionen (in diesem Fall *SIGView*).

2.1.7.4 Mengenoperationen beim Zuweisen von Rechnern an DPUs

In einem abgeleiteten DPU-Pool kann die Zuweisung von Rechnern an DPUs aus dem Vorgängerpool nachträglich geändert werden. Dies geschieht durch folgende Mengenoperationen:

```
<DPU-Instanz>.Computer += {<Rechner-/Aliasname>, ...};
```

Die im Vorgänger-Pool definierte Rechnerliste der DPU <DPU-Instanz> wird mit der neuen vereinigt.

```
<DPU-Instanz>.Computer -= {<Rechner-/Aliasname>, ...};
```

Von der im Vorgänger-Pool definierten Rechnerliste der DPU <DPU-Instanz> wird die Schnittmenge mit der neuen abgezogen.

```
<DPU-Instanz>.Computer = {<Rechner/Aliasname>, ...};
```

Die im Vorgänger-Pool definierte Rechnerliste der DPU <DPU-Instanz> wird komplett neu gesetzt.

Ein einfaches Beispiel demonstriert die Möglichkeiten dieser Mengenoperationen: Angenommen, ein Simulator soll um einen zusätzlichen Grafikrechner namens `GRAPHICS_NEW` erweitert werden. Dann müssen einige DPUs zusätzlich zu den bestehenden Grafikrechnern auch auf dem neuen berechnet werden. Ein Beispiel für eine solche DPU ist die `DPUSCNX` (vgl. Abschnitt 2.1.7.2). Hierzu wird vom bestehenden DPU-Pool `Sim` (also dem ohne den neuen Grafikrechner) ein DPU-Pool `Sim_new` abgeleitet und der Instanz `SCNX` von `DPUSCNX` der neue Grafikrechner hinzugefügt:

```
Pool Sim_new : Sim
{
    ...
    SCNX.Computer += {GRAPHICS_NEW};
    ...
};
```

2.1.7.5 Berechnungsreihenfolge von DPUs ändern

Die Berechnungsreihenfolge der DPUs eines Rechners wird über den Parameter `Index` einer DPU-Instanz festgelegt (s. Abschnitt 2.1.3). Es kann vorkommen, dass diese Reihenfolge in einem abgeleiteten Pool verändert werden soll. Dies kann durch folgende Zuweisung im abgeleiteten Pool erreicht werden:

```
Pool <Name abgel. Pool> : <Name Vorgänger-Pool>
{
    ...
    <DPU-Instanz>.Index = <neuer Index>;
    ...
};
```

2.1.7.6 Rechnerspezifische Variableninitialisierung

Wie in Abschnitt 2.1.4 erläutert, können nicht verknüpfte Eingänge von DPUs eines Vorgängerpools in einem abgeleiteten Pool initialisiert werden. Dies geschieht über Zuweisungen der Art

```
<DPU-Instanz>.<Eingang> = <Wert>;
```

Wenn die DPU <DPU-Instanz> auf mehreren Rechnern parallel ausgeführt wird, d.h. in ihrer Computerliste mehr als ein Rechner eingetragen wurde, dann wird <DPU-Instanz>.<Eingang> auf *jedem* dieser Rechner auf <Wert> gesetzt. Es gibt DPUs, die zwar auf mehreren Rechnern parallel laufen müssen, auf jedem dieser Rechner jedoch unterschiedlich parametrisiert werden.

Ein Beispiel für eine solche DPU ist die `DPUSGEView`. Sie repräsentiert in einer Fahrsimulation einen Bildkanal der Grafik. Angenommen, ein Simulator hat drei Frontsicht-Kanäle, von denen jeder horizontal 60° des Sichtfeldes abdeckt. Jeder dieser Kanäle soll von einem eigenen Computer berechnet werden (im folgenden `FrontLeft`, `FrontCenter`, `FrontRight` genannt). Deswegen wird auf jedem dieser

Rechner eine Instanz der DPUSGEView benötigt. Die Instanzen unterscheiden sich im Blickwinkel:

- Auf `FrontLeft` ist die Blickrichtung um 60° nach links gedreht.
- `FrontCenter` stellt die Grafik mit 0° Blickwinkel dar.
- Auf `FrontRight` ist die Blickrichtung um 60° nach rechts gedreht.

Der für die Blickrichtung zuständige Parameter der DPUSGEView ist `CamGamma`. Das Problem wird in der Konfiguration über eine sog. rechnerspezifische Initialisierung von `CamGamma` gelöst:

```
Pool Sim
{
    ...
    DPUSGEView View
    {
        # DPUSGEView läuft auf allen Frontkanälen
        Computer = {FrontLeft, FrontCenter, FrontRight};

        ...
    };

    # Blickwinkel rechnerspezifisch einstellen
    FrontLeft.View.CamGamma = -60.0;
    FrontCenter.View.CamGamma = 0.0;
    FrontRight.View.CamGamma = 60.0;

    ...
};
```

Die rechnerspezifische Initialisierung von Variablen einer DPU, die in einem Vorgängerpool instanziiert wurde, kann auch im abgeleiteten Pool vorgenommen werden.

2.1.7.7 Auftrennen von Verknüpfungen

In Abschnitt 2.1.4 wurde gezeigt, wie DPU-Verknüpfungen, die in einem Vorgängerpool definiert wurden, in einem abgeleiteten Pool überschrieben werden. Daneben gibt es noch die Möglichkeit, eine Verknüpfung komplett aufzutrennen. Die Syntax hierfür ist:

```
Connections =
{
    ...
    default -> <DPU-Instanz>.<Eingang>,
    ...
};
```

2.2 Allgemeines

Bevor auf die genaue Struktur von Konfigurationen eingegangen wird, finden Sie in diesem Abschnitt noch einige allgemeine Informationen zu Konfigurationen.

2.2.1 Editieren und Überprüfen von Konfigurationsdateien

Konfigurationen sind reine Textdateien, die mit jedem beliebigen Texteditor editiert werden können. Wenn eine Konfigurationsdatei in einem Texteditor auf Korrektheit überprüft werden soll, dann muss der verwendete Texteditor die Möglichkeit bieten, externe Kommandos einzubinden. Dies ist beispielsweise in den Editoren „Geany“

und „Crimson Editor“ möglich. Nähere Informationen zur Installation und Konfiguration eines Texteditors finden Sie in der *Installationsanleitung SILAB 7.1*, Abschnitt *Einrichten eines Texteditors*.

2.2.2 Notation

Die Sprache, in der eine Konfiguration von SILAB definiert ist, wird im Folgenden anhand sog. formaler Grammatiken spezifiziert. Die Grammatiken enthalten Regeln, mit denen syntaktisch korrekte Konfigurationsdateien formuliert werden können.

Folgendes Beispiel demonstriert dies:

Grammatik 1 Zahl

$\langle \text{Zahl} \rangle \rightarrow$
 $(0,1)\langle \text{Vorzeichen} \rangle \langle \text{Vorkomma-Anteil} \rangle_{(0,1)} \langle \text{Nachkomma-Anteil} \rangle_{(0,1)} \langle \text{Exponent} \rangle$

$\langle \text{Vorzeichen} \rangle \rightarrow$
 $+$ | $-$

$\langle \text{Vorkomma-Anteil} \rangle \rightarrow$
 $(1...*)\langle \text{Ziffer} \rangle$

$\langle \text{Nachkomma-Anteil} \rangle \rightarrow$
 $\cdot (1...*)\langle \text{Ziffer} \rangle$

$\langle \text{Ziffer} \rangle \rightarrow$
 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

$\langle \text{Exponent} \rangle \rightarrow$
 $e_{(0,1)} \langle \text{Vozeichen} \rangle_{(1...*)} \langle \text{Ziffer} \rangle$

Grammatik 1 definiert die Schreibweise einer Zahl in der wissenschaftlichen Darstellung wie z.B. +1234.56e+012, -55.2 oder 2002. Sie besteht aus einer Reihe von Ersetzungsregeln der Form

$\langle \dots \rangle \rightarrow \dots$

Bezeichner in spitzen Klammern sind Nicht-Terminalsymbole und werden gemäß den Regeln, auf deren linker Seite sie erscheinen, ersetzt. Auf der rechten Seite stehen entweder Nicht-Terminalsymbole, für die wiederum eine eigene Ersetzungsregel existiert, oder Terminalsymbole. Die Terminalsymbole (in fester Schrift dargestellt) sind Worte über einer Menge von Zeichen, die Sie tatsächlich „eintippen“. Nicht-Terminalsymbolen auf der rechten Seite einer Regel sind gelegentlich Quantifizierer vorangestellt, die angeben, wie oft das Symbol ersetzt wird. So besagt die erste Regel in Grammatik 1 beispielsweise, dass eine Zahl aus keinem oder einem Vorzeichen, einem Vorkomma-Anteil, keinem oder einem Nachkomma-Anteil und aus keinem oder einem Exponenten besteht.

Mögliche Quantifizierer sind:

- (1) oder kein vorangestellter Quantifizierer: genau einmalige Ersetzung
- (0,1) : keine oder einmalige Ersetzung
- (1...*) : ein- oder mehrmalige Ersetzung
- (0...*) : keine oder mehrmalige Ersetzung
- (,) : ein oder mehrmalige Ersetzung. Die einzelnen Ersetzungen werden durch Kommata getrennt.

Die Quantifizierer beziehen sich immer nur auf das nachfolgende Nicht-Terminalsymbol. Werden mehrere Symbole einem Quantifizierer zugeordnet, dann werden diese in eckige, kursive Klammern gefasst.

Für die Ersetzung auf der rechten Seite einer Regel kann es Alternativen geben, die durch das Zeichen | getrennt sind. So kann in Grammatik 1 das Nicht-Terminalsymbol *<Vorzeichen>* durch die Symbole + oder – ersetzt werden. Auch die vorletzte Regel der Grammatik nutzt diese Schreibweise: Eine Ziffer ist eine 0 oder eine 1 oder ... oder eine 9.

Die Spezifikation der Konfigurationssprache wird in mehrere Teil-Grammatiken aufgeteilt, um beschreibenden Text einfügen zu können. Wenn eine Teil-Grammatik dabei ein Nicht-Terminalsymbol verwendet, dessen Ersetzungsregel in einer anderen Teil-Grammatik definiert ist, wird darauf verwiesen.

Grammatik 2 verwendet den in der Teil-Grammatik 1 definierten Bezeichner *<Zahl>* und verweist dabei auf Grammatik 1. Sie beschreibt gültige Zuweisungen zu Parametern und Eingängen einer DPU.

Grammatik 2 Parameterzuweisungen

```

<Parameterzuweisung> →
    <Name des Parameters/Eingangs/Ausgangs> = <Wert>;
<Wert> →
    <Bool> | <String> | <Zahl(→Grammatik 1 Seite 30)> | <Nummer> |
    <Token> | <Tupel> | <Set> | <Range>
<Bool> →
    true | false | TRUE | FALSE
<String> →
    "<Text6 innerhalb der Anführungszeichen>"
<Nummer> →
    (1...*)<Ziffer>(→Grammatik 1 Seite 30)
<Token> →
    <Wort7 ohne Anführungszeichen>
<Tupel> →
    ( (<Wert>) )
<Set> →
    { (<Wert>) }
<Range> →
    <Wert> ... <Wert>

```

Soll zum Beispiel ein Parameter AcceleratorPedalGain einer DPU in der Konfiguration auf den Wert 1.2 gesetzt werden, dann muss in den Abschnitt der DPU die Zeile

```
AcceleratorPedalGain = 1.2;
```

aufgenommen werden.

2.2.3 include

Ähnlich wie durch den „#include“-Befehl der Programmiersprache C++ können Sie auch in SILAB Konfigurationsdateien in Einzeldateien aufteilen. An beliebiger Stelle eines Skripts kann mit

```
include <Pfad>
```

ein Teilskript eingebunden werden. Zunächst such SILAB die einzubindende Datei in Ihrem Unterordner von \SILAB\Projects. Sollte Sie da nicht gefunden werden, dann wird der <Pfad> vorne um \silab\lib\configurations ergänzt. Beispielsweise binden Sie durch die Anweisung

```
include test\irgendwas.cfg
```

⁶ (0...*)^[^\\n] : kann also auch leer sein ("") und darf beliebige Zeichen mit Ausnahme von Anführungszeichen oder Zeilenumbrüchen enthalten.

⁷ (1)[_a...zA...Z](0...*)[_a...zA...Z0...9] : es sind also keine Umlaute und Leerzeichen erlaubt

die Datei `\silab\lib\configurations\test\irgendwas.cfg` in das Skript ein. Sie können diesen Mechanismus dazu benutzen, spezifische Konfigurationen für Ihr Projekt (als Unterverzeichnis von `\SILAB\Projects`) einzubinden oder eine Bibliothek häufig verwendeter Teilkonfigurationen anzulegen, in der sich z.B.

- verschiedene Versionen von Rechnerkonfigurationen (vgl. Abschnitt 2.6)
- oft verwendete DPUs samt deren Verknüpfungen (vgl. Abschnitt 2.7)
- Typen von Situationsmodulen
- Typen von Verkehrsknotenpunkten

befinden.

2.2.3.1 includeif

Alternativ zu `include` kann auch die Anweisung

```
includeif <Pfad>
```

notiert werden. In diesem Fall wird keine Fehlermeldung ausgegeben und die restliche Konfiguration wird trotzdem geladen, falls die Datei `<Pfad>` nicht existiert.

Ansonsten verhält sich `includeif` genau wie `include`.

2.2.3.2 includeat

Die `includeat`-Anweisung kann benutzt werden, um systemspezifische Konfigurationsabschnitte in die Konfiguration einzubinden. Mit der Syntax

```
includeat <Name> <Pfad>
```

wird die Datei `<Pfad>` nur dann eingebunden, wenn der Systemname `<Name>` in der Datei `sys.par` eingetragen ist (siehe Kap. 4). Andernfalls wird die `includeat`-Anweisung einfach ignoriert.

Für `<Name>` kann auch eine durch Kommas (aber keine Leerzeichen!) getrennte Liste von Systemnamen verwendet werden. Die `include`-Anweisung wird dann auf allen genannten Systemen ausgeführt.

Zusätzlich kann `includeat` benutzt werden, um betriebssystemabhängig Konfigurationsabschnitte einzubinden. Für `<Name>` kann dazu `Windows` oder `Linux` eingegeben werden. Die `includeat`-Anweisung wird dann nur ausgeführt, wenn auf dem Operator-Rechner das angegebene Betriebssystem installiert ist⁸.

2.2.4 Kommentare

Das Kommentarzeichen in SILAB-Konfigurationen ist die Raute „`#`“. Alles, was in einer Zeile hinter der Raute steht, wird von SILAB ignoriert.

Mehrzeilige Kommentare können mit der aus C bekannten Syntax `/* ... */` eingefügt werden. Alles, was zwischen dem Kommentaranfang `/*` und dem kommentierende `*/` steht, wird ignoriert. Mehrzeilige Kommentare sind auch sehr nützlich, um größere Blöcke vorläufig aus der Konfiguration zu entfernen.

2.2.5 Dateibezeichnung

Der Name einer Konfigurationsdatei darf sich aus Buchstaben, Zahlen und Unterstrichen zusammensetzen. Minuszeichen und Leerzeichen dürfen nicht verwendet werden. An einem PC alleine kann im Einzelfall eine solche Konfigurationsdatei mit feh-

⁸ Bitte beachten Sie, dass `include`-Anweisungen nur auf dem Operator-Rechner ausgeführt werden. Daher ist es nicht relevant, welches Betriebssystem auf den weiteren Simulator-Rechnern installiert ist.

lerhaftem Namen funktionieren, jedoch keinesfalls bei einer Konfiguration mit mehreren beteiligten Computern.

2.3 Grundlegender Aufbau einer Konfiguration

Die Konfiguration von SILAB erfolgt mittels einer Textdatei. Die Textdatei kann bis zu drei Abschnitte enthalten:

Grammatik 3 Grundlegender Aufbau einer Konfiguration

```
(0,1)<SILAB Systemkonfiguration> (→Grammatik 4 Seite 34)
(1)< Rechner- und DPU-Konfiguration > (→Grammatik 7 Seite 40)
(0,1)<Datenbasis-Abschnitt >
```

Sie besteht aus einer Systemkonfiguration, aus einem Abschnitt mit Informationen über die in der Simulation eingebundenen Computer sowie einem Abschnitt mit DPUs. Soll die Simulation in einem Straßennetz erfolgen, wird im dritten Abschnitt der Konfiguration die Datenbasis beschrieben. Dieser Abschnitt wird von den DPUs DPUSCNX, DPUSCNVISX, DPUMOV, DPUTRFX⁹ verarbeitet.

2.4 SILAB System Abschnitt

Im System-Abschnitt werden Parameter definiert, die global das Verhalten von SILAB beeinflussen. Damit kann beispielsweise auf die Ablaufsteuerung von SILAB Einfluss genommen werden. Der System-Abschnitt hat folgende Syntax:

Grammatik 4 SILAB Systemkonfiguration

```
<SILAB Systemkonfiguration>→
  SILAB System
  {
    (0,1)<Launch Process> (→Grammatik 5 Seite 35)
    (0...10)<Launchdaten> (→Grammatik 6 Seite 36)
    (0,1)<Watchdogparameter> (→Abschnitt 2.4.3)
    (0,1)<Logausgabeparameter> (→Abschnitt 2.4.4)
    (0,1)<Systemsteuerungsparameter> (→2.4.5)
  };
```

2.4.1 LaunchProcess im Systemabschnitt

⁹ s. Dokumente „Referenz Datenbasis SCNX“, „Referenz Fußgängersimulation MOV“, „Referenz Verkehrssimulation TRFX“

Grammatik 5 Launch Process

```

<Launch Process>→
    LaunchProcess = {(0,1)<LaunchToken>};
<LaunchToken>→
    LC_SWITCHON_ON_LOAD |
    LC_LAUNCH_ON_SWITCHON |
    LC_START_ON_LAUNCH |
    LC_SWITCHOFF_ON_STOP |
    LC_CLOSE_ON_SWITCHOFF
  
```

Mit dem Eintrag **LaunchProcess** können Sie das Ablaufverhalten der Simulation verändern. Die Bezeichner in der Menge **LaunchProcess = {...}** stehen jeweils für einen bestimmten Übergang zwischen den Zuständen der Applikation.

Token	Beschreibung
LC_SWITCHON_ON_LOAD	SILAB kontaktiert automatisch die Remoterechner, wenn die Konfiguration geladen wird (Standardeinstellung, kann nicht deaktiviert werden).
LC_LAUNCH_ON_SWITCHON	SILAB startet automatisch den Launchschritt, wenn die Applikation die Remoterechner erfolgreich kontaktiert hat.
LC_START_ON_LAUNCH	SILAB startet automatisch nach dem Launchschritt die Simulation.
LC_SWITCHOFF_ON_STOP	SILAB schaltet die Remoteapplikationen automatisch aus, wenn die Simulation gestoppt wurde.
LC_CLOSE_ON_SWITCHOFF	SILAB beendet sich automatisch, wenn die Simulation ausgeschaltet wurde.

Bei dem automatisch ablaufenden Schritt LC_LAUNCH_ON_SWITCHON werden von SILAB keine Launchdaten (s. nächster Abschnitt) abgefragt.

2.4.2 Launchdaten im Systemabschnitt

Bei Launch-Daten handelt es sich um Informationen, die Sie vor jedem Start einer Simulation den DPU-Instanzen mitteilen können. So kann beispielsweise für jeden Simulationslauf ein Dateiname für die Aufzeichnungsdatei angegeben werden (s. „DPUDataFile.pdf“) oder der Aufsetzpunkt im Straßennetz ausgewählt werden (s. „Referenz Datenbasis SCNX.pdf“). SILAB zeigt Ihnen hierzu vor dem Start der Simulation noch einen „Launchdaten-Dialog“, der die definierten Launchdatas abfragt. Die abzufragenden Daten werden folgendermaßen definiert:

Grammatik 6 Launchdaten

```

<Launchdaten> →
    LaunchData<Nummer> =
        ("<Titeltext>", <Variablenname>, <Variablentyp>) |
        ("<Titeltext>", <Variablenname>, <Variablentyp>, "<Defaultwerttext>")
        ("<Titeltext>", <Variablenname>, LD_COMBO,
            (2,*) "<string selection>") |
        ("<Titeltext>", <Variablenname>, LD_COMBO_DEFAULT,
            "<Defaultwerttext>", (2,*)<Textauswahlelemente>")

```

Der Eintrag *LaunchData1* beschreibt dabei den ersten Eintrag im Dialog, der Eintrag *LaunchData2* den zweiten Eintrag, usw. Die eingegebenen Werte werden an alle im Computerpool befindlichen Rechner versendet und von deren DPUs ausgewertet.

Aus Kompatibilitätsgründen wird auch die folgende veraltete Bezeichnung für die Launchdaten-Einträge akzeptiert:

Aktueller Name	Veralteter Name
LaunchData1	LaunchData1A
LaunchData2	LaunchData1B
LaunchData3	LaunchData2A
LaunchData4	LaunchData2B
usw.	usw.

Die Werte für <Variablenname> werden von den DPUs festgelegt, die sich für die Information interessieren (vgl. DPU-Dokumentationen).

Der Typ der Variablen hängt von der verwendeten Variablen ab.

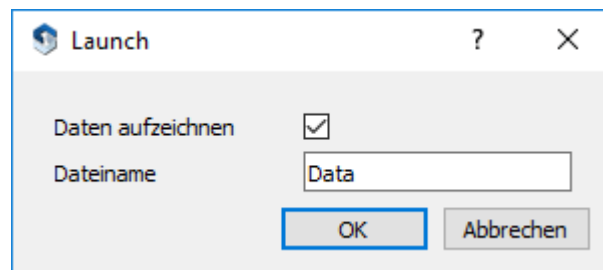
Variablentyp	Beschreibung
LD_EDIT	Eingabefeld für Texteingabe, leer
LD_EDIT_DEFAULT	Eingabefeld für Texteingabe mit dem Defaulttext als Inhalt
LD_BOOL	Checkbox zum Ankreuzen
LD_BOOL_DEFAULT	Checkbox mit Defaultwert belegt "1" für angekreuzt und "0" für nicht angekreuzt
LD_COMBO	Drop-down Menü mit den vorgegebenen Textelementen zur Auswahl
LD_COMBO_DEFAULT	Drop-down Menü mit dem Defaulttext und den vorgegebenen Textelementen zur Auswahl

Alternativ zum System-Abschnitt können Sie die Launchdaten auch im Abschnitt für den Operator-Rechner angegeben (vgl. 2.6). Die Launchdaten im globalen Systemabschnitt überschreiben die Angaben des Operatorrechners.

Beispielsweise können Sie der DPUDataFile (s. Dokument „DPUDataFile.pdf“) über die Variable „VAR_SAVEDATA“ (Typ LD_BOOL) vor jedem Start mitteilen, ob für diesen Simulationslauf eine Datenaufzeichnung erfolgen soll. Über die Variable „VAR_DATAFILENAME“ (Typ LD_EDIT) legen Sie den Namen für die Aufzeichnungsdatei fest. Die Launchdaten-Definition hierfür sieht so aus:

```
SILAB System
{
    LaunchData1 = ("Daten aufzeichnen", VAR_SAVEDATA,
                  LD_BOOL_DEFAULT, "1");
    LaunchData2 = ("Dateiname", VAR_DATAFILENAME,
                  LD_EDIT_DEFAULT, "Data");
};
```

Vor jedem Start der Simulation erscheint damit folgender Bildschirm:



2.4.3 Watchdogparameter im Systemabschnitt

Die Überwachung der Remoterechner durch sog. Watchdogs erfolgt periodisch vom Operatorrechner aus. Sobald die Remoterechner für eine bestimmte Zeitdauer die Anfragen des Operator-Rechners nicht mehr beantworten, wird die Simulation angehalten.

Mit den folgenden Parametern können Sie diese Periode sowie die Zeitvorgaben für verschiedene Zustände von SILAB verändern:

Parameter	Bedeutung
WatchdogPeriod	Periode des Watchdogs in ms
WatchdogRemote_Lost	Anzahl der Versuche einen Remoterechner zu erreichen
WatchdogRemote_LostIdle	Anzahl der Versuche einen Remoterechner zu erreichen
WatchdogDispatchConfigs	Timeout (ms) für den Versuch, die geöffnete Konfigurationsdatei an die Remoterechner zu übertragen.
WatchdogExecApps	Timeout (ms): Startvorgang der Konfiguration
WatchdogControllerInit	Timeout (ms): Vorbereitungspause
WatchdogShutDownControllers	Timeout (ms): Shutdownvorgang
WatchdogTerminateApps	Timeout (ms): Schließen der Remoteapplikation
WatchdogKillApps	Timeout (ms): Hartes Beenden der Remoteapplikation

Normalerweise müssen Sie diese Parameter nicht verändern. Ein möglicher Anwendungsfall könnte so aussehen: Ein (langsamer) Rechner benötigt nach dem Laden einer Konfiguration zur Vorbereitung der Simulation länger, als die Watchdog-

Parameter es erlauben. Bevor also der Rechner die Vorbereitung abgeschlossen hat, wird die Simulation aufgrund eines Watchdog-Timeouts beendet. Um dies zu verhindern, sollten Sie den Parameter *WatchdogPeriod* Schrittweise erhöhen, bis das Problem nicht mehr auftritt.

2.4.4 Unterdrückung / Aktivierung von Logausgaben

Mit den folgenden Parametern des Systemabschnitts können Sie die Logausgaben, die im Ausgabefenster von SILAB angezeigt werden, unterdrücken bzw. aktivieren:

Parameter	Symbol	Defaultwert	Bedeutung
LogShowInfo		true	Allgemeine Informationen
LogShowWarning		true	Warnungen
LogShowError		true	Fehlermeldungen
LogShowVersionError		true	Versionsfehler
LogShowDebug		false	Debug-Informationen

Die Ausgabe der Meldungen in der log-Datei „SILAB.sys.log“ erfolgt in jedem Fall, unabhängig von Einstellungen im Systemabschnitt.

2.4.5 Systemsteuerungsparameter

Mit folgenden Parametern können Sie weitere Eigenschaften von SILAB beeinflussen.

Parameter	Defaultwert	Bedeutung
ShutDownOnDPUErrror	false	Simulation abbrechen, wenn eine DPU einen Fehler (z.B. Speicherzugriffsfehler) auslöst. Selbst wenn dies nicht getan wird, wird auf jeden Fall eine Meldung in der Log-Datei „SILAB.error.log“ eingetragen.
MinSleepIdleTime	2.0	Wenn nach Abarbeiten eines Simulationsschrittes noch mindestens <i>MinSleepIdleTime</i> Millisekunden übrig sind, wird der Prozessor schlafen gelegt. Abhängig von der Hintergrundbelastung des Systems können höhere Werte die Anzahl auftretender Verzögerungen (die als Ausgang <i>TimeError</i> der <i>DPUClock</i> ausgegeben werden) reduzieren.
TransferProject	false	Zum einfacheren Test von Projekten kann das Projektverzeichnis bei jedem Start der Simulation automatisch auf die Simulatorrechner kopiert werden, indem dieser Wert auf <i>true</i> gesetzt wird. Hinweis: Dies verzögert den Start der Simulation, auch wenn keine Dateien geändert wurden. Es wird daher empfohlen, vor dem Start einer Versuchsreihe den Eintrag wieder zu entfernen.
CRC	false	Ein CRC-Prüfwert kann optional beim Laden eines Projekts über das komplette

		Projektverzeichnis inklusive aller Unterverzeichnisse erstellt werden. Dieser Prüfwert wird aus allen Files mit Ausnahme der Workspaces ('.wsp'-Files) erstellt. Hiermit lässt sich nachvollziehen, ob an einem Projekt Änderungen durchgeführt wurden. Neben dem Prüfwert wird die Anzahl der überprüften Dateien und Verzeichnisse angezeigt. Hinweis: Bei größeren Projekten kann die Ermittlung des Prüfwerts den Start der Simulation verzögern
--	--	---

2.5 Rechner- und DPU-Konfiguration

Der Abschnitt zur Festlegung der Rechner- und DPU-Konfiguration wird mit „SILAB Configuration“ eingeleitet. Er enthält zwei weitere Abschnitte, welche die für die Simulation verwendeten Computer und die Verteilung der DPUs auf die verschiedenen Computer enthalten.

Grammatik 7 Aufbau einer Rechner- und DPU-Konfiguration

<Rechner- und DPU-Konfiguration> →

SILAB Configuration

{

(1)<Computerkonfiguration>

(1)<DPU Konfiguration>

};

<Computerkonfiguration> →

Computerconfiguration *<Name der Computerkonfiguration>*

{

(1...*)<Computer Pool (→ Grammatik 8 Seite 41)>

};

<DPU Konfiguration> →

DPUConfiguration *<Name der DPU Konfiguration>*

{

(1...*)<DPU Pool (→ Grammatik 9 Seite 45)>

};

2.6 Computer-Konfiguration

Die Computer-Konfiguration enthält eine beliebige Anzahl von Computerpools. Durch einen Vererbungsmechanismus kann ein Pool von einem anderen Pool abgeleitet werden. Der abgeleitete Pool erbt dabei alle im Vorgänger enthaltenen Computer und kann diesen Pool um zusätzliche Computer erweitern. Die verschiedenen Computerpools werden Ihnen beim Öffnen der Konfiguration in SILAB als Auswahl präsentiert. In der Liste der auswählbaren Pools werden nur die aufgenommen, deren **Executable** Eintrag auf **true** gesetzt worden ist.

Grammatik 8 Aufbau eines Computerpools

<Computerpool> →

```
Pool <Name des Pools> (0,1)[ :<Name des Vorgängerpools> ]
{
    Executable = <Bool>;
    (1...*)<Computerinstanz>
};
```

<Computerinstanz> →

```
Computer <Name des Computers> | LOCALHOST
{
    IP = "<IP-Adresse des Computers>";
    (0,1)Name = "<Name des Computers>";
    PeriodMS = <Periode des Computers>; |
    Frequency = <Frequenz des Computers>;
    (0,1)Operator = <Bool>;
    (0,1)Alias = {(.)<Name>};
    (0,1)Group = <Name>;
    (0,1)Displayname = <Name>;
    (*)<Launchdaten>;
};
```

<Launchdaten> →

```
LaunchData<Nummer> =
    ("<Titeltext>",<Variablenname>,<Variablentyp>) |
    ("<Titeltext>",<Variablenname>,<Variablentyp>,"<Defaultwerttext>")
```

Der <Name des Computers> muss dem im Betriebssystem vergebenen Rechnernamen entsprechen. In SILAB besteht er stets ausschließlich aus Großbuchstaben. Anstatt des Namens können Sie auch alternativ **LOCALHOST** angeben. In Computerpools mit nur einem Computer ist damit immer der gemeint, auf dem SILAB gestartet wurde (unabhängig von seinem tatsächlichen Rechner-Namen). Die Portierbarkeit von Konfigurationen mit *einem* Computer wird dadurch vereinfacht. Falls der Betriebssystem-Rechnername Sonderzeichen enthält (insbesondere einen Bindestrich -), dann muss stattdessen am Anfang der Computerinstanz ein „Dummy-Name“ angegeben werden. In diesem Fall wird der tatsächliche Name dann mit dem **Name**-Eintrag angegeben. Besteht der Betriebssystem-Rechnername hingegen nur

aus Buchstaben und Zahlen, so kann der **Name**-Eintrag weggelassen werden. Der eigentlich Betriebssystemname wird verwendet um den Computereintrag im Pool richtig zuzuordnen. Soll ein aussagekräftiger Name in SILAB statt dem Betriebssystemname verwendet werden, dann kann dieser unter **Displayname** angegeben werden.

Zusätzlich zu seinem eigentlichen (Betriebssystem-)Namen kann ein Rechner beliebig viele andere sog. **Alias**-Namen haben. Im Abschnitt der DPU-Konfiguration können Sie den Rechner entweder über seinen Betriebssystem-Namen oder über einen seiner Alias-Namen identifizieren. Die Möglichkeiten, Aliasnamen zu nutzen, werden in den Abschnitten 2.1.5, 2.1.7.1 und 2.1.7.2 erläutert.

Die Zuweisung eines Rechners zu einer **Group** (Gruppe) beeinflusst die Verknüpfung von Variablen (siehe Abschnitt 2.7.2). Variablen, die auf mehreren Remote-Computern zur Verfügung stehen, werden bevorzugt von einem Computer bezogen, der in derselben *Group* wie der lokale Computer ist.

Die DPUs eines Computers werden mit dem unter **PeriodMS** angegebenen Zeitschritt (in Millisekunden) bzw. mit der unter **Frequency** angegebenen Frequenz aktualisiert. Sie können bei der Angabe der Frequenz Fließkommazahlen verwenden (z.B. 20.5, 51.23). Sind die DPUs eines Rechners sehr rechenintensiv, dann kann es vorkommen, dass der Rechner diese Zeitschritte nicht einhalten kann. Das hat zur Folge, dass der Rechner im Laufe der Simulation mit der Aktualisierung seiner DPUs immer mehr in Verzug gerät. Die entstehenden Zeitverzögerungen werden für jeden Rechner im Treeview von SILAB angezeigt. Grundsätzlich sollten in einer Simulation keine Zeitverzögerungen auftreten. Dies können Sie bei einem überlasteten Rechner erreichen, indem Sie **PeriodMS** auf einen höheren Wert bzw. **Frequency** auf einen niedrigeren Wert setzen oder die Rechenlast auf mehrere Rechner der Simulation verteilen.

Falls es sich bei einem Rechner um den handelt, von dem aus die Simulation bedient werden soll („Operatorrechner“), dann müssen Sie **Operator = true** setzen. Es kann nur einen solchen Rechner geben.

Die *<Launchdaten>* sind nur dann relevant, wenn der Computer als Operator dient. Sie ermöglichen Ihnen, vor dem eigentlichen Start der Simulation wichtige Daten einzugeben, die bestimmte DPUs benötigen. Ein Beispiel für die Launchdaten ist der Dateiname, unter dem die aufzuzeichnenden Daten abgelegt werden sollen (siehe „DPUDataFile.pdf“). Sie können die Launchdaten auch unabhängig vom Operator-Rechner im globalen System-Abschnitt der Konfiguration angeben. Eine Beschreibung des Mechanismus von LaunchDatas finden Sie in Abschnitt 2.4.2.

Beispiel:

Als Beispiel für eine Computerkonfiguration soll eine Fahrsimulation dienen, die aus drei Rechnern besteht:

- Bedienrechner (Im Betriebssystem eingestellter Rechnername „OPERATOR“)
- Hauptrechner (Rechnername „MAIN“)
- Visualisierungsrechner für einen Frontkanal (Rechnername „FRONT“)

Die Computerkonfiguration hierfür sieht so aus:

```
SILAB Configuration
{
  Computerconfiguration Rechner
  {
    Pool SimRechner
    {
      Executable = true;
      Computer OPERATOR
      {
        IP = "10.1.2.180";
        Frequency = 50.0;
        Operator = true;
        LaunchData1 = ("Daten aufzeichnen?", VAR_SAVEDATA,
                      LD_BOOL_DEFAULT, "1");
        LaunchData2 = ("Dateiname:", VAR_DATAFILENAME,
                      LD_EDIT_DEFAULT, "Data");
      };
      Computer MAIN
      {
        IP = "10.1.2.181";
        PeriodMS = 10;
      };
      Computer FRONT
      {
        IP = "10.1.2.182";
        Frequency = 60;
      };
    };
  };
};

...
```

In obigem Beispiel wird für den Operatorrechner die Rechenfrequenz der DPUs mit 50 Hz (entspricht einer Periode von 20 ms) festgelegt. Diese Frequenz ist für die Displays von SILAB (vgl. Abschnitt 5.3) ausreichend, da auf dem Operator keine DPUs laufen. Der Hauptrechner benötigt eine viel höhere Frequenz (100 Hz), da auf diesem Rechner die Fahrdynamik berechnet werden soll. Der für die Visualisierung vorgesehene Rechner wird mit einer Frequenz von 60 Hz getaktet, um eine gute Bildwiederholrate zu erreichen.

In der Beispielfunktion ist eine Datenaufzeichnung vorgesehen. Deswegen muss der Operator vor jedem Simulationsstart den anderen Rechnern über die Launch-Daten mitteilen, ob überhaupt aufgezeichnet werden soll und wie der Dateiname der Aufzeichnung lautet. Die entsprechenden Launch-Daten werden in die Definition des Operatorrechners (alternativ auch im System-Abschnitt, s. 2.4.2) eingetragen. Sie werden von der DPUDatFile verarbeitet.

2.7 DPU-Konfiguration

2.7.1 Instanziierung von DPUs, Zuweisung von Rechnern, Berechnungsreihenfolge

Grammatik 9 Aufbau eines Pools der DPU Konfiguration

```

<DPU Pool> →
    Pool <Name des Pools> (0,1)[ : <Name des Vorgängerpools> ]
    {
        Executable = <Bool> ;
        (0...*)<DPU Instanz>
        (0...*)<Anpassung Computer>
        (0...*)<Anpassung Index>
        (0...*)<Anpassung Alias>
        (0...*)<DPU Verknüpfung>
        (0...*)<DPU Variableninitialisierung>
    };

<DPU Instanz> →
    <DPU Typ> <DPU Name>
    {
        Computer = { (,)<Computernamen/Aliasnamen> };
        Index = <Nummer> ;
        (0..*)Alias = { (,)<DPU Name> };
        PeriodFactor = <Nummer> ;
        (0...*)<Parameterzuweisung> (→Grammatik 2 Seite 32)
    };

<Anpassung Computer> →
    <DPU Name>.Computer = {(,)<Computer- oder Aliasnamen>}; |
    <DPU Name>.Computer += {(,)<Computer- oder Aliasnamen>}; |
    <DPU Name>.Computer -= {(,)<Computer- oder Aliasnamen>};

<Anpassung Alias> →
    <DPU Name>.Alias = {(,)<DPU Name>}; |
    <DPU Name>.Alias += {(,)<DPU Name>}; |
    <DPU Name>.Alias -= {(,)<DPU Name>};

<Anpassung Index> →
    <DPU Name>.Index = <Nummer>;

```

Sie können einen DPU-Typ mehrmals instanzieren. Wichtig ist dabei, dass jede Instanz einen eindeutigen DPU Namen erhält.

Die bei **Computer = { ... }** angegebene Menge enthält die Namen der Rechner aus der Computerkonfiguration, auf denen die DPU-Instanz laufen soll. Als Rechnername sind auch Alias-Namen zugelassen (s. vorhergehender Abschnitt). Werden mehrere Rechner aufgeführt, dann läuft in der Simulation die DPU-Instanz parallel auf diesen Rechnern („Klone einer DPU-Instanz“). Die Zuweisung von Rechnern zu einer DPU lässt sich an späterer Stelle des Pools oder in einem abgeleiteten Pool verändern. Hierzu stehen folgende Operationen zur Verfügung:

<DPU Name>.Computer = {(<Computer- oder Aliasname>);

Die Liste der Rechner, auf denen die DPU laufen soll, wird komplett neu gesetzt.

<DPU Name>.Computer += {(<Computer- oder Aliasname>); |

Die vorher definierte Liste der Rechner wird mit der angegebenen Menge vereinigt.

<DPU Name>.Computer -= {(<Computer- oder Aliasname>);

Die Schnittmenge der vorherdefinierten Rechnerliste und der angegebenen wird von der vorherdefinierten abgezogen.

Sie können beliebig viele solche Operationen in einem DPU-Pool aufführen. Sie werden in der Reihenfolge abgearbeitet, in der sie in der Datei stehen. Auch die Rechnerlisten von DPUs, die in einem Vorgängerpool instanziiert wurden, können in einem abgeleiteten Pool auf diese Weise modifiziert werden. Ein Anwendungsbeispiel hierfür finden Sie in Abschnitt 2.1.5.

Die Reihenfolge der Berechnung der DPUs *eines* Rechners legen Sie über den Eintrag **Index=...**; der jeweiligen DPU-Instanz fest. Der absolute Wert des Index spielt dabei keine Rolle. Der Index einer DPU-Instanz kann an späterer Stelle des Pools oder in einem abgeleiteten Pool noch verändert werden (s. *<Anpassung Index>* in obiger Grammatik).

Zusätzlich zu dem bei der Instanzierung vergebenen DPU-Namen können für eine DPU auch zusätzliche DPU-Aliasnamen vergeben werden. Dazu können eine oder mehrere Aliasnamen in der Menge **Alias = { ... }**; angegeben werden.

Es können auch nachträglich (z.B. in einem abgeleiteten DPU-Pool) zusätzliche Aliasnamen für eine DPU vergeben werden.

Die Syntax

<DPU Name>.Alias = {(<DPU Name>);

<DPU Name>.Alias += {(<DPU Name>);

<DPU Name>.Alias -= {(<DPU Name>);

ist dabei analog zu der oben beschriebenen nachträglichen *Computer*-Anpassung.

Die DPU-Aliasnamen können bei einer rechnerabhängigen oder rechnerunabhängigen Variableninitialisierung im DPU-Pool (siehe Abschnitt 2.7.3) genau wie der ursprüngliche DPU-Name verwendet werden.

Auch bei der nachträglichen Anpassung von Computer, Alias, oder Berechnungsreihenfolge für die DPU können anstelle des ursprünglichen DPU-Namens die DPU-Aliasnamen verwendet werden.

Hinweis: Zur Laufzeit der Simulation wird im Treeview von SILAB und in allen geöffneten Displays nur der bei der Instanzierung vergebene DPU-Name und nicht der Alias-Name angezeigt. Daher empfiehlt es sich, DPU-Aliasnamen nur sparsam zu verwenden. Anwendungsbeispiel: In der SILAB-Standardkonfiguration werden sie lediglich dazu eingesetzt, um die Kompatibilität von Konfigurationen über verschiedene SILAB-Versionen hinweg zu erhöhen. Dazu werden für einige DPUs der Standardkonfiguration (z.B. DPUSCNX, DPUSGEView) zusätzlich zu ihrem eigentlichen DPU-Instanznamen (SCNX bzw. SGEView) noch DPU-Aliasnamen definiert, die gleich lauten wie die DPU-Instanznamen in früheren SILAB-Versionen (SCN bzw. SIGView).

Über **PeriodFactor** kann die Periode, mit der eine DPU-Instanz aktualisiert wird, gegenüber der Periode ihres Rechners vervielfacht werden. Hat beispielsweise der Rechner, auf dem die DPU-Instanz „XY“ läuft, eine Periode von 10 ms, dann wird „XY“ bei Angabe von **PeriodFactor** = 2 mit einer Frequenz von 50 Hz aktualisiert.

2.7.2 Verknüpfungen von DPUs

Grammatik 10 DPU Verknüpfungen

```
<DPU Verknüpfung> →
    Connections =
    {
        (,)[<Variablenverknüpfung> | <Verknüpfung auftrennen>]
    };
```

```
<Variablenverknüpfung> →
    <DPU Name>.<Ausgang> -> <DPU Name>.<Eingang> |
    <Computer>.<DPUName>.<Ausgang> ->
    <Computer>.<DPUName>.<Eingang>
```

```
<Verknüpfung auftrennen> →
    default -> <DPU Name>.<Eingang>
```

Eine Verknüpfung von DPUs entspricht einer Verschaltung eines DPU-Ausgangs **A** (der Quell-DPU **Q**) mit einem DPU-Eingang **B** (der Ziel-DPU **Z**). Während die Simulation läuft, wird nach jedem Simulationsschritt der Wert von **Q.A** an den Zieleingang **Z.B** übertragen. Falls **Q** und **Z** auf verschiedenen Computern laufen, überträgt SILAB die Daten dazu automatisch über das Netzwerk; die Schreibweise der Verknüpfung ist davon nicht betroffen.

Als <DPUName> ist dabei stets der Instanzname der DPU oder ihr Alias-Name anzugeben.

Falls eine DPU-Instanz auf mehreren Computern der Simulation läuft („geklont ist“), werden bei einer „allgemeinen Variablenverknüpfung“ (1. Fall, ohne Angabe eines <Computer>) alle Klone der DPU verknüpft.

Beispiel:

```
DPUAmplifier Q {  
    Computer = {COMP_A, COMP_B};  
};  
DPUOscillator Z {  
    Computer = {COMP_C, COMP_D};  
};
```

Die Verknüpfung

```
Q.Out -> Z.In
```

stellt Verknüpfungen zu allen Computern, auf denen Z läuft, her:

```
von COMP_A.Q.Out zu COMP_C.Z.In  und  
von COMP_A.Q.Out zu Comp_D.Z.In.
```

Dabei gelten folgende Regeln für die Auswahl des Quellcomputers:

- Falls ein Klon von Q auf dem Zielcomputer läuft, wird dieser verwendet.
- Falls ein Klon von Q auf einem Remotecomputer läuft, der in derselben Rechnergruppe (**Group**, siehe Kap. 2.6) wie der Zielcomputer ist, wird dieser verwendet.
- Andernfalls wird ein beliebiger Klon von Q als Quelle ausgewählt.

Durch die zusätzliche Angabe eines *<Computer>* (2. Fall) kann eine Verbindung nur zwischen einzelnen Klonen der DPU hergestellt werden.

Bei der Angabe

```
COMP_B.Q.Out <-> COMP_D.Z.In
```

wird nur diese eine Verknüpfung erstellt. Außerdem kann so explizit festgelegt werden, welcher der beiden möglichen Quellcomputer gewählt wird.

Wie bei den Computerpools können die DPU-Pools voneinander abgeleitet werden. Der abgeleitete Pool erbt alle DPUs seines Vorgängers samt deren Verknüpfungen. Da ein Eingang einer DPU sinnvollerweise nur mit einem einzigen Ausgang einer anderen DPU verschaltet werden kann, können Verknüpfungen im abgeleiteten Pool überschrieben werden. Ein- und Ausgänge werden immer gemäß der in der Konfiguration zuletzt angegebenen Verknüpfungen verschaltet. Die Vererbung kann beispielsweise verwendet werden, um einen DPU-Pool, der die grundlegenden Verschaltungen einer Simulation definiert, um spezielle DPUs (wie z.B. Fahrerassistenzsysteme) zu erweitern. Dieser Mechanismus ist Grundlage für die Standard-Konfigurationsarchitektur, auf die in Abschnitt 3 eingegangen wird.

Eine Verknüpfung können Sie an späterer Stelle der Konfiguration oder in einem abgeleiteten Pool wieder auftrennen (s. *<Verknüpfung auftrennen>* in obiger Grammatik). Ein Anwendungsbeispiel hierfür finden Sie in Abschnitt 2.1.7.7.

Eine Konfiguration kann beliebig viele *Connections* - Abschnitte enthalten¹⁰. SILAB verarbeitet die Verknüpfungen erst, wenn alle DPUs der Konfiguration instanziiert sind. Es ist deshalb im *Connections* - Abschnitt erlaubt, auf eine DPU zu verweisen, die in der Konfiguration erst später instanziiert wird.

2.7.3 Variableninitialisierung

Grammatik 11 DPU Variableninitialisierung

```

<DPU Variableninitialisierung> →
    <rechnerunabhängige Initialisierung> |
    <rechnerspezifische Initialisierung>

<rechnerunabhängige Initialisierung> →
    <DPU Name>.<Variablenname> >= <Wert (→Grammatik 2 Seite 32)>; |
    <DPU Name>.<Variablenname>= (<Defaultwert>, <Min>, <Max>,
                                   <Forbidden>, <Epsilon>);

<rechnerspezifische Initialisierung> →
    <Rechner-/Aliasname>.<DPU Name>.<Variablenname> >= <Wert (→Grammatik 2
                                                                Seite 32)>;

```

Sie haben drei Möglichkeiten, Werte von unverschalteten Eingängen von DPUs festzulegen.

1. Angeben der Werte innerhalb der DPU-Instanz (s. *<Parameterzuweisung>* in Grammatik 9).
2. *<rechnerunabhängige Initialisierung>* : Die entsprechende Variable einer DPU-Instanz *und aller ihrer Klone* wird auf den angegebenen Wert gesetzt.
3. *<rechnerspezifische Initialisierung>* : Nur die Variable eines bestimmten Klons wird auf den angegebenen Wert gesetzt.

Sie dürfen beliebig viele Initialisierungen von Typ 2 und 3 in einem DPU-Pool aufführen. Sie werden in der Reihenfolge abgearbeitet, in der sie in der Datei stehen. Sie können auch die Variablen von DPUs aus einem Vorgängerpool in einem abgeleiteten Pool auf diese Weise initialisieren.

2.7.4 Beispiel:

Die DPU-Konfiguration der einfachen Fahrsimulation aus dem vorhergehenden Abschnitt besteht aus den Pools:

- Pool Grundkonfiguration
- Pool Aufzeichnung

Der Pool Aufzeichnung wird vom ersten Pool Grundkonfiguration abgeleitet und erbt die komplette DPU-Verschaltung der Grundkonfiguration. Diese Verschaltung wird um eine weitere DPU ergänzt, welche die ausgewählten Daten aufzeichnet.

¹⁰ Zur Verbesserung der Lesbarkeit sollten Sie die einzelnen DPUs direkt nach ihrer Instanzierung verknüpfen.

```

...
DPUConfiguration DPUs
{
    Pool Grundkonfiguration
    {
        Executable = true;
    }
}
...

```

Die folgende Grafik zeigt schematisch den Aufbau des DPU-Pools „Grundkonfiguration“.

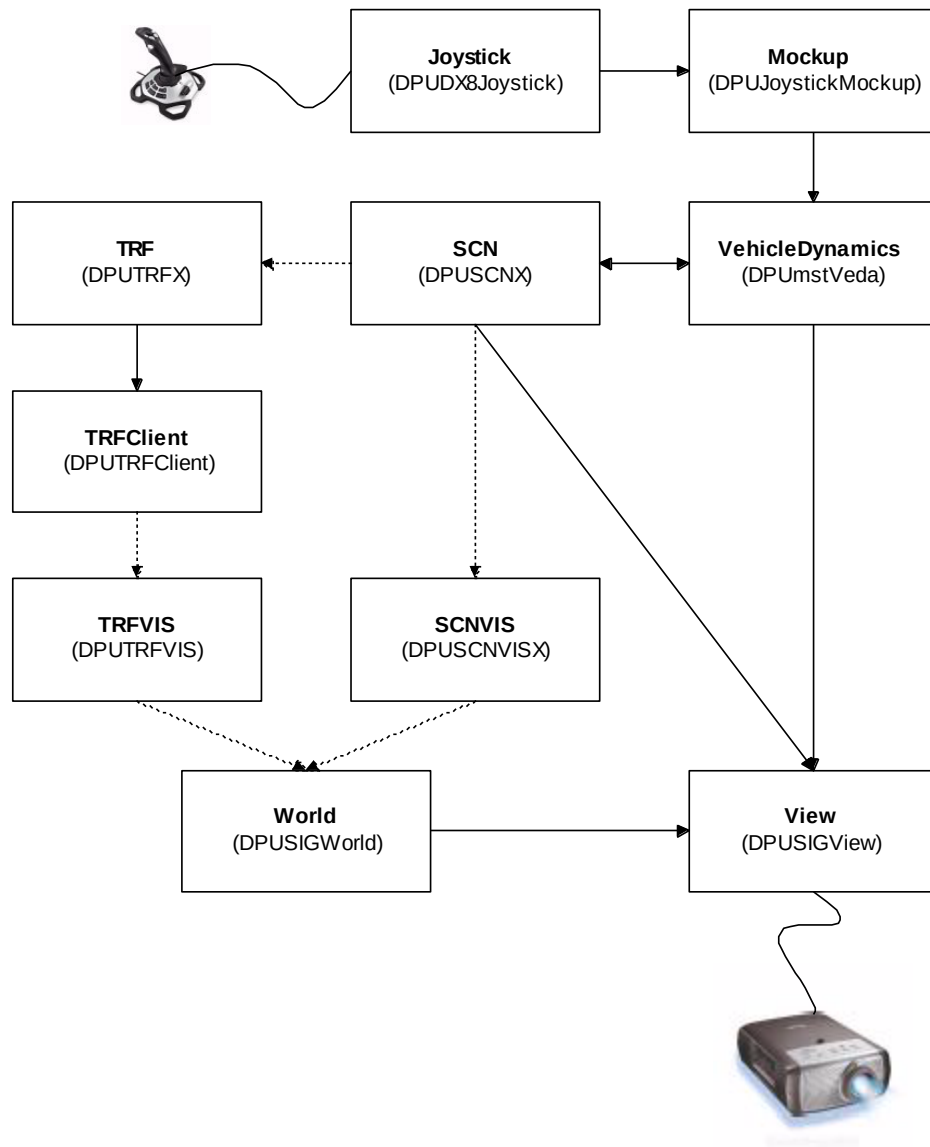


Abbildung 6: Pool "Grundkonfiguration" der Beispiel-Simulation. Gepunktete Linien symbolisieren interne Kommunikation der DPUs, d.h. Kommunikation, die nicht über DPU-Verknüpfungen definiert wird.

Die Fahrer fährt in der Simulation über einen am Hauptrechner („MAIN“) angeschlossenen Joystick. Die Abfrage eines Microsoft DirectX kompatiblen Joysticks übernimmt die DPUDX8Joystick. Sie wird mit der DPUJoystickMockup verbunden. Diese

DPU hat die Aufgabe, Joystick-Eingaben so umzurechnen, dass an ihren Ausgängen Signale wie z.B. Gaspedal-Stellung und Lenkwinkel zur Verfügung stehen:

```
...
    DPUDX8Joystick Joystick
    {
        Computer = {MAIN};
        JoystickID = 1;
        Index = 10;
    };
    DPUJoystickConsole Mockup
    {
        Computer = {MAIN};
        Index = 20;
        JoystickAMin = 0;
        JoystickAMax = 1;
        JoystickBMin = 0;
        JoystickBMax = -1;
    };
    Connections =
    {
        Joystick.x <-> Mockup.JoystickX_In,
        Joystick.y <-> Mockup.JoystickA_In,
        Joystick.y <-> Mockup.JoystickB_In,
        Joystick.Button4 <-> Mockup.GearUp_In,
        Joystick.Button3 <-> Mockup.GearDown_In
    };
...
```

Die Reihenfolge der Berechnung der DPUs *eines* Rechners wird über den Eintrag `Index=...`; der jeweiligen DPU festgelegt. Im Beispiel wird `Joystick` vor `Mockup` berechnet. Somit liegen an den Eingängen von `Mockup` die aktuellen Werte der Ausgänge des Joysticks im gleichen Rechentakt an. Die absoluten Werte des Index sind dabei nicht von Bedeutung.

Im `Connections` - Abschnitt werden die Ausgänge des Joysticks mit den entsprechenden Eingängen von `Mockup` verschaltet.
Durch die Verknüpfung

```
Joystick.Button1 <-> Mockup.GearUp_In
```

wird zum Beispiel festgelegt, dass mit dem `Button1` des Joysticks der nächst höhere Gang gewählt wird. Der Eingang `GearDown_In` wird mit dem `Button2` des Joysticks verknüpft. Liegen diese Tasten für den Fahrer ungünstig, so können die beiden Eingänge auch auf andere Tasten gelegt werden.

Die Fahrdynamik wird instanziiert und mit dem `Mockup` verknüpft:

```

...
    DPUMstVeda VehicleDynamics
    {
        Computer = {MAIN};
        Index = 30;
    };
    Connections =
    {
        Mockup.AcceleratorPedal <-> VehicleDynamics.AcceleratorPedal,
        Mockup.BrakePedal <-> VehicleDynamics.BrakePedal,
        Mockup.SteeringWheel <-> VehicleDynamics.SteeringWheel,
        Mockup.Gear <-> VehicleDynamics.Gear
    };
...

```

Sie berechnet u.a. die aktuelle Position und Orientierung des Fahrers im ortsfesten Koordinatensystem. Diese werden der Datenbasis (Instanz der `DPUSCNX`) mitgeteilt. Als Rückmeldung von der Datenbasis bekommt die Fahrdynamik die aktuelle Steigung der Straße sowie Positionsoffsets, die beim Aufsetzen auf die rechteste Spur eines Streckenabschnitts entstehen.

Die Instanz `SCN` der Datenbasis läuft geklont auf zwei Rechnern: Auf dem Hauptrechner `MAIN`, auf dem die Datenbasis u.a. zur Positionierung der anderen Verkehrsteilnehmer verwendet wird, und dem Grafikrechner `FRONT`, auf dem das Bild der Straße und der Landschaft aus der Datenbasis erzeugt wird. Das Klonen der Datenbasis minimiert den Kommunikationsaufwand innerhalb der Simulation, da zur Synchronisation der Klone nur die Position und die Orientierung des Fahrers übertragen werden müssen.

```

...
    DPUSCNX SCN
    {
        Computer = {MAIN,FRONT};
        Index = 40;
    };
    Connections =
    {
        VehicleDynamics.X <-> SCN.X,
        VehicleDynamics.Y <-> SCN.Y,
        VehicleDynamics.Z <-> SCN.Z,
        VehicleDynamics.yaw <-> SCN.yaw,
        VehicleDynamics.pitch <-> SCN.pitch,
        VehicleDynamics.roll <-> SCN.roll,
        SCN.PitchRoad <-> VehicleDynamics.PitchRoad,
        SCN.RollRoad <-> VehicleDynamics.RollRoad,
        SCN.XOffset <-> VehicleDynamics.XOffset,
        SCN.YOffset <-> VehicleDynamics.YOffset,
        SCN.ZOffset <-> VehicleDynamics.ZOffset
    };
...

```

Die Simulation des umgebenden Verkehrs besteht aus Instanzen der DPUs `DPUTRFX` und `DPUTRFClient`. `DPUTRFX` berechnet das Verhalten der einzelnen Verkehrsteilnehmer und setzt es in Kommandos wie „Beschleunigung auf XXX m/s² ändern“ und „aktuelle Spurabweichung auf XXX m ändern“ um. Da die `DPUTRFX` sehr rechenintensiv ist, wird auf den Grafikmaschinen nur die `DPUTRFClient` instanziiert. Sie enthält nur den Teil der

DPUTRFX, der die generierten Kommandos in tatsächliche Manöver eines Fahrzeugs im Straßennetzwerk umsetzt. Beim Zusammenspiel der beiden DPUs werden mittels DPU-Verknüpfungen nur Synchronisationsdaten (`TrafficData`) übertragen. Die restliche Kommunikation erfolgt intern.

```
...
    DPUTRFX TRFX
    {
        Computer = {MAIN};
        Index = 50;
    };
    DPUTRFClient TRFClient
    {
        Computer = {FRONT};
        Index = 50;
        ServerIP = "10.1.2.181";
    };
    Connections =
    {
        TRFX.Trafficdata <-> TRFClient.Trafficdata
    };
...

```

Die Visualisierung von Datenbasis und umgebendem Verkehr wird aus Instanzen von vier DPU-Typen aufgebaut:

- **DPUSGEWorld:** Verwaltet alle Objekte der Visualisierung (Verkehrsteilnehmer, Straßennetzwerk, Bäume, Schilder usw.). Die Kommunikation zwischen dieser DPU und den restlichen an der Visualisierung beteiligten DPUs erfolgt intern, also nicht über DPU-Verknüpfungen.
- **DPUSCNXSGE:** Erzeugt die grafische Repräsentation der Datenbasis.
- **DPUTRFSGE:** Erzeugt die grafische Repräsentation des umliegenden Verkehrs.
- **DPUSGEView:** Stellt die in `DPUSGEWorld` verwalteten Objekte in einem Ausgabefenster aus einer bestimmten Kameraperspektive dar. Die Kameraperspektive aus der Sicht des Fahrers wird aus Daten berechnet, welche die Fahrdynamik und die Datenbasis liefern.

```
...
    DPUSGEWorld World
    {
        Computer = {FRONT};
        Index = 60;
    };
    DPUSCNXSGE SCNXSGE
    {
        Computer = {FRONT};
        Index = 70;
    };
    DPUTRFSGE trfvis
    {
        Computer = {FRONT};
        Index = 80;
    };
    DPUSGEView View
    {
        Computer = {FRONT};
        Index = 90;
        WindowW = 800;
        WindowH = 600;
        WindowX = 0;
        WindowY = 0;
        ViewID = 1;
        DistortionMode = 0;
    };
    Connections =
    {
        VehicleDynamics.X <-> View.X,
        VehicleDynamics.Y <-> View.Y,
        VehicleDynamics.yaw <-> View.yaw,
        VehicleDynamics.pitch <-> View.pitch,
        VehicleDynamics.roll <-> View.roll,
        SCN.ZOut <-> View.Z,
        SCN.PitchRoad <-> View.pitchRoad,
        SCN.RollRoad <-> View.rollRoad
    };
...
```

Der Pool Grundkonfiguration ist damit fertig.

Der Pool Aufzeichnung soll den Pool Grundkonfiguration um eine Datenaufzeichnung erweitern. Dies ist schematisch in folgender Abbildung dargestellt:

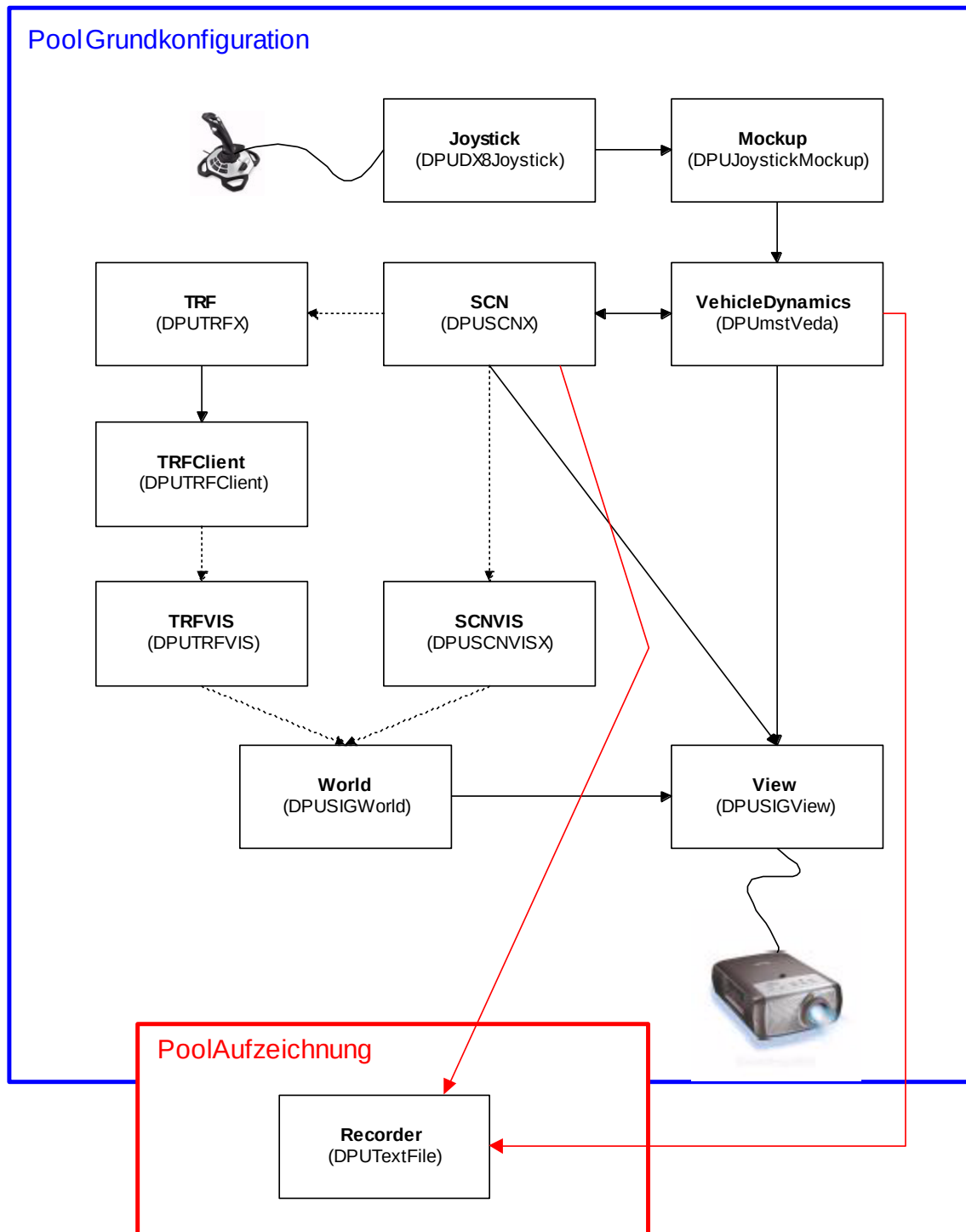


Abbildung 7: Der Pool Aufzeichnung (rot) ist vom Pool Grundkonfiguration (blau) abgeleitet.

Die Aufzeichnung der Daten übernimmt die DPUDataFile. Sie hat mehrere Eingangssignale, jedes davon entspricht einer Spalte in der Textdatei, die von ihr erzeugt wird. In jedem Simulationsschritt werden die aktuellen Werte der Eingangssig-

nale als Zeile in die Textdatei geschrieben (vgl. Dokument „DPUDataFile.pdf“). Der Name der Textdatei kann vom Benutzer vor jedem Start der Simulation eingegeben werden. Dies wurde im vorhergehenden Abschnitt über Launch-Daten vorbereitet. Im Beispiel sollen die aktuelle Geschwindigkeit des Fahrers, die aktuelle Längsbeschleunigung und die Abweichung von der Idealspur aufgezeichnet werden:

```
...
Pool Aufzeichnung : Grundkonfiguration
{
    Executable = true;

    DPUDataFile Recorder
    {
        Computer = {MAIN};
        Index = 100;

        Channels = 3;

        Record = {
            VehicleDynamics.v ("Geschwindigkeit"),
            VehicleDynamics.ax ("Längsbeschleunigung"),
            SCN.LateralDistance ("Lateraler_Abstand")
        };
    };
};
};
};
```

Dabei ist zu beachten, dass die DPUDataFile die benötigten Verknüpfungen automatisch selbst herstellt. Sie müssen lediglich in der „Records“ Liste die Variablennamen der aufzuzeichnenden Werte und in Klammern dahinter die Spaltenbezeichnung in der Textdatei angeben.

2.8 Multithreading

2.8.1 Überblick

In den letzten Jahren hat sich bei den PC-Prozessoren ein Wandel in der Entwicklung vollzogen. Statt immer höhere Taktraten zu erreichen, haben neue Generationen von Prozessoren inzwischen immer mehr Prozessorkerne. Somit können mehrere Aufgaben parallel ausgeführt werden, was als Symmetric multiprocessing (SMP) bezeichnet wird.

SILAB bietet mehrere Möglichkeiten, um von mehreren Prozessorkernen Gebrauch zu machen:

- Auf Grafikrechnern können einzelne Views der Visualisierung SGE in einem eigenen Thread ausgeführt werden (siehe Dok. „DPUSGEView“).
- Die Simulation von Fußgängern (MOV) kann seit Version 2.0 in einem eigenen Thread ausgeführt werden (siehe Dok. „DPUMOV“).

In beiden Fällen handelt es sich um Speziallösungen, bei denen nur einzelne Teile der von den beteiligten DPUs durchgeführten Berechnungen auf einen zusätzlichen Prozessorkern verlagert werden können.

In Version 2.5 wurde die Möglichkeit eingeführt, beliebige DPUs auf unterschiedlichen Threads auszuführen und dadurch mehrere Berechnungen gleichzeitig auszuführen.

Das bereits aus früheren Versionen bekannte Modell, in dem mehrere Computer parallel Simulationsschritte durchführen, wurde dadurch erweitert.

2.8.2 Definition von Threads und Zuweisung von DPUs

Die Definition der an der Simulation beteiligten Computer innerhalb des Computer-Pools wird erweitert:

Grammatik 12 Definition eines Computers mit mehreren Threads

<Computerinstanz> →

```
Computer <Name des Computers> | LOCALHOST
{
    IP = "<IP-Adresse des Computers>";
    PeriodMS = <Periode des Computers>; |
    Frequency = <Frequenz des Computers>;
    Operator = <Bool>;
    Alias = {(<Name>)};
    (0..*)<Threadinstanz>
};
```

<Threadinstanz> →

```
Thread <Name des Threads>
{
    Frequency = <Frequenz des Threads>;
    Alias = {(<Name>)};
};
```

Mit der Computerdefinition wird bereits der erste Thread („Haupt-Thread“) angelegt und dessen Simulationsfrequenz festgelegt. Die Simulationsfrequenz des „Haupt-Threads“ ist auch die Frequenz, mit der Daten von diesem Computer (über das Netzwerk) zu anderen an der Simulation beteiligten Rechnern übertragen werden.

Weitere Simulationsthreads können auf diesem Computer gestartet werden, indem Thread-Definitionen innerhalb der Computerdefinition eingetragen werden.

Für jeden zusätzlichen Thread kann also ebenfalls eine Simulationsfrequenz und weitere Aliasnamen festgelegt werden.

Abhängig davon, welcher Rechner (bzw. Alias) bei einer DPU-Definition im DPU-Pool angegeben wird, wird die DPU im jeweiligen Thread ausgeführt:

- Wird der „Name des Computers“ oder einer der Computer-Aliase angegeben, so wird die DPU im „Haupt-Thread“ ausgeführt.
- Wird der „Name des Threads“ oder einer der Thread-Aliase angegeben, so wird die DPU in diesem Thread ausgeführt.

Achtung: Für einen Computer und alle darauf laufenden Threads darf ein bestimmter Aliasname nur einmal verwendet werden. Eine DPU mit demselben Instanznamen kann nicht auf mehreren Threads desselben Computers gestartet werden (wohl aber auf mehreren Computern im ComputerPool).

2.8.3 Anforderungen und Einschränkungen

- Die Verknüpfung von Variablen zwischen DPUs, die in unterschiedlichen Threads laufen, ist beliebig möglich. Dabei muss aber folgendes beachtet werden:
 - Der Variablenaustausch zwischen einzelnen Threads wird analog zum Austausch zwischen den Computern synchronisiert. Das bedeutet, wenn eine DPU D1 in Thread T1 auf einen Satz Variablen einer anderen DPU D2 in Thread T2 zugreift, erhält sie während eines Simulationsschritts immer dieselben Werte, nämlich die, die im letzten vollständig abgeschlossenen Simulationsschritt von Thread T2 berechnet wurden.
 - Die Berechnungsreihenfolge der DPUs ist nur lokal, innerhalb jedes Threads durch den Index-Parameter festgelegt.
- Einige zentral für die Simulation wichtige DPUs müssen derzeit in demselben Thread ausgeführt werden. Dies gilt z.B. für alle DPUs von MOV, TRF, SCNX, SIG und SGE. Davon unberührt sind die in Kap. 2.8.1 genannten Möglichkeiten zu Views und Fußgängern.

3 STANDARD-KONFIGURATIONSARCHITEKTUR

3.1 Überblick und Verwendung

Im Lieferumfang von SILAB befindet sich die sog. Standard-Konfigurationsarchitektur, die Sie als Basis für Ihre eigenen, an Ihre Fragestellungen angepassten Konfigurationsdateien für die Fahrsimulation, verwenden sollten. In diesen Standard-Konfigurationsdateien wurden die im vorhergehenden Abschnitt beschriebenen Mechanismen konsequent ausgenutzt, so dass ihre Verwendung folgende Vorteile hat:

- Die Anpassung von SILAB an die hard- und softwaretechnischen Besonderheiten eines Fahrsimulators ist mit einem Minimum an Konfigurationsaufwand möglich.
- Konfigurationsdateien sind zwischen Fahrsimulatoren mit unterschiedlichen Hardware-Komponenten weitgehend kompatibel. Beispielsweise ist es leicht möglich, eine Konfigurationsdatei mit Szenarien, die Sie auf einem Arbeitsplatzrechner erstellt und getestet haben, in einem Fahrsimulator mit Mockup und mehreren Sichtkanälen zu fahren.
- Der Support, den wir bei Problemen mit Ihren Konfigurationsdateien leisten können, wird erheblich vereinfacht.

Im Folgenden wird eine Versuchsreihe, eine Studie oder auch ein kleines Experiment, das Sie mit dem Fahrsimulator durchführen möchten, allgemein als „Projekt“

bezeichnet. Gehen Sie wie folgt vor, um ein solches Projekt anzulegen und damit zu arbeiten:

Im Haupt-Installationsverzeichnis von SILAB (z.B. „C:\SILAB“ oder „D:\SILAB“) finden Sie einen Unterordner „Projects“. Jedes Ihrer Fahrsimulations-Projekte sollte in einem Unterordner dies „Projects“-Ordners gespeichert werden. Wenn Ihr Projekt also „XYZ“ heisst, dann befinden sich die zugehörigen Dateien (insbesondere Konfigurationsdateien) im Ordner „\SILAB\Projects\XYZ“. Innerhalb des Unterordners „XYZ“ hat sich folgende Struktur bewährt:

SILAB

Projects

XYZ

includes	Teile Ihrer Konfigurationsdateien, die in die Hauptdatei mittels include eingebunden werden (z.B. Konfigurationsdateien für Area2, die von SILABAEEdit erzeugt werden).
AEdit	Area2 Quelldateien (*.aed)
models	Projektspezifische, grafische Modelle (z.B. solche, die von SILABAEEdit erzeugt werden)
texture	Projektspezifische Texturen

Das Grundgerüst für Konfigurationsdateien von Projekt XYZ sieht so aus:

```
# System-Grundeinstellungen (Standard-Launchdatas, Watchdog)
include system/SYSBase.inc

# optionaler Abschnitt: Eigene Anpassungen an System-Einstellungen
SILAB System
{
    # ...
};

SILAB Configuration
{
    Computerconfiguration Rechner
    {
        # Computer-Konfiguration. Die bereitgestellten ComputerPools
        # sind vom jeweiligen Fahrsimulator abhängig. Zumindest die
        # Pools ‚Simulator‘ und ‚Workstation‘ (Einzelplatz) sind aber
        # immer vorhanden.
        include System/CompBase.inc
    };
    DPUConfiguration DPUs
    {
        # DPU-Konfiguration. Es wird der DPUPool ‚Full‘
        # bereitgestellt, von dem bei Bedarf eigene DPUPools
        # abgeleitet werden können.
        include System/DPUBase_SGE.inc

        # <Hier bei Bedarf eigene von ‚Full‘ abgeleitete DPUPools einfügen>
```

```

};

# Standard-Konfiguration von MOV.
include MOV/MOP.inc
# Standard-Konfiguration von SCNXSGE.
include scnx/SCNXSGE.cfg

SILAB TRF
{
    # Konfiguration von TRFX. Stellt einige grundlegende
    # Verhaltensmuster und Fahrzeuggruppen bereit.
    include SILAB_TRFX.cfg
};

SILAB SCN
{
    # Grundkonfiguration von SCNX.
    include scnx/SCNX.cfg

    # <Hier die Strecken- und Verkehrsdefinition einfügen.>
};

```

Bitte beachten Sie noch folgende Hinweise:

- Um das Projekt XYZ auf einen anderen Simulator zu übertragen, müssen Sie lediglich den Ordner XYZ an die entsprechende Stelle auf den Operator-Rechner des Simulators kopieren. Danach übertragen Sie seinen Inhalt mit dem Programm SILABAdmin an die restlichen Rechner des Simulators.
- In Ihren (vom Basis-Pool „Full“) abgeleiteten DPU-Pools definieren Sie üblicherweise folgende projektspezifischen Funktionalitäten:
 - Hardware-Anbindungen (z.B. physiologische Messwerterfassung, spezielle Eingabegeräte)
 - Softwarekomponenten (z.B. Nebenaufgaben, Fahrerassistenzsysteme)
 - Datenaufzeichnung (mit der DPUDataFile)
- Bitte verwenden Sie in Ihrem abgeleiteten DPU-Pool möglichst
 - die vordefinierten Rechnergruppen (s. Abschnitt 3.2).
 - die in Form von sog. Interfaces zur Verfügung gestellten Variablen (s. Abschnitt 3.3).
- Wenn Sie in Ihrem Projekt XYZ mehrere verschiedene Strecken haben (z.B. für verschiedene Teile einer Studie) aber immer die gleiche Computer- und DPU-Konfiguration, dann sollten Sie den blau markierten Teil in eine eigene Datei (z.B. mit Name „XYZ_config.inc“) im Unterordner „includes“ des Verzeichnisses „XYZ“ speichern. Die Konfigurationsdatei für eine bestimmte Strecke sieht dann so aus:

```

# Einbinden der Computer- und DPU-Konfiguration
include includes/XYZ_config.inc

# Standard-Konfiguration von MOV.
include MOV/MOP.inc
# Standard-Konfiguration von SCNXSGE.
include scnx/SCNXSGE.cfg

```

```

SILAB TRF
{
    # Konfiguration von TRFX. Stellt einige grundlegende
    # Verhaltensmuster und Fahrzeuggruppen bereit.
    include SILAB_TRFX.cfg
};

SILAB SCN
{
    # Grundkonfiguration von SCNX.
    include scnx/SCNX.cfg

    # <Hier die Strecken- und Verkehrsdefinition einfügen.>
};

```

3.2 Vordefinierte Rechnergruppen

Wenn Sie – was wir dringend empfehlen – die Standard-Konfigurationsarchitektur als Basis für Ihre eigenen Konfigurationen nehmen, dann sollten Sie Ihre DPUs nicht mehr auf physikalischen Rechnern (d.h. unter deren im Betriebssystem eingestellten Rechnernamen) instanzieren, sondern eine der folgenden Rechner-Gruppen verwenden:

Gruppe	Bezeichnung	Beschreibung
ALL	Alle	Alle Rechner des Simulators
ASS	Assistenzsysteme	Ein Rechner, der Assistenzsysteme simuliert (Tempomat etc.)
BEV	BirdsEyeViewer	Ein Rechner, auf dem eine schematische Vogelperspektive der aktuellen Strecke angezeigt wird
DATA	Datenaufzeichnung	Ein Rechner, auf dem Fahrdaten aufgezeichnet werden
DSP	Display	Ein Rechner, der zusätzliche Displays des Mockups ansteuert (z.B. für Navigation, HMI, Nebenaufgaben)
EYE	EyeTracking	Ein Rechner, der die Schnittstelle zu einem Eye Tracking System berechnet
EYEVIS	EyeTracking Visualization	Ein Rechner, auf dem die Blickrichtung in die 3D-Szenerie eingeblendet wird
MOCKUP	Mockup	Ein Rechner, der das Mockup ansteuert
MOTION	MotionSystem	Ein Rechner, der das Bewegungssystem steuert
MOV	MovingPeople	Ein Rechner, auf dem die Fußgängersimulation berechnet wird
OPERATOR	Operator	Ein Rechner, auf dem SILAB gestartet und der Workspace angezeigt wird
PPM	Physiologische Messungen	Ein Rechner, an den Puls-, EKG-, ... Messgeräte angeschlossen sind

RECORD	Video/ Replay-Recorder	Ein Rechner auf dem ein Online-Video im AVI oder Replay-Format aufgezeichnet wird
SCN	Datenbasis	Alle Rechner, auf denen die Datenbasis (SCNX) läuft
SND	Sound	Ein Rechner, auf dem die Soundsimulation berechnet wird
TRF	Traffic	Ein Rechner, auf dem die Verkehrssimulation berechnet wird
VDYN	VehicleDynamics	Ein Rechner, auf dem die Fahrdynamik des Egofahrzeugs berechnet wird
VIS	Visualization	Alle Rechner, die eine 3D Grafik berechnen

Welchen Gruppen ein bestimmter physikalischer Rechner zugehört, ist in der Datei „SILAB\lib\configurations\system\COMPBase.inc“ festgelegt. Diese Datei wird normalerweise bei der Installation von SILAB auf Ihrem Fahrsimulator von uns oder mit uns zusammen angepasst. Wenn Sie Ihren Fahrsimulator um einen zusätzlichen Rechner (z.B. einen weiteren Sichtkanal) erweitern, dann muss er in dieser Datei mit seiner entsprechenden Gruppenzugehörigkeit ergänzt werden.

3.3 Interfaces

Um Konfigurationsdateien möglichst unabhängig von den speziellen technischen Gegebenheiten eines Simulators zu machen, werden die DPUs, welche die Ansteuerung von Mockup und Fahrdynamik übernehmen, mithilfe sog. Interfaces abstrahiert. Ein Interface für das Mockup bietet beispielsweise eine einheitliche Schnittstelle mit Ausgängen für Lenkradwinkel, Gaspedalstellung usw., unabhängig davon, welche Art von Mockup-Hardware eingesetzt wird (z.B. reales Fahrzeug, Gaming-Controller, PC Tastatur). Interfaces sind Instanzen der DPUInterface. Folgende Interfaces werden von der Standard Konfigurationsarchitektur definiert:

Interface	Datei	Beschreibung
Mockup	DPUMockup.inc	Eingabelemente des Mockups
Display	DPUMockup.inc	Anzeigeelemente des Mockups oder MMIs
Buttons	DPUMockup.inc	Zusatztasten (z.B. am Lenkrad) des Mockups
PPM	DPUPPM.inc	Physio-/Psychologische Messwerte
VDyn	DPUVeda.inc	Original-Werte der Fahrdynamik (Egofahrzeug-Modell)
VDynMotion	DPUVeda.inc	Werte der Fahrdynamik, die an das Bewegungssystem weitergegeben werden ¹¹
VDynView	DPUVeda.inc	Werte der Fahrdynamik, die an die Ansichten weitergegeben werden ¹²

¹¹ Die Werte in VDynMotion sind normalerweise gleich denen in VDyn, manchmal müssen sie aber an die Hardwarebeschränkungen des Bewegungssystems angepasst werden.

¹² Die Werte in VDynView sind normalerweise gleich denen in VDyn, manchmal sind aber Anpassungen für ein bestimmtes Sichtsystem nötig.

Eine Liste mit den wichtigsten Variablen, die von diesen Interfaces und weiteren DPUs der Standard-Konfigurationsarchitektur zur Verfügung gestellt werden, finden Sie im *Handbuch SILAB Essential* im Abschnitt „Übersicht über DPUs und zugehörige Parameter“.

3.4 Dateien

Die Dateien, aus denen die Standard-Konfigurationsarchitektur besteht, befinden sich im Verzeichnis „SILAB\lib\configurations\system“. Diese Tabelle zeigt die wichtigsten davon:

Datei	Zweck	Art
COMPBase.inc	Basis Computerkonfiguration. Systemspezifisch.	ComputerPool
CONVeda.inc	Verbinden von Fahrdynamik und Mockup	Connections
DPUBase_SGE.inc	Basispool 'Full' für SCNX mit der Visualisierung SGE	DPUPool
DPUClock.inc	DPUClock	Sysunabh. DPU
DPUEnvironment.inc	Steuerung der Umweltbedingungen in der Simulation (DPUEnv-SCNX) für die Visualisierung SGE	Sysunabh. DPU
DPUEyeTracking.inc	EyeTracking DPUs	sysunabh. DPU
DPUHeli.inc	Einbindung Heli-Fahrdynamik statt mstVeda	sysunabh. DPU
DPUMakeAVI.inc	Erzeugung von Offline-Videos aus aufgezeichneten Fahrdaten	sysunabh. DPU
DPUMockup.inc	Interface für das Mockup mit Bedien- und Anzeigeelementen sowie Knöpfen	Interface
DPUMotion.inc	Interface für die Steuerung des Bewegungssystems	Interface
DPUMOV.inc	DPUs für Fußgängersimulation	Basis-DPU
DPUmstVeda.inc	Fahrdynamik	Basis-DPU
DPUPlayDriveData.inc	Replay von aufgezeichneten Fahrdaten	sysunabh. DPU
DPUPPM.inc	Interface für physio-/psychologische Messungen	Interface
DPURecordDriveData.inc	Aufzeichnung von Fahrdaten	sysunabh. DPU
DPUSCNX.inc	DPUSCNX	Basis-DPU
DPUScreenshotSeries.inc	Automatisches Erzeugen von Screenshots alle 15s während der Simulation	sysunabh. DPU
DPUSGE.inc	Basis-DPUs für Visualisierung SGE	Basis-DPU
DPUSGEVidGen.inc	Erzeugung von Online-Videos (live, während der Fahrt) mit der Visualisierung SIG	sysunabh. DPU

DPU SoundSim.inc	Soundsimulation	Basis-DPU
DPUTRFDDataFull.inc	Gibt Daten über umliegende Fahrzeuge als Variablen aus	sysunabh. DPU
DPUTRFX.inc	DPUs für Verkehrssimulation	Basis-DPU
DPUVeda.inc	Interface für die Fahrdynamik	Interface
DPUVDS.inc	Konfiguration der Fahrdynamik VDS	sysunabh. DPU
SYSBase.inc	Basis-Systemeinstellungen (Launchdata, Watchdog)	System-Abschnitt
SYSMockup.inc	Ansteuerung des Mockups	sysspez. DPU
SYSMotion.inc	Ansteuerung des Bewegungssystems	sysspez. DPU
SYSMotionSensor.inc	Implementierung des Sensors für die tatsächliche Bewegung des Bewegungssystems	sysspez. DPU
SYSNav_SGE.inc	Einblenden von Navigationssymbolen über Hedgehogs (Nav – Image) für die Visualisierung SGE	sysspez. DPU
SYSNight_SGE.inc	Ansteuerung der Tageshelligkeit über Umweltparameter (TimeOfDay, TimeOfYear, Latitude) für die Visualisierung SGE	sysspez. DPU
SYSRecordOptions.inc	Systemspezifische Parameter für die Replayaufzeichnung und Videoaufzeichnung	DPU-Parameter
SYSReplayOptions.inc	Systemspezifische Parameter für SILAB-Replay-Videos	DPU-Parameter
SYSSGEPARAMS.inc	Systemspezifische Parameter für die Visualisierung SGE	DPU-Parameter
SYSVeda.inc	Implementierung der Fahrdynamik mstVeda	sysspez. DPU
SYSVDS.inc	Implementierung der Fahrdynamik VDS	sysspez. DPU
SYSVideoOptions.inc	Systemspezifische Parameter für AVI-Videos	DPU-Parameter
SYSWeather_SGE.inc	Ansteuerung des Wetters über Umweltparameter (Precipitation, Thunderstorm) für die Visualisierung SGE	sysspez. DPU

Von diesen Dateien sind alle, die mit „DPU“ beginnen, systemunabhängig, d.h. sie können bei der Einrichtung eines neuen Simulators auf diesen kopiert werden und werden auch bei Updates von SILAB überschrieben.

Dateien, die mit „SYS“, „COMP“ oder „CON“ beginnen, sind systemabhängig und müssen bei Neuinstallationen oder Updates auf jedem Simulator gesondert angepasst werden. Für den einfachen Fall eines Einzelplatzsystems werden passende Dateien allerdings im SILAB-Installer mitgeliefert und können optional mit installiert

werden. Bei komplexeren Fahrsimulatoren übernehmen in der Regel wir die Anpassung dieser Dateien.

4 SYSTEMKONFIGURATION

Unabhängig von den in den vorhergehenden Abschnitten beschriebenen Konfigurationsdateien müssen einige sehr grundlegende Eigenschaften von SILAB an den verwendeten PC bzw. das Betriebssystem angepasst werden. Manche Konfigurationseigenschaften betreffen auch die zu SILAB gehörenden Hilfsprogramme (z.B. *SILABButler*, *SILABAEEdit*), die keine Konfigurationsdateien (.cfg-Dateien) einlesen.

Diese systemabhängigen Einstellungen werden in der Datei `sys.par` vorgenommen, die im SILAB-Binärverzeichnis liegen muss (d.h. das Verzeichnis, das auch `SILAB.exe` enthält).

Es handelt sich um eine Datei im INI-Format, die unterteilt ist in einzelne benannte Abschnitte (geschrieben als `[Abschnitt]`). Jeder Abschnitt kann beliebig viele Parameter-Wertezuweisungen enthalten (`Parameter=Wert`). Zeilen, denen ein Semikolon (;) vorangestellt wird, sind Kommentare.

Ein Beispiel:

```
[Abschnitt1]
Parameter1=Wert1
Parameter2=Wert2
; Dies ist ein Kommentar.
[Abschnitt2]
Parameter4=Wert7
```

Es folgt eine Beschreibung der einzelnen Abschnitte und der darin enthaltenen Parameter.

[System]

<i>Name</i>	Name des Simulators bzw. Systems, auf dem SILAB installiert ist. Der Name muss auf allen zum System zugehörigen Rechnern gleich gewählt werden. Er kann zum Einbinden systemabhängiger Konfigurationsabschnitte mit dem <i>includeat</i> -Mechanismus (siehe Kap. 2.2.3) verwendet werden.
<i>Priority</i>	Prozess-Priorität für die Ausführung von SILAB und SILABRemote auf diesem PC. Der Standardwert ist <code>Normal</code> . Alternativ können <code>High</code> oder <code>Highest</code> angegeben werden, um SILAB Vorrang vor anderen Prozessen zu gewähren, sowie <code>Low</code> oder <code>Lowest</code> , um den anderen Prozessen Vorrang zu gewähren.
<i>UMask</i>	Legt (unter Linux) fest, mit welchen Berechtigungen Dateien (z.B. Workspaces und Datenaufzeichnungen) erstellt werden. Standardwert: <i>(keine Angabe)</i> , d.h. die Systemeinstellung wird verwendet.

Beispiel: `UMask=000` lässt Zugriffe aller Benutzer auf die erstellten Dateien zu; `UMask=077` nur die des eigenen Benutzers.

[Path]

Install

Verzeichnis, in dem SILAB installiert ist. Von diesem Verzeichnis ausgehend werden alle benötigten Dateien (Konfigurations-Includes, grafische Objekte, Texturen, MMI-Bilder, etc.) gesucht.

Standardwert: `\SILAB\`

Normalerweise wird die Option vom Installer definiert und sollte nicht verändert werden, es sei denn, das SILAB-Verzeichnis wird umbenannt oder verschoben.

Als Spezialfall kann `Install=!` eingetragen werden, dann ist das Install-Verzeichnis gleich dem Verzeichnis, in dem die Programmdateien (*.exe) liegen.

[Ports]

Butler

Legt die verwendeten TCP-Ports für die Kommunikation zwischen SILAB, SILABAdmin und dem SILABButler fest. Es werden bis zu 20 aufsteigend nummerierte Ports, ausgehend von der angegebenen Zahl, benötigt.

Standard: 17100

Sys

Legt die verwendeten TCP-Ports für die Kommunikation zwischen SILAB und SILABRemote fest.

Es werden bis zu 20 aufsteigend nummerierte Ports, ausgehend von der angegebenen Zahl, benötigt.

Standard: 17120

DPU

Legt die verwendeten TCP-Ports für die Kommunikation zwischen den SILAB-DPUs (z.B. DPUTRFX, DPUMOV) auf deren Server- und Clientrechnern fest.

Es werden bis zu 20 aufsteigend nummerierte Ports, ausgehend von der angegebenen Zahl, benötigt.

Standard: 17140

Normalerweise werden die *Ports* vom SILAB-Installer korrekt festgelegt. Wenn mehrere Versionen von SILAB parallel installiert sind und gleichzeitig lauffähig sein sollen (d.h. mehrere Instanzen von SILABButler sind gestartet), muss darauf geachtet werden, dass ihre Port-Bereiche sich nicht überschneiden.

Außerdem dürfen die angegebenen Ports nicht von anderen Applikationen belegt sein und müssen in einer evtl. installierten Firewall freigeschaltet werden.

[Option]

Allgemeine Programmeinstellungen:

Mode

Legt fest, welche Möglichkeiten in der Benutzeroberfläche von *SILAB* angeboten werden.

0 (Standard-„Forschungsoberfläche“): Beliebige Konfigurationsdateien, Workspaces und DPUs können geladen werden. Der System-Explorer wird angezeigt.

1 (Eingeschränkte „Forschungsoberfläche“): Nur Konfigurationsdateien und Workspaces können geladen werden. Der System-Explorer wird angezeigt.

2 (Eingeschränkte „Trainings-Oberfläche“): Nur Konfigurationsdateien und Workspaces können geladen werden. Der System-Explorer wird *nicht* angezeigt.

<i>Minimize</i>	Legt fest, ob <i>SILABButler</i> und <i>SILABRemote</i> minimiert (1) oder als normales Fenster (0) gestartet werden. Standard (falls nicht angegeben): <i>SILABRemote</i> wird als normales Fenster, <i>SILABButler</i> minimiert gestartet.
<i>Language</i>	Gibt an, in welcher Sprache Menüoptionen und Logmeldungen erscheinen sollen. Eine entsprechende Sprachdatei muss im Verzeichnis <code>\SILAB\lib\messages</code> vorhanden sein. Standard (falls nicht angegeben): Alle Texte sind in deutscher Sprache. Beispiel: <code>Language=en</code> für eine englischsprachige Benutzeroberfläche.

Einstellungen für die Visualisierung SGE:

Seit SILAB 7.0 befinden sich die Einstellungen für die Visualisierung SGE in der separaten Konfigurationsdatei `lib/configurations/SGE/SGE_700.cfg`. Bitte lesen Sie hierzu das Kapitel 2.2.1 in der „Referenz Visualisierung SGE“.

[Region]

<i>Name</i>	Name des aktivierten Regionalverzeichnisses. Falls ein Regionalverzeichnis eingetragen ist, wird immer zunächst <code>SILAB\lib\configurations_Name</code> durchsucht, bevor das Standardverzeichnis <code>SILAB\lib\configurations</code> benutzt wird.
<i>ReplaceMode</i>	Gibt an, ob bestimmte Ressourcen nur aus dem Regionalverzeichnis, aber nicht aus dem Standardverzeichnis geladen werden dürfen (<i>ReplaceMode</i> = 1) oder ob immer beide durchsucht werden (<i>ReplaceMode</i> = 0, Standardeinstellung).
<i>DriveRight</i>	Gibt an, ob in der aktivierten Region rechts (<i>DriveRight</i> = 1, Standardeinstellung) oder links (0) gefahren wird.

Zusätzlich zur globalen Regionseinstellung in der Datei `sys.par` wird auch die Datei `region.par` im aktuellen Projektverzeichnis berücksichtigt. Die Einstellungen im Projektverzeichnis überschreiben die Einstellungen in `sys.par`.

Beispiel:

Folgende Datei `sys.par` enthält die Standardeinstellungen, die der Installer für SILAB Version 2.5 oder neuer vornimmt:

[Path]

; Das Installationsverzeichnis wird abhängig davon eingestellt,
; was der Benutzer bei der Installation angegeben hat.

Install=\SILAB\

[Ports]

Butler=22500

Sys=22520

DPU=22540

5 WEITERE ELEMENTE DER BENUTZEROBERFLÄCHE VON SILAB

Folgende grundlegende Informationen über die Benutzeroberfläche von SILAB finden Sie im „Handbuch SILAB Essential“:

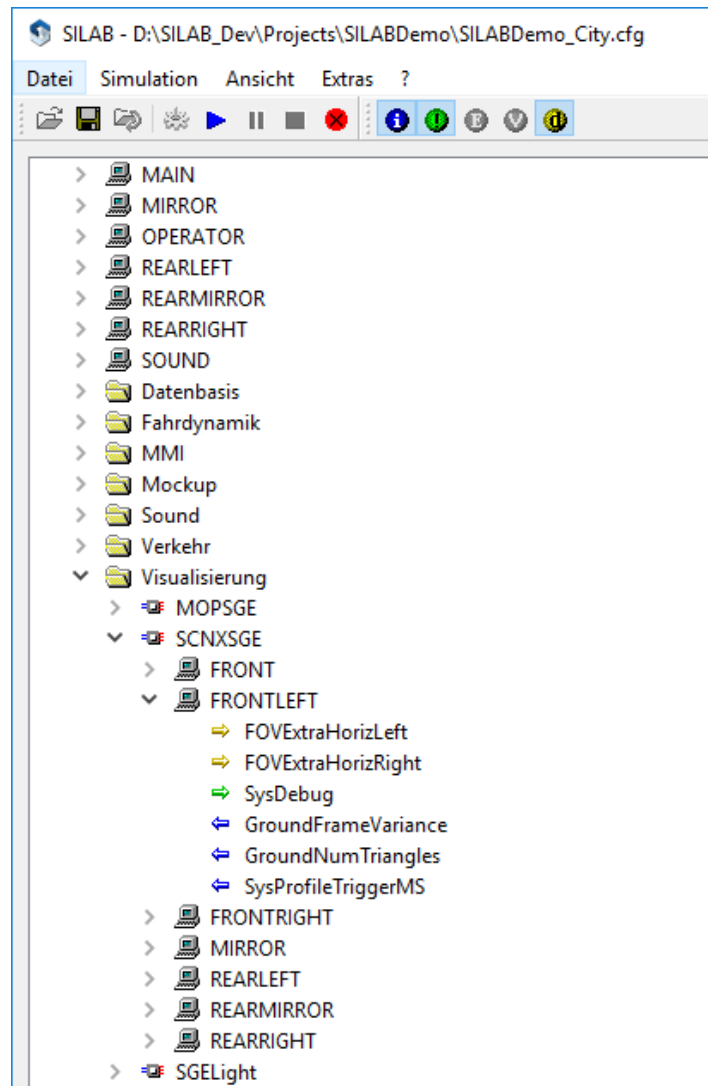
- Steuerung des Simulationsablaufs
(beinhaltet das Öffnen von Konfigurationsdateien)
- Überwachung von Parametern
- Abspeichern der ausgewählten Parameter als Arbeitsbereich

Dieses Kapitel geht auf einige dieser Punkte etwas detaillierter ein. Es beschreibt zudem weitere Möglichkeiten, mit der Benutzeroberfläche zu arbeiten.

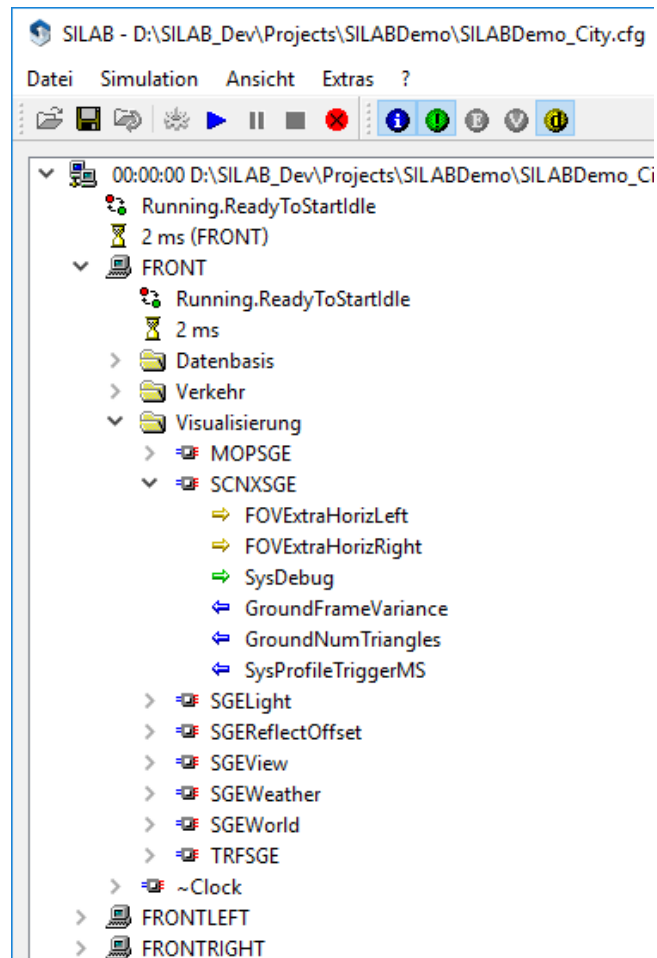
5.1 Anpassen der Baumansicht

Seit SILAB 4.0 kann die Baumansicht von SILAB, in der DPUs und deren Variablen angezeigt werden, mit verschiedenen Ansichtsmodi dargestellt werden, wodurch die Darstellung übersichtlicher wird und an die Nutzerwünsche angepasst werden kann. Die Optionen können über das Menü *Ansicht* ausgewählt werden. Folgende Möglichkeiten sind vorhanden:

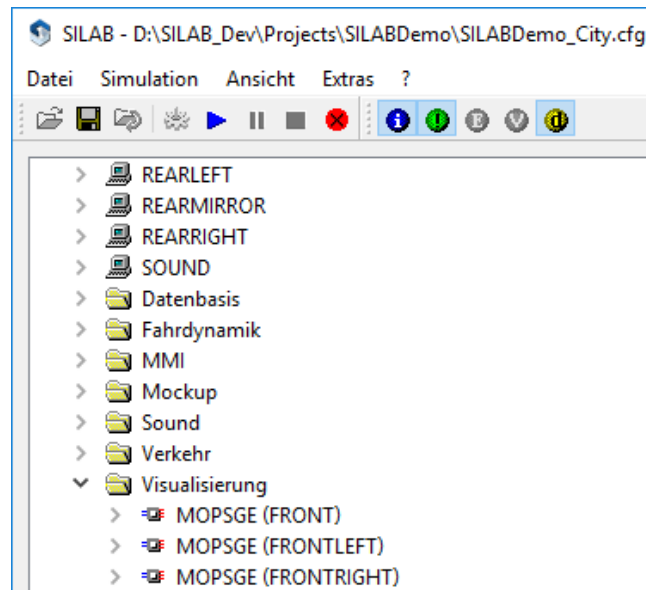
- **Gruppierung nach DPU-Name:**
Hiermit werden alle DPUs mit gleichem Namen zu eine Gruppe zusammengefasst. Unterhalb einer Gruppe „DPU-Name“ folgt die Auflistung aller Computer, auf denen die DPU läuft. Dies ist besonders hilfreich bei Simulatoren mit vielen Bildkanälen, da viele SILAB-DPUs auf jedem Bildkanal laufen.



- **Gruppierung nach Computer:**
Alle DPUs werden unter dem Computer eingeordnet, auf dem sie laufen.



- Versteckte DPUs (~) anzeigen:**
 DPUs, deren Name mit einer Tilde (~) beginnt, sind üblicherweise simulator-spezifisch und ihre Ein- und Ausgänge haben möglicherweise keine standardisierten Einheiten. Daher sollte im normalen Gebrauch nicht darauf zugegriffen werden und diese DPUs werden nun standardmäßig nicht mehr angezeigt. Über die entsprechende Menüoption können sie aber angezeigt werden, z.B. wenn Änderungen an der Simulatorkonfiguration vorgenommen werden.
- Gruppierung nach Aufgabenbereichen:**
 Die Gliederung in der Baumansicht wird standardmäßig um eine zusätzliche Ebene, den Aufgabenbereich, erweitert. Der Aufgabenbereich wird für die DPUs in der Konfigurationsdatei mit dem Parameter `TreeviewGroup = "XYZ"`; festgelegt. In der SILAB-Standardkonfiguration sind die Aufgabenbereiche *Datenbasis*, *Fahrdynamik*, *Mockup*, *Verkehr* und *Visualisierung* vordefiniert. Die Aufgabenbereiche werden in der Baumansicht durch Ordnersymbole dargestellt:



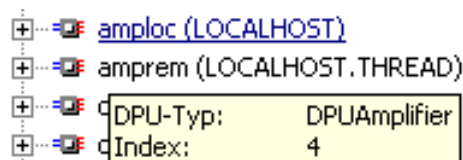
Die Einstellungen der Baumansicht werden im Workspace mit abgespeichert.

5.2 Tooltips in der Baumansicht

Zu DPUs und Variablen werden zusätzliche Informationen in einem Tooltip angezeigt, wenn Sie mit der Maus in der Baumansicht länger über einer solchen verweilen.

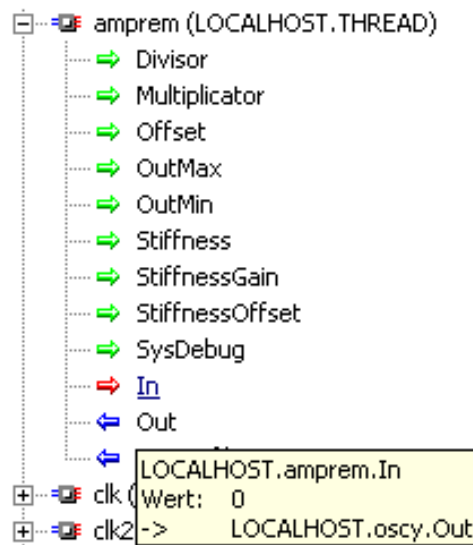
Für DPUs werden folgende Informationen angezeigt:

- Der Typ der DPU (im Beispiel „DPUAmplifier“)
- Der Index-Wert der DPU (dieser legt die Berechnungsreihenfolge der DPUs auf einem Computer fest).



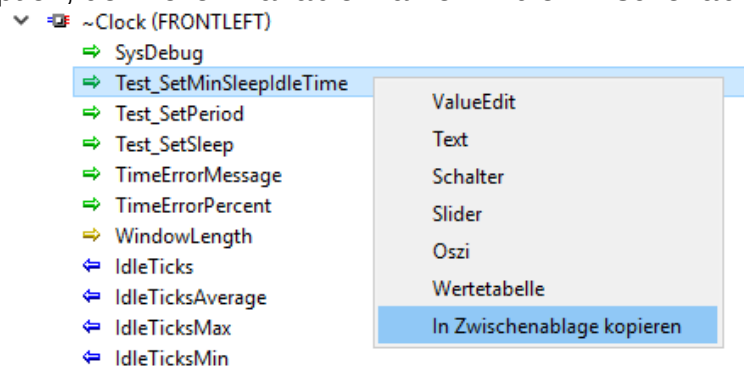
Für Variablen werden folgende Informationen angezeigt:

- Der volle Name der Variablen (inklusive Rechner- und DPU-Instanzname)
- Der aktuelle Wert der Variablen (nur für lokale Variablen und für solche, für die ein Display geöffnet ist)
- Alle Verknüpfungen der Variablen, d.h. die Quelle (->), falls diese Variable Ziel einer Verknüpfung ist, oder die Ziele (<-), falls diese Variable Quelle von Verknüpfungen ist.



5.3 Zwischenablage

Mit einem Rechtsklick auf eine Variable wird ein Kontextmenü geöffnet. Dieses enthält neben den jeweils möglichen Displays für die jeweilige Variable (siehe Abschnitt 5.4) auch die Option, den vollen Variablennamen in die Zwischenablage zu kopieren.



Dies bietet sich z.B. für die Datenaufzeichnung bei komplexen Variablennamen an, um Syntaxfehler zu vermeiden.

5.4 Displays

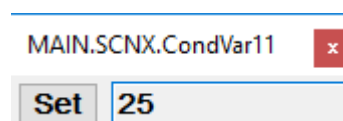
5.4.1 Displaytypen

Die Displays dienen der Überwachung von ebenso wie der Modifikation von Variablen der DPUs während der Simulation.

5.4.2 Display „ValueEdit“

5.4.2.1 Darstellung

Ein ValueEdit-Display wird folgendermaßen am Bildschirm dargestellt:



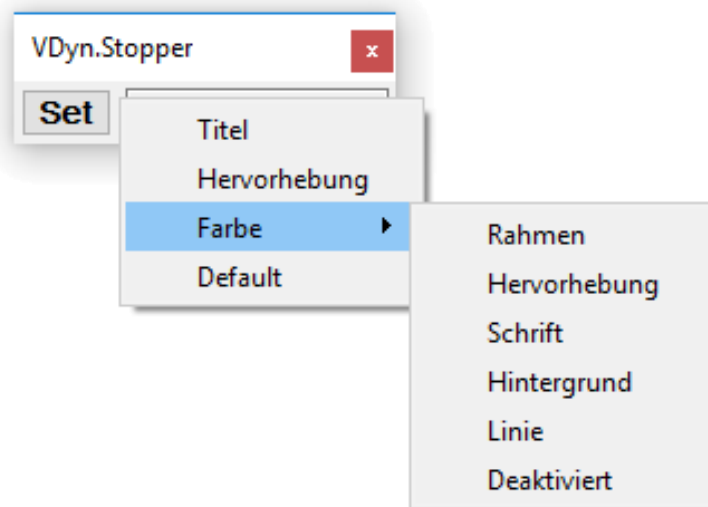
Ein ValueEdit-Display erlaubt es Ihnen, gezielt die Werte einzelner Variablen während der Simulation zu ändern. Beim Erzeugen eines solchen Displays wird der aktuelle Wert der entsprechenden Variablen geladen und in dem Edit-Feld dargestellt. Ein ValueEdit-Display wird bei jedem Start der Simulation mit dem Default-Wert der Variablen initialisiert. Entweder Sie haben diesen Default-Wert in der DPU-Konfiguration festgelegt oder es wird der vom Entwickler der DPU festgelegte Wert verwendet. Es ist deshalb nicht sinnvoll, einer Variablen bereits vor dem Start der Simulation einen Wert über das Edit-Feld zuzuweisen.

Wenn die Simulation läuft, können Sie einen neuen Wert in das Feld eintragen und diesen entweder durch Betätigen des Set-Buttons oder der Eingabetaste setzen. Die Eingabe ist auch in Exponentendarstellung möglich z.B. 1e2 anstatt des Wertes 100.

Zu beachten ist, dass jede DPU einen Bereich gültiger Werte für ihre Variablen festlegen kann. Dabei wird ein Minimal- und Maximalwert, der für die Variable zulässig ist, definiert. Der im ValueEdit eingegebene Wert wird ebenfalls auf diesen Gültigkeitsbereich eingeschränkt.

5.4.2.2 Optionen

Die Optionen eines Displays können über das Kontextmenü (rechte Maustaste) verändert werden. Bei einem ValueEdit-Display wird folgendes Menü angezeigt:



Sie haben die Möglichkeit,

- den Titel des Displays zu ändern
- die Farben aller Komponenten anzupassen
- einen Wertebereich zu definieren, in dem der Fensterrahmen seine Farbe wechselt (Hervorhebung)

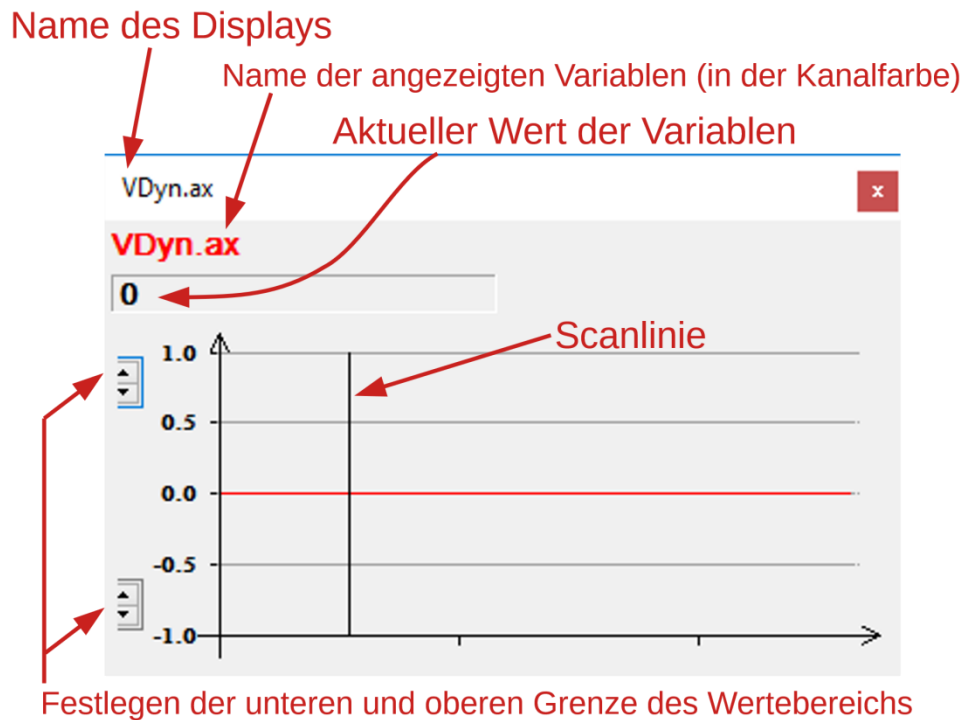
Der „Default“-Eintrag setzt die Farben auf die Standardwerte (Windows-Farben) zurück.

5.4.3 Display „Oszi“

Ein Oszi-Display dient zur Darstellung eines Wertes entlang der Zeitachse wie bei einem Oszilloskop.

5.4.3.1 Einfache Darstellung

Die Darstellung einer Variable in einem Oszi-Display sieht folgendermaßen aus:



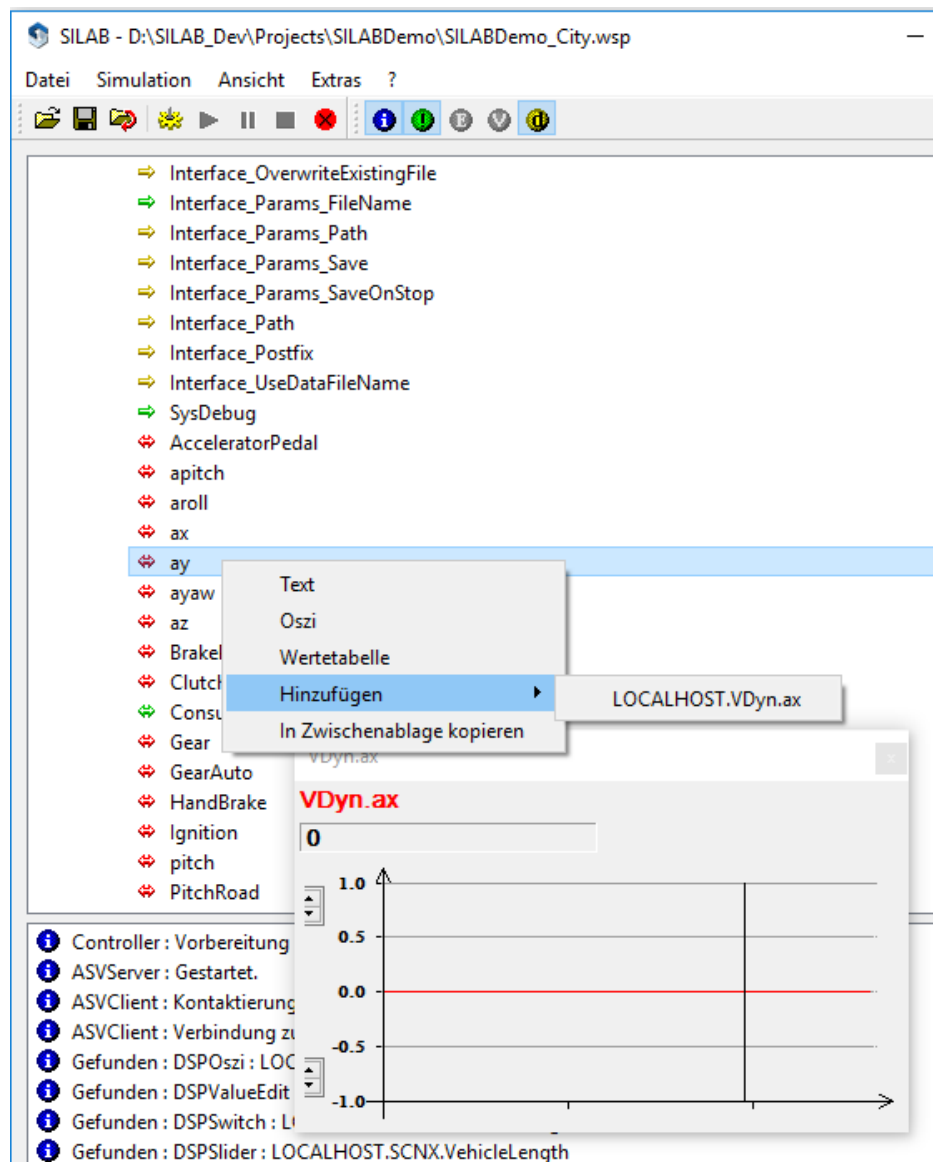
5.4.3.2 Mehrkanaldarstellung

Indem Sie andere Variablen in dasselbe Oszilloskop-Display einfügen, können Sie bis zu 8 Variablen in sog. Kanälen parallel darstellen.

Bei der Darstellung von mehreren Kanälen ist zu beachten, dass jeder Kanal seine eigene y-Achse besitzt. Links und rechts wird jeweils die Achse eines Kanals angezeigt. Im nächsten Abschnitt wird ausführlich erläutert, wie Sie einen bestimmten Kanal einer Achse zuordnen. Anhand der Farbe, mit der der Name einer Variablen dargestellt wird, können Sie die Kurven identifizieren.

Vorausgesetzt wird ein geöffneter Arbeitsbereich mit einem Oszi, in dem die Variable „VDyn.ax“ dargestellt wird.

Der zweite Kanal des Oszis soll die Variable „VDyn.ay“ darstellen. Die nachstehende Abbildung zeigt das Hinzufügen dieser Variablen als zweiten Kanal.



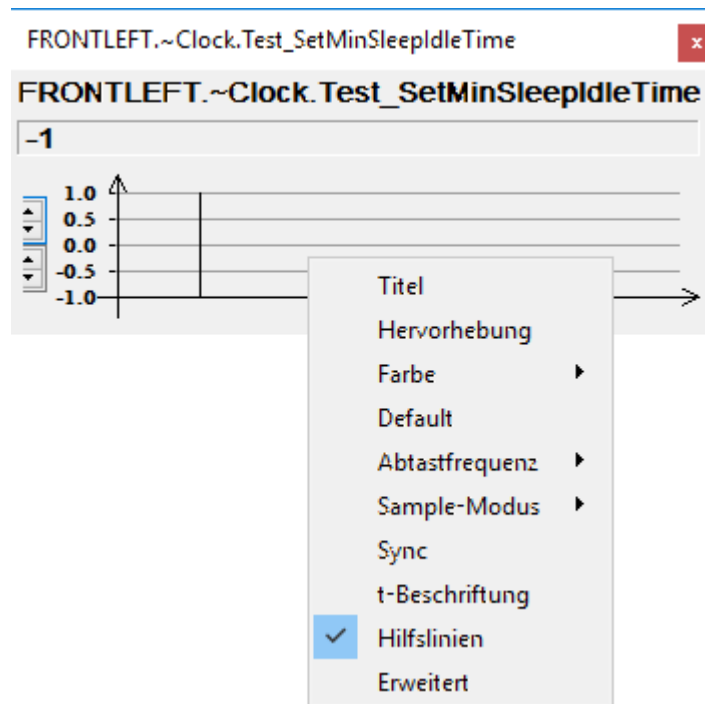
Weiterhin wird die Variable „VDyn.az“ als dritter Kanal in das Oszi eingefügt. Das endgültige Oszi sieht wie folgt aus:



Für die Variable „ax“, deren Name links steht, gilt auch die linke y-Achse, analog für die Variable „ay“ auf der rechten Seite. Die Werte der dritten Variable werden durch die blaue Kurve repräsentiert, deren Achse in dieser Ansicht nicht sichtbar ist. Im nächsten Abschnitt wird genauer erläutert, wie Sie die Achsen weiterer Kanäle betrachten und auch modifizieren können.

5.4.3.3 Optionen

Oszi-Displays bieten mehrere Optionen, die das Arbeiten komfortabler gestalten sollen. Über das Kontextmenü können Sie diese Optionen ändern:



Titel, Hervorhebung, Farbe, Default

Die Einstellmöglichkeiten in diesen Punkten entsprechen denen von ValueEdit-Displays.

Abtastfrequenz

Unter dem Menüpunkt „Abtastfrequenz“ können, ausgehend von der Frequenz des Operators, Variablen auch mit niedrigerer Frequenz dargestellt werden. Dadurch ist es möglich, die Variablen in der Zeit (x-Achse) kompakter darzustellen. Ein Oszi stellt normalerweise pro Pixel den Wert einer Variablen in einem Takt dar (Taktrate des Operators). Eine Änderung der Abtastfrequenz bewirkt, dass in ein Pixel eine entsprechend höhere Anzahl an Werten einfließt. Als mögliche Abtastfrequenzen werden immer die Faktoren 1, 1/2, 1/3, 1/4, 1/5 und 1/10 der Operatorfrequenz angeboten.

Die Art und Weise, wie diese Werte in den Graphen einfließen, richtet sich nach dem gewählten Sample-Modus.

Sample-Modus

Sample-Modus	Bedeutung
Min/Max	Es wird der kleinste und größte Wert der Variable aus den vergangenen Takten dargestellt. Daraus ergibt sich über die Zeit ein Band, welches die obere und untere Schranke der Werte darstellt.
Auslassen	Es wird immer der letzte Wert aller durchlaufenen Takte dargestellt. Alle übrigen Werte werden verworfen.
Mittelwert	Es wird der Mittelwert der Variable in den vergangenen Takten dargestellt.

Der Default-Modus ist dabei Min/Max. Bei maximaler Abtastfrequenz liefern jedoch alle drei Modi dieselben Resultate.

Sync

In der Praxis kommt es häufig vor, dass man mehrere Oszis untereinander platziert und die Werte der dargestellten Variablen vergleichen möchte. Eine Möglichkeit ist, ein Oszi zu erzeugen, welches alle Variablen enthält. Eine andere Möglichkeit ist die Funktion „Sync“, die alle dargestellten Displays synchronisiert. Das bedeutet, dass alle Oszis erneut an Position 0 mit der Darstellung beginnen.

t-Beschriftung

Die Zeitachse kann wahlweise in Millisekunden beschriftet werden. Jedes Pixel entspricht einem Takt des Operators bei voller Abtastrate. Durch Verändern der Taktrate wird auch die Zeitachse entsprechend angepasst.

Hilfslinien

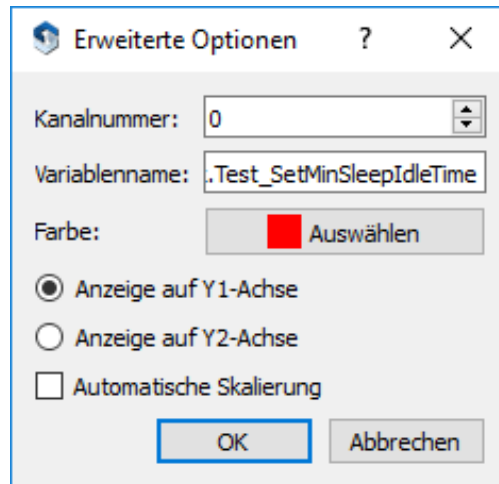
Um Ihnen eine Zuordnung der Punkte zu den Werten zu erleichtern, wird zu jeder Achsenbeschriftung auf der linken y-Achse eine entsprechende Gerade parallel zur x-Achse gezogen.

Y-Achsen vereinheitlichen

Die Y-Achsen aller dargestellten Kanäle werden vereinheitlicht. Dabei werden die Wertebereiche so festgelegt, dass sie alle bisher festgelegten Wertebereiche einschließen.

Erweitert

Die erweiterten Optionen bieten vor allem die Möglichkeit, mehrere dargestellte Kanäle zu steuern. Über den folgenden Dialog können Sie die erweiterten Optionen modifizieren:



Wählen im Tab den Namen des Kanals, den Sie verändern wollen.

Dieser Name kann im Feld „Variablenname“ verändert werden.

Für jeden Kanal kann eine Farbe ausgewählt werden mit der die Werte der Variablen dargestellt werden.

Die Auswahlfelder dienen dazu, festzulegen, welche Variable an welcher Achse angezeigt werden soll. Die „Y1-Achse“ stellt die linke y-Achse dar, während die „Y2-Achse“ die rechte y-Achse repräsentiert.

Achsen können automatisch skaliert werden. Das bedeutet, dass die Achsen automatisch einen größeren Bereich abdecken, sobald die Werte einer Variable den darstellbaren Bereich verlassen.

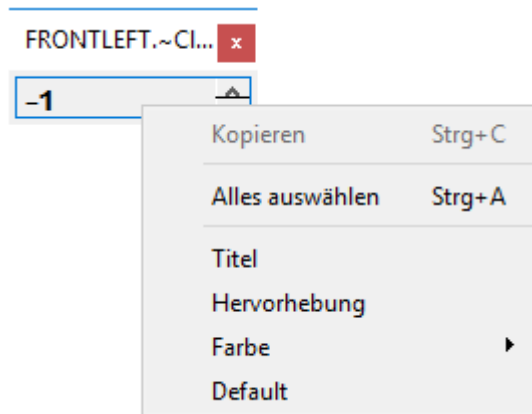
Standardmäßig ist diese Option deaktiviert. Ist eine Achse ohne automatischer Skalierung z.B. auf einen Bereich von –1 bis 1 eingestellt, so führt jeder Wert größer 1 zu einer Markierung auf der 1 und jeder Wert kleiner –1 zu einer Markierung bei –1.

Alternativ kann der Wertebereich der Y-Achse des Kanals auch manuell durch die Vorgabe des „Min“ und „Max“ Wertes festgelegt werden.

5.4.4 Display „Text“

Bei einem Text-Display wird der Wert einer Variablen wird als Fließkommazahl in dem Textfeld dargestellt.

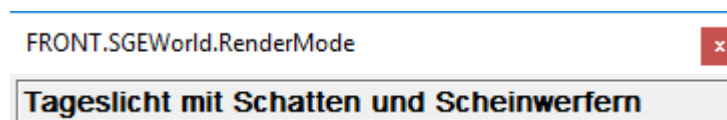
Mit der rechten Maustaste öffnen Sie das folgende Kontextmenü:



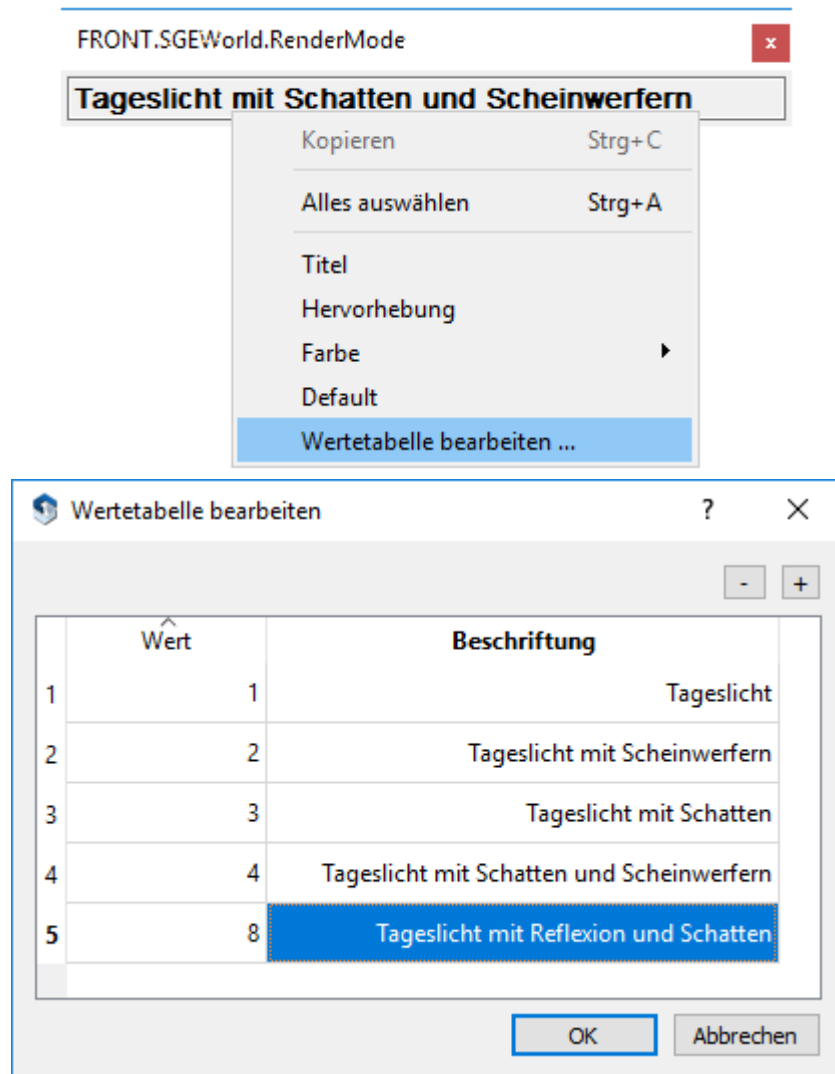
Die Einstellungsmöglichkeiten in diesen Punkten entsprechen denen von ValueEdit-Displays.

5.4.5 Display „Wertetabelle“

Bei einer Wertetabelle wird der Wert als textuelle Beschreibung im Textfeld dargestellt. Der numerische Wert der Variablen wird dazu in einer internen – im SILAB-Arbeitsbereich gespeicherten – Wertetabelle nachgeschlagen und die passende Beschreibung angezeigt.



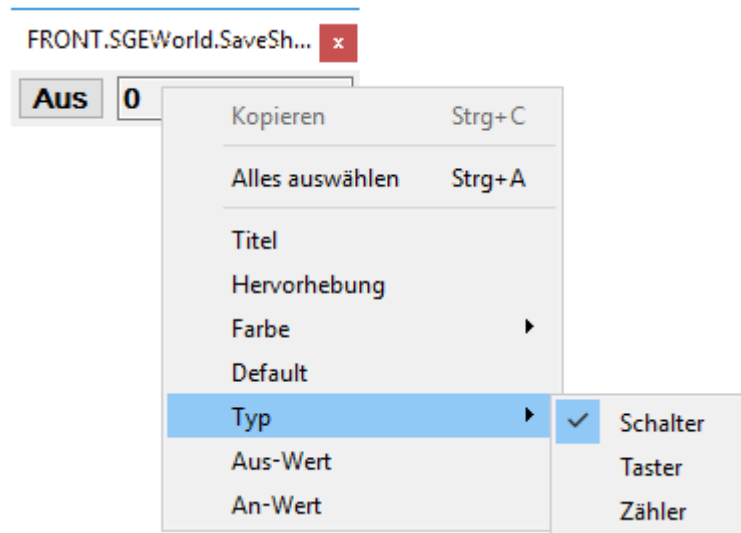
Mit der Kontextmenü-Option „Wertetabelle bearbeiten...“ können Sie neue Einträge in die Wertetabelle eintragen bzw. vorhandene bearbeiten.



Die weiteren Einstellmöglichkeiten entsprechen denen von ValueEdit-Displays.

5.4.6 Display „Schalter“

Ein Schalter-Display kann als Schalter, Taster und als Zähler verwendet werden. Dies legen Sie über das Kontextmenü fest:



Zunächst wird die Verwendung als Schalter (dies ist die Standardeinstellung) und als Taster erklärt und danach wird auf die Verwendung als Zähler eingegangen.

5.4.6.1 Displaytyp „Schalter“ als Schalter und Taster

Nach dem Öffnen einer Variablen wird der aktuelle Wert der Variablen im rechten Textfeld angezeigt.

Die Taste links dient dazu den Wert zwischen zwei Werten (zunächst 0 in „Off“ und 1 in „On“) umzuschalten. D.h. nach dem ersten Betätigen der Taste wird der Wert auf 1 gesetzt und der Schalter befindet sich im Zustand On:



Nach einem weiteren Druck auf die Taste befindet sich der Schalter wieder im Zustand Off und die Variable wird auf den Wert 0 gesetzt. Zu beachten ist, dass in dem Textfeld auf der rechten Seite immer der Wert, den die Variable angenommen hat, angezeigt wird und nicht unbedingt der, den Sie setzen wollten. Beispielsweise ist im Falle der Variable Divisor der DPUAmplifier der Wertebereich $] -0.1; 0.1[$ ausgenommen, d.h. die Variable wird im Off-Zustand den Wert 0.1 und nicht 0.0 aufweisen.

Mit der rechten Maustaste wird das folgende Kontextmenü geöffnet. Neben den bereits vorgestellten Optionen (siehe Beschreibung vorhergehender Displays) kann der On- und Off-Wert des Schalters frei eingestellt werden und der Typ des Schalter-Displays festgelegt werden.

Das Display verhält sich als „Taster“ genauso wie als Schalter, mit dem Unterschied, dass der Button immer wieder in den Zustand „Off“ zurückspringt, nachdem die Maus losgelassen wurde.

5.4.6.2 Displaytyp „Schalter“ als Zähler

Wenn Sie das Display als Zähler verwenden, geht der Button ebenfalls immer wieder in den Zustand „Off“, aber er zählt den Wert von 0 beginnend mit jedem Tastendruck

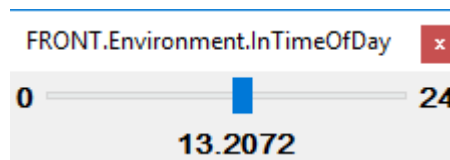
um eins hoch. Die eingegebenen Werte für „On“ und „Off“ werden bei einem Zähler nicht berücksichtigt.

5.4.7 Display „Slider“

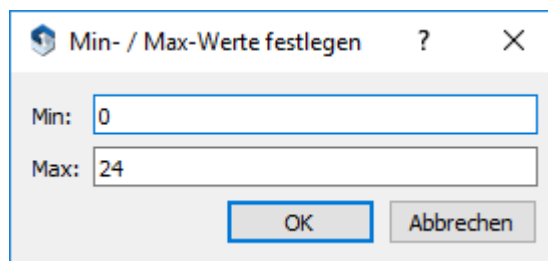
Ein „Slider“-Display wird als Schieberegler dargestellt und zeigt den aktuellen Wert einer Variable sowie den Minimal- und Maximalwert des Wertebereichs an.



Sie können einen „Slider“ als Eingabeelement (für Eingänge einer DPU) verwenden. Durch Ziehen mit der Maus am Slider verändern Sie den Wert der Variablen.



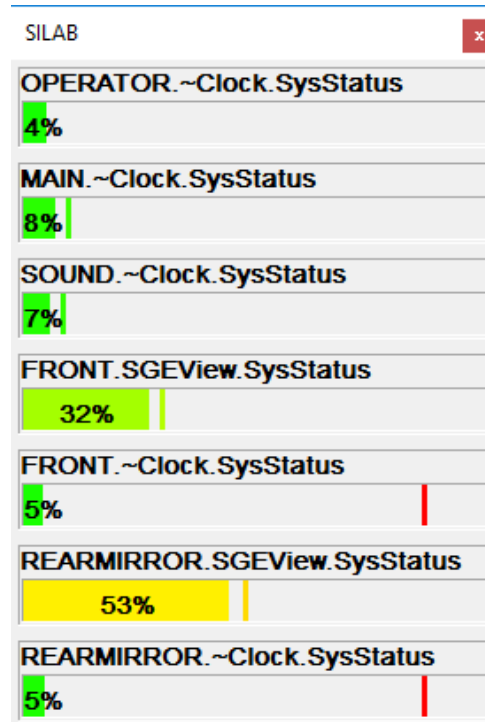
Damit der „Slider“ verwendet werden kann, müssen Sie zunächst den sinnvollen Wertebereich der Variablen einstellen. Dies geschieht durch den Dialog „Min/Max“, den Sie über das Kontextmenü erreichen.



Mit Hilfe eines Sliders kann die Variable nicht auf Werte außerhalb des eingestellten Wertebereichs gesetzt werden.

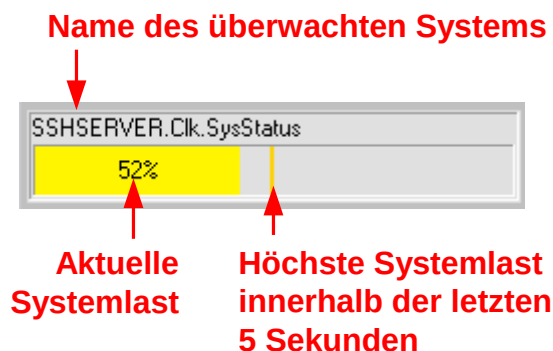
5.4.8 Display „Status“

Eine „Status“-Anzeige dient zur Überwachung der Rechenbelastung der Simulatorrechner. Im Unterschied zu den anderen Displays ist sie nicht einer bestimmten Variablen zugeordnet. Sie kann durch die Menüoption „Ansicht → Simulations-Status“ geöffnet werden. Es wird dann automatisch der Zustand aller Simulatorrechner dargestellt.



5.4.8.1 Grafische Darstellung

In jedem Feld einer „Status“-Anzeige wird die Systemlast eines Simulatorrechners¹³ dargestellt. Dabei wird der Name des Rechners, die aktuelle Systemlast und die höchste Last innerhalb der letzten 5 Sekunden dargestellt:



5.4.8.2 Datenquellen

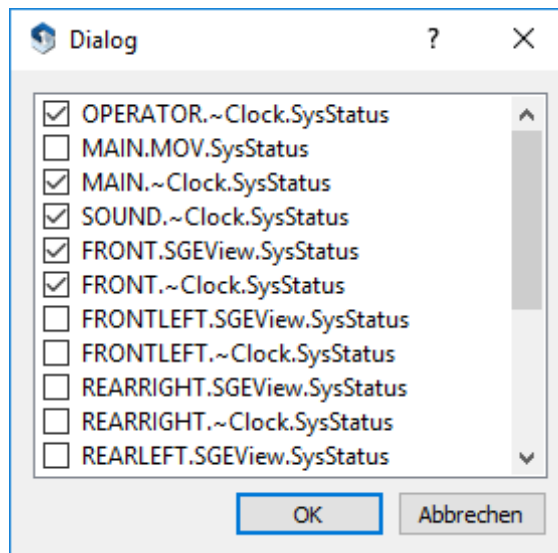
Eine „Status“-Anzeige überwacht die Auslastung jedes an der Simulation beteiligten Prozessors. Dazu werden die folgenden Datenquellen angezeigt:

- Die DPUClock (die auf jedem Simulatorrechner laufen sollte) überwacht den Zustand des Prozessors, auf dem die meisten DPUs laufen.
- Grafikansichten (DPUSIGView oder DPUSGEView) werden meist auf einem eigenen Prozessor ausgeführt (DPU-Parameter „Threaded=true“). Die Systemlast dieses Prozessors bzw. der angeschlossenen Grafikkarte wird dann ebenfalls von der „Status“-Anzeige überwacht.

¹³ bzw. eines seiner Prozessoren

- Es besteht zusätzlich die Möglichkeit, die Simulation von Fußgängern auf einem eigenen Prozessor durchzuführen (Parameter „Threaded=true“ der DPUMOV). Falls hiervon Gebrauch gemacht wird, kann die MOV-Systemlast ebenfalls von der „Status“-Anzeige überwacht werden.

Bei Bedarf können einzelne Datenquellen aus der Anzeige ausgeblendet werden. Klicken Sie dazu mit der rechten Maustaste auf das Status-Element und wählen Sie mit der Option „Sichtbare auswählen...“ in folgendem Dialog die Datenquellen aus, die angezeigt werden sollen.



5.4.8.3 Einsatzempfehlungen

Wir empfehlen, die Systemlast der Simulatorrechner mindestens bei den folgenden Gelegenheiten mit einer „Status“-Anzeige zu überprüfen:

- Wenn Sie eine **eigene DPU entwickelt** haben, sollten Sie überprüfen, dass diese nicht zu viel Rechenzeit beansprucht und dadurch evtl. die Simulation stört.
- Wenn neue **komplexe SILAB-DPU-Konfigurationen** mit einer Vielzahl von DPUs zusammengestellt werden (beispielsweise um ein neues Fahrer-Assistenzsystem zu simulieren).
- Wenn **komplexe, grafisch aufwändige Strecken** gestaltet wurden.

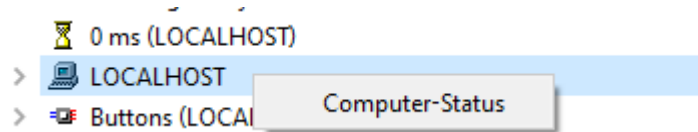
Wenn Sie mit Hilfe der „Status“-Anzeige eine Systemüberlastung in der DPUClock feststellen, sollten Sie versuchen, die Belastung des Systems zu verringern, indem Sie einige DPUs auf andere Simulatorrechner verlagern. Beachten Sie dabei bitte die Abhängigkeiten der einzelnen DPUs untereinander.

Wenn Sie eine Überlastung in der grafischen Ausgabe (DPUSGEView) feststellen, sollten Sie versuchen, die Grafikeinstellungen anzupassen (siehe „Referenz Visualisierung SGE“) oder die grafische Komplexität der Strecke zu reduzieren.

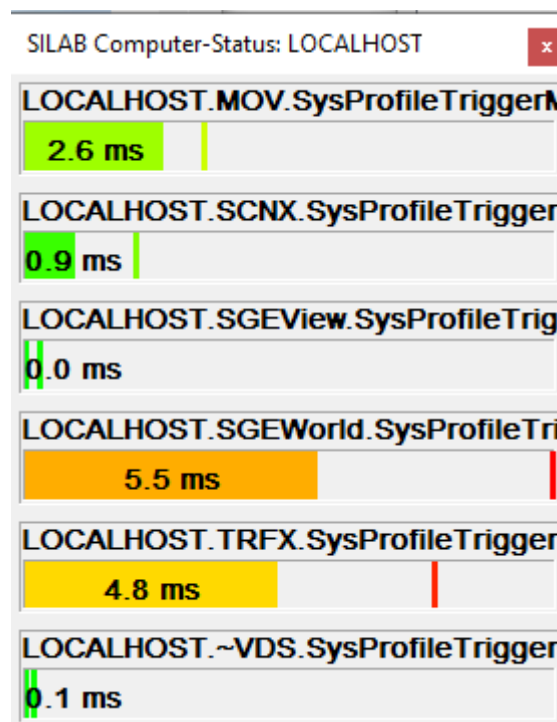
5.4.8.4 Weitere Details ermitteln

Wenn die Systemlast eines Simulatorrechners zu hoch ist, kann zusätzlich eine Analyse der Rechenzeit aller auf diesem Rechner ausgeführten DPUs durchgeführt werden.

Dazu können Sie die „Status“-Anzeige auf einem einzelnen Computer öffnen. Klicken Sie dazu mit der rechten Maustaste auf einen Computer und wählen Sie „Computer-Status anzeigen“:



Die „Status“-Anzeige stellt nun die von jeder DPU benötigte Rechenzeit, bezogen auf den Simulationstakt des Computers, dar:



Die weitere Bedienung der „Status“-Anzeige ist wie in Kap. 5.4.8.1 und 5.4.8.2 beschrieben.

5.5 Anzeige der Applikation auf Remote-Rechnern

Damit der Operator die Remote-Rechner steuern kann, muss auf diesen die Applikation SILABButler laufen. Der SILAB-Installer trägt sie als Autostart-Anwendung des Betriebssystems ein, so dass sie bei jedem Neustart automatisch aktiv ist.

Auf den Remote-Rechnern wird nach dem Start von SILAB lediglich das Meldungsfenster angezeigt, in dem entsprechende Informationen, Warnungen und Fehler ausgegeben werden.

6 SILABADMIN

6.1 Überblick

Mit dem Hilfsprogramm „SILABAdmin“ können Sie die an einer Simulation beteiligten Rechner (im Folgenden „Simulations-Rechner“ genannt) verwalten. Es bietet folgende Möglichkeiten:

- Simulations-Rechner einschalten
- Simulations-Rechner neu starten oder ausschalten
- Ein Unterverzeichnis des Ordners „SILAB“ des Rechners, auf dem SILABAdmin läuft, auf andere Rechner der Simulation übertragen.
- Einzelne Dateien aus dem Ordner „SILAB“ (oder einem Unterordner davon) des Rechners, auf dem SILABAdmin läuft, auf andere Rechner der Simulation übertragen.
- Abfrage einiger Systeminformationen von Simulations-Rechnern.

6.2 Konfiguration

Bevor Sie mit SILABAdmin arbeiten können, müssen Sie eine Konfigurationsdatei über den Menübefehl „*Datei/Öffnen*“ laden. Alternativ hierzu können Sie den Pfad zu dieser Konfigurationsdatei dem Programm SILABAdmin auch als Kommandozeilenparameter übergeben.

Eine Konfigurationsdatei ist eine Textdatei mit folgender Syntax:

```
SILABADMIN
<Anzahl N der aufgeführten Rechner, oder 0 = komplett laden>
<Name Rechner 1>
<IP-Adresse Rechner 1>
<MAC-Adresse Rechner 1>
.
.
.
<Name Rechner N>
<IP-Adresse Rechner N>
<MAC-Adresse Rechner N>
```

Leere Zeilen und Kommentarzeilen (die mit einer Raute # beginnen) werden dabei ignoriert.

Eine Konfigurationsdatei mit 8 Rechnern könnte also folgendermaßen aussehen:

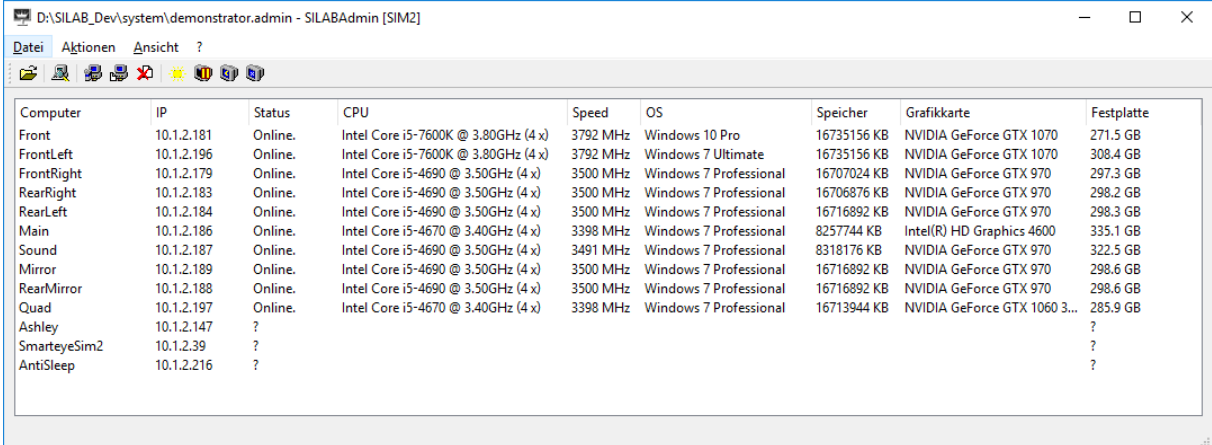
```
SILABADMIN
8
Main
10.1.2.107
00-11-2f-19-7b-11
Sound
10.1.2.108
00-0e-a6-d1-e7-19
FrontLeft
10.1.2.101
00-11-2f-03-4c-de
FrontMid
10.1.2.102
00-11-2f-03-4c-c3
FrontRight
10.1.2.103
00-11-2f-0a-f5-47
RearLeft
10.1.2.104
00-11-2f-0b-8a-b4
RearMid
10.1.2.105
00-11-2f-0b-8b-12
RearRight
10.1.2.106
00-11-2f-03-4c-14
```

ACHTUNG: Der Rechner, auf dem SILABAdmin ausgeführt wird (in den meisten Fällen handelt es sich hierbei um den Operator-Rechner der Simulation), darf in der Konfigurationsdatei nicht aufgeführt werden.

Die MAC-Adresse eines Rechners erhalten Sie unter Windows 7/10, wenn Sie im Start-Menü den Befehl „Ausführen...“ wählen, „cmd“ eingeben und dann „OK“ klicken. Im daraufhin erscheinenden Fenster geben Sie „*ipconfig /all*“ ein. Die MAC-Adresse wird unter „*physikalische Adresse*“ aufgeführt. Zur besseren Übersicht können innerhalb der MAC-Adresse Bindestriche „-“ oder Doppelpunkte „:“ zur Trennung der einzelnen Ziffernblöcke verwendet werden.

6.3 Hauptfenster

Nach dem erfolgreichen Laden einer Konfigurationsdatei wird folgendes oder – je nach Konfigurationsdatei – ein ähnliches Hauptfenster angezeigt:



The screenshot shows the SILABAdmin [SIM2] window with a menu bar (Datei, Aktionen, Ansicht, ?) and a toolbar. Below is a table listing computer configurations.

Computer	IP	Status	CPU	Speed	OS	Speicher	Grafikkarte	Festplatte
Front	10.1.2.181	Online.	Intel Core i5-7600K @ 3.80GHz (4 x)	3792 MHz	Windows 10 Pro	16735156 KB	NVIDIA GeForce GTX 1070	271.5 GB
FrontLeft	10.1.2.196	Online.	Intel Core i5-7600K @ 3.80GHz (4 x)	3792 MHz	Windows 7 Ultimate	16735156 KB	NVIDIA GeForce GTX 1070	308.4 GB
FrontRight	10.1.2.179	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16707024 KB	NVIDIA GeForce GTX 970	297.3 GB
RearRight	10.1.2.183	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16706876 KB	NVIDIA GeForce GTX 970	298.2 GB
RearLeft	10.1.2.184	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16716892 KB	NVIDIA GeForce GTX 970	298.3 GB
Main	10.1.2.186	Online.	Intel Core i5-4670 @ 3.40GHz (4 x)	3398 MHz	Windows 7 Professional	8257744 KB	Intel(R) HD Graphics 4600	335.1 GB
Sound	10.1.2.187	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3491 MHz	Windows 7 Professional	8318176 KB	NVIDIA GeForce GTX 970	322.5 GB
Mirror	10.1.2.189	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16716892 KB	NVIDIA GeForce GTX 970	298.6 GB
RearMirror	10.1.2.188	Online.	Intel Core i5-4690 @ 3.50GHz (4 x)	3500 MHz	Windows 7 Professional	16716892 KB	NVIDIA GeForce GTX 970	298.6 GB
Quad	10.1.2.197	Online.	Intel Core i5-4670 @ 3.40GHz (4 x)	3398 MHz	Windows 7 Professional	16713944 KB	NVIDIA GeForce GTX 1060 3...	285.9 GB
Ashley	10.1.2.147	?						?
SmarteyeSim2	10.1.2.39	?						?
AntiSleep	10.1.2.216	?						?

Abbildung 8: Hauptfenster von SILABAdmin.

Die Listenansicht zeigt die Rechner, die in der Konfigurationsdatei aufgeführt sind. Zu jedem Rechner werden in den Spalten Informationen angezeigt:

- **Computer:** Name des Rechners
- **IP:** IP-Adresse des Rechners
- **Status:** „Online.“ bedeutet, dass der Rechner für eine SILAB-Simulation einsatzbereit ist. „Reboot“ zeigt an, dass der Rechner über SILABAdmin neu gestartet wurde. Wird in dieser Spalte für einen Rechner keine Information angezeigt, kann das folgende Gründe haben:
 - Der Rechner ist nicht eingeschaltet.
 - Kein Benutzer ist angemeldet.
 - SILAB ist auf diesem Rechner unter dem angemeldeten Benutzer nicht ordnungsgemäß installiert. In diesem Fall muss das Setup-Programm von SILAB auf dem betroffenen Rechner neu ausgeführt werden.
 - Die in der Konfigurationsdatei angegebene IP-Adresse für diesen Rechner ist nicht korrekt.
- **CPU:** Informationen über den/die Prozessor(en) des Rechners. Je nach Betriebssystem des Rechners kann es vorkommen, dass in dieser Spalte nichts angezeigt wird. Ein Beispiel ist Abbildung 8.
- **Speed:** Geschwindigkeit des Prozessors/der Prozessoren.
- **OS:** Betriebssystem. Achtung: Bei neuen Betriebssystem-Versionen kann es manchmal zu einer fehlerhaften Darstellung kommen.
- **Speicher:** Größe des auf dem Rechner installierten physikalischen Speichers.
- **Grafikkarte:** Typ der verbauten Grafikkarte.
- **Festplatte:** Freier Speicher auf dem Festplattenlaufwerk, auf dem SILAB installiert ist.

6.4 Menübefehle

Alle Menübefehle beziehen sich auf Rechner, die in der Listenansicht selektiert wurden. Die Selektion eines einzelnen Rechners erfolgt über Anklicken des Rechnernamens. Mehrere Rechner werden durch Anklicken des ersten Rechnernamens und Anklicken der weiteren Rechnernamen bei gedrückter Strg-Taste selektiert.

6.4.1 Aktionen/System-Informationen ...

In einem Fenster werden detaillierte Informationen zum ersten der in der Listenansicht selektierten Rechner angezeigt.

6.4.2 Aktionen/Update Verzeichnis ...

Mit diesem Befehl können Sie einen Unterordner des SILAB-Verzeichnisses des Rechners, auf dem SILABAdmin ausgeführt wird, auf alle selektierten Rechner übertragen.

Nach der Anwahl des Menübefehls erscheint ein Dialog, in dem Sie das gewünschte SILAB-Unterverzeichnis auswählen. Anschließend wird abgefragt, ob die Unterordner des ausgewählten Ordners rekursiv ebenfalls übertragen werden sollen. Nach einer nochmaligen Bestätigung startet der Kopiervorgang.

Wurden beim Kopieren System-Dateien von SILAB übertragen (dies ist z.B. beim Kopieren des Ordners „SILAB\bin“ der Fall), muss SILABButler auf den selektierten Rechnern anschließend neu gestartet werden. SILABAdmin löst diese Aktion nach dem Abschluss des Kopiervorgangs automatisch aus.

ACHTUNG: Beim Übertragen von Dateien werden die auf den selektierten Rechnern bereits bestehenden überschrieben. Insbesondere bei der Übertragung der Ordner „SILAB\bin“ und „SILAB\lib“ kann die Stabilität und Funktionsweise von SILAB beeinträchtigt werden, wenn veraltete oder falsche Dateiversionen übertragen werden.

6.4.3 Aktionen/Update Dateien ...

Mit diesem Befehl werden einzelne Dateien aus Unterordnern des SILAB-Verzeichnisses des Rechners, auf dem SILABAdmin ausgeführt wird, auf alle selektierten Rechner übertragen.

Es erscheint zunächst ein Dialog, in dem Sie die Dateien auswählen, die übertragen werden sollen. Die Übertragung startet nach einer weiteren Bestätigung.

ACHTUNG: Beim Übertragen von Dateien werden die auf den selektierten Rechnern bereits bestehenden überschrieben. Insbesondere bei der Übertragung der Ordner „SILAB\bin“ und „SILAB\lib“ kann die Stabilität und Funktionsweise von SILAB beeinträchtigt werden, wenn veraltete oder falsche Dateiversionen übertragen werden.

6.4.4 Aktionen/Wake up ...

Die selektierten Rechner werden gestartet. Voraussetzung hierfür ist, dass die MAC-Adresse der selektierten Rechner in der Konfigurationsdatei korrekt ist und das „Wake On LAN“-Feature im Bios aktiviert ist.

6.4.5 Aktionen/Reboot ...

Bei den selektierten Rechnern wird ein Neustart ausgeführt. Voraussetzung hierfür ist, dass bei den Rechnern in der Spalte Status „Online.“ angezeigt wird.

6.4.6 Aktionen/Shutdown ...

Die selektierten Rechner werden heruntergefahren. Voraussetzung hierfür ist, dass bei den Rechnern in der Spalte Status „Online“ angezeigt wird.

7 SILABEXPLORER

7.1 Überblick

Mit dem Programm SILABExplorer ist es möglich, von einem beliebigen Rechner aus über TCP/IP eine Simulation mit SILAB fernzusteuern und zu überwachen. Ein typisches Anwendungsszenario sieht folgendermaßen aus: Sie möchten - während die Simulation läuft und während Sie selbst fahren - die Parameter einer bestimmten DPU anpassen. Dazu nehmen Sie ein Laptop, das mit dem Rechnernetz der Simulation verbunden ist, mit in das Mockup des Simulators. Mit SILABExplorer stellen Sie vom Laptop aus eine Verbindung zum Operator-Rechner der Simulation her. Sie öffnen einen Workspace und startet die Simulation. SILABExplorer zeigt Ihnen jetzt den aus SILAB bekannten Treeview mit allen Rechnern, DPUs und Variablen der Simulation. Sie öffnen Displays (Text, ValueEdit, Oszis und Schalter) und beobachten/manipulieren die Variablen, die Sie interessieren.

SILABExplorer bietet also folgende Möglichkeiten:

- Herstellen einer Verbindung zum Operator-Rechner einer SILAB-Simulation über das Netzwerk (TCP/IP).
- Laden eines Workspace auf dem Operator-Rechner.
- Start/Stop/Pause/Resume der Simulation.
- Beobachtung und Modifikation von DPU-Variablen. Die Frequenz, mit der eine Variable beobachtet wird (insbesondere bei Oszis-Displays) kann dabei gewählt werden.
- Beenden von SILAB.
- Mehrere SILABExplorer können sich (z.B. von verschiedenen Rechnern aus) gleichzeitig oder nacheinander mit dem Operator-Rechner verbinden.

7.2 Arbeiten mit SILABExplorer

Beim Start von SILABExplorer (`\SILAB\bin\SILABExplorer.exe`) erscheint folgende Benutzeroberfläche:

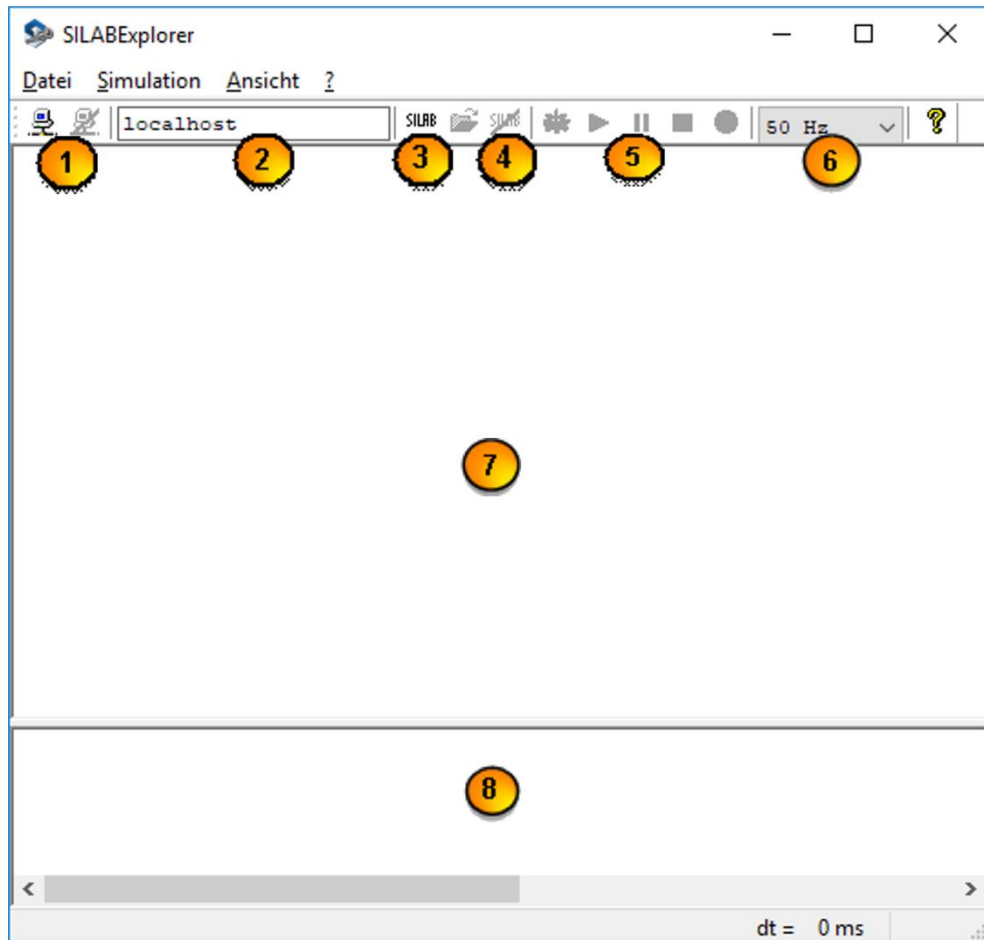


Abbildung 9: Benutzeroberfläche von SILABExplorer

Die Benutzeroberfläche besteht aus folgenden Komponenten:

- ① Schaltflächen zum Herstellen/Trennen einer Verbindung zum Operator-Rechner einer Simulation. Die IP-Adresse des Rechners muss unter ② eingetragen werden.
- ② IP-Adresse oder Rechnername des Operator-Rechners, zu dem Verbindung aufgenommen werden soll.
- ③ Schaltfläche zum Starten von SILAB. Bei bestehender Verbindung zu SILAB: Öffnet eine Konfigurationsdatei oder einen Workspace.
- ④ Bei bestehender Verbindung zu SILAB: Beendet SILAB.

- 5 Steuerung des Simulationsablaufs: Launch / Start / Pause / Stop / Shutdown der Simulation.
- 6 Frequenz, mit der Variablen im SILABExplorer (z.B. in einem Oszi) aktualisiert werden.
- 7 Sobald die Verbindung zu SILAB hergestellt ist und ein Workspace in SILAB geöffnet ist, erscheint hier der in Abschnitt 5.2 beschriebene Treeview.
- 8 Systemmeldungen.

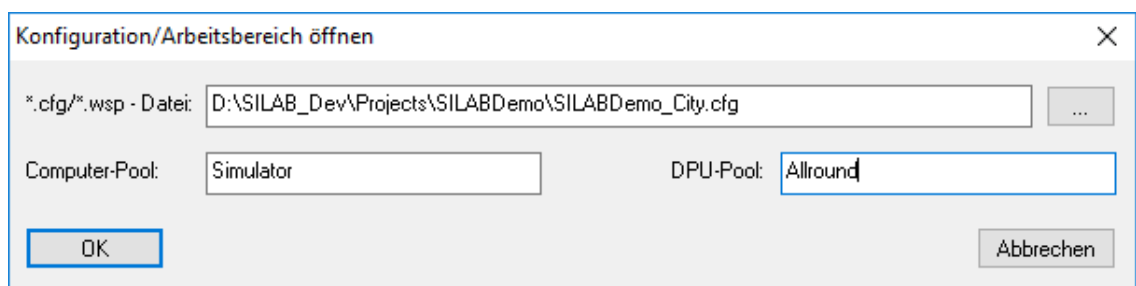
Es folgt eine Beschreibung typischer Arbeitsschritte mit SILAB-Explorer.

Ausgangssituation 1:

- SILAB ist auf dem Operator noch nicht gestartet. (Der SILABButler muss aber gestartet sein.)
- Es ist noch kein Workspace oder keine Konfigurationsdatei in SILAB geöffnet.

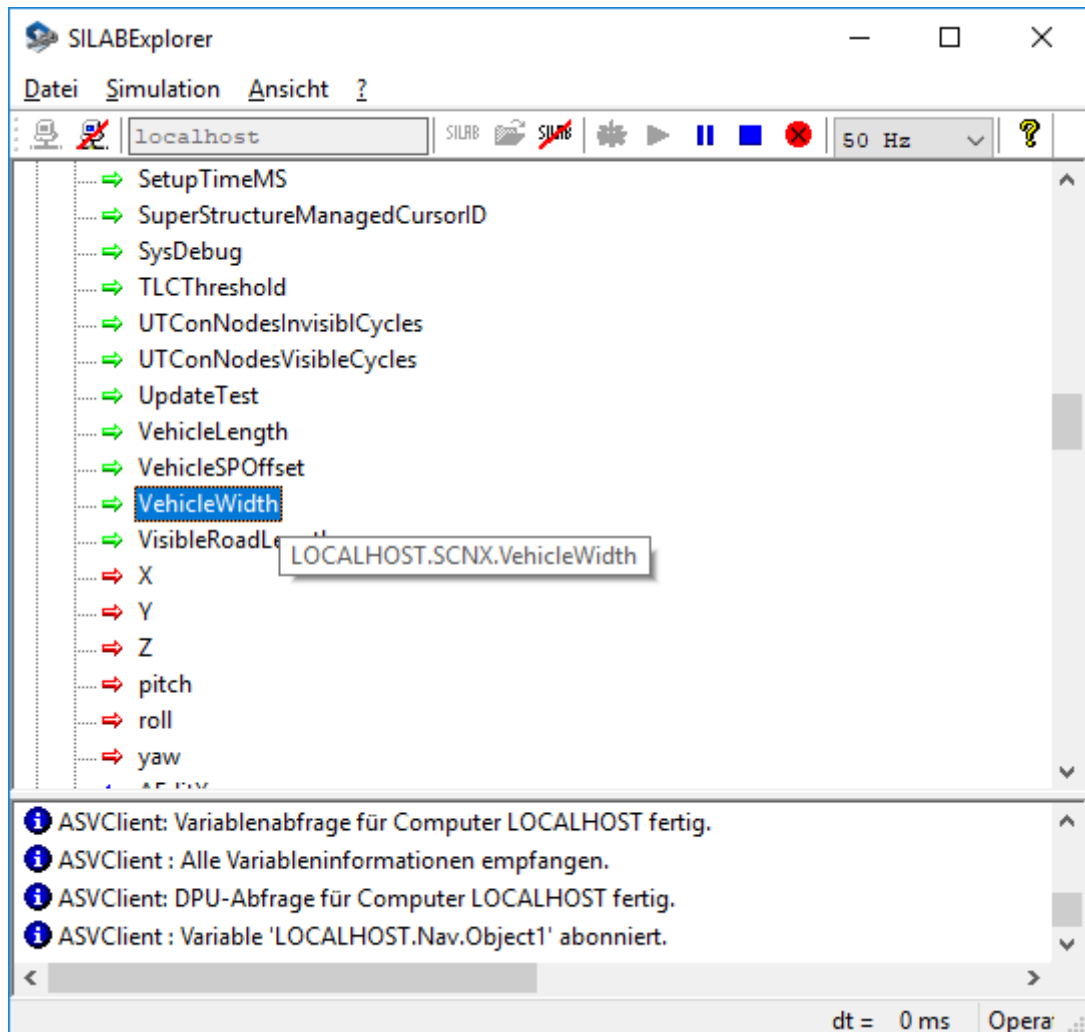
Arbeitsschritte:

1. Unter 2 wird die IP-Adresse des Operators eingetragen. Im Beispiel ist dies „10.1.2.201“.
2. Durch Klicken auf die linke Schaltfläche von 3 wird SILAB auf dem Operator-Rechner gestartet und eine Verbindung dazu hergestellt.
3. Sobald die Verbindung steht, kann mit 4 eine Konfigurationsdatei oder ein Workspace ausgewählt werden. Es erscheint folgender Dialog:



- Zu beachten ist, dass die in „*.cfg/*.wsp – Datei“ gewählte Datei im selben Pfad auf dem Operator existieren muss. Wenn eine *.cfg-Datei gewählt wurde, dann müssen noch Computer-Pool und DPU-Pool angegeben werden. Bei einer *.wsp-Datei ist dies nicht erforderlich.
4. Nach dem Klick auf OK in obigem Dialog öffnet SILABExplorer die gewählte Konfigurations- bzw. Workspace-Datei auf dem Operator-Rechner. Danach holt sich SILABExplorer alle Rechner-, DPU- und Variablen-Informationen und stellt sie in dem aus Abschnitt 5.2 bekannten Treeview dar.

5. Die Simulation kann jetzt über die Schaltflächen  gesteuert werden. Nach dem Klick auf „Launch“ () und „Start“ () zeigt SILABExplorer folgendes an:



6. Zum Beenden der Simulation klicken Sie auf die Stop-Schaltfläche in .
7. Optional kann SILAB auf dem Operator durch Klicken auf die Schaltfläche  geschlossen werden.

Ausgangssituation 2:

- SILAB ist auf dem Operator gestartet.
- Ein Workspace oder eine Konfigurationsdatei ist bereits in SILAB geöffnet.
- Die Simulation läuft noch nicht.

Arbeitsschritte:

1. Unter  wird die IP-Adresse des Operators eingetragen. Im Beispiel ist dies „10.1.2.201“.

2. Durch Klicken auf die linke Schaltfläche von  wird die Verbindung zum Operator-Rechner hergestellt.
3. Da bereits ein Workspace bzw. eine Konfigurationsdatei in SILAB geladen ist, wird nach dem Herstellen der Verbindung zum Operator-Rechner der Tree-view sofort mit den vorhandenen DPUs und Variablen gefüllt.

Die restliche Bedienung folgt der oben beschriebenen ab dem Arbeitsschritt 5.

Ausgangssituation 3:

- SILAB ist auf dem Operator gestartet.
- Ein Workspace oder eine Konfigurationsdatei ist bereits in SILAB geöffnet.
- Die Simulation läuft.

Arbeitsschritte:

1. Unter  wird die IP-Adresse des Operators eingetragen. Im Beispiel ist dies „10.1.2.201“.
2. Durch Klicken auf die linke Schaltfläche von  wird die Verbindung zum Operator-Rechner hergestellt.
3. Da bereits ein Workspace bzw. eine Konfigurationsdatei in SILAB geladen ist, wird nach dem Herstellen der Verbindung zum Operator-Rechner der Tree-view sofort mit den vorhandenen DPUs und Variablen gefüllt.
4. Die Schaltflächen  sind – je nach Zustand der laufenden Simulation (z.B. Pause) grau geschaltet.

Die restliche Bedienung folgt der in Ausgangssituation 1 beschriebenen ab dem Arbeitsschritt 5.